CS 182 - HW 2 - Jie Wen

1. Optimizers as Penalized Linear Improvement with different norm penalties

(a) Euclidean Distance Penalty

The objective is $u = \arg\min_{\Delta\theta} g^T \Delta\theta + \frac{1}{\alpha}\|\Delta\theta\|_2^2$.

Take the gradient with respect to $\Delta\theta$: $g + \frac{2}{\alpha}\Delta\theta = 0$.

Solving gives $\Delta\theta = -\frac{\alpha}{2}g$, so $u = -\frac{\alpha}{2}g$.

This recovers standard gradient descent with step size $\frac{\alpha}{2}$.

(b) Infinity Norm Penalty

Now use the infinity norm:

$$d(\Delta\theta) = \|\Delta\theta\|_\infty = \max_i |\Delta\theta_i|.$$

The objective becomes:

$$L(\Delta\theta) = g^T \Delta\theta + \frac{1}{\alpha}\|\Delta\theta\|_\infty.$$

Let:

$$\Delta\theta = -t \cdot \text{sign}(g), \quad t > 0,$$

where $\text{sign}(g)$ is the component-wise sign of g. Then:

- $g^T \Delta\theta = -t\|g\|_1$,
- $\|\Delta\theta\|_\infty = t$,

so the objective becomes:

$$L = -t\|g\|_1 + \frac{t}{\alpha} = t\left(\frac{1}{\alpha} - \|g\|_1\right).$$

If $\|g\|_1 > \frac{1}{\alpha}$, then $L \to -\infty$ as $t \to \infty$, and the problem is unbounded. To avoid this, consider the **constrained** version:

$$u = \underset{\Delta\theta}{\text{argmin}} \left\{g^T \Delta\theta\right\} \quad \text{subject to} \quad \|\Delta\theta\|_\infty \leq \delta.$$

The solution to this is:

$$\Delta\theta_i = -\delta \cdot \text{sign}(g_i) \quad \text{for all } i.$$

Letting $\delta = \alpha$:

$$u = -\alpha \cdot \text{sign}(g).$$

This corresponds to **signed gradient descent**, where each parameter is updated by a fixed amount in the direction of the negative gradient.

---

2. Optimizers and their convergence

(a) Vanilla SGD

The update rule for vanilla SGD is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha \nabla f(\boldsymbol{\theta}_t)$$

The gradient is:

$$\nabla f(\boldsymbol{\theta}) = -2(1 - \mathbf{x} \cdot \boldsymbol{\theta})\mathbf{x}$$

Starting from $\boldsymbol{\theta}_0 = \mathbf{0}$, gradient descent converges to the minimum norm solution:

$$\boldsymbol{\theta}^* = \frac{\mathbf{x}}{\|\mathbf{x}\|^2}$$

Compute:

$$\|\mathbf{x}\|^2 = 1^2 + 0.1^2 + 0.01^2 = 1.0101$$

Thus,

$$\boldsymbol{\theta}^* = \left(\frac{1}{1.0101}, \frac{0.1}{1.0101}, \frac{0.01}{1.0101} \right)$$

(b) Simplified Adam-Style Optimizer

The update rule is:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \alpha M_t \nabla f(\boldsymbol{\theta}_t)$$

Where M_t is a diagonal matrix with entries:

$$M_{ii} = \frac{1}{\sqrt{\mathbb{E}[g_i^2]}}$$

At $\boldsymbol{\theta} = \mathbf{0}$, the gradient is:

$$\nabla f(\mathbf{0}) = -2\mathbf{x} = [-2, \ -0.2, \ -0.02]$$

So the squared gradients are:

$$[4, \ 0.04, \ 0.0004]$$

Hence,

$$M = \text{diag}\left(\frac{1}{2}, \frac{1}{0.2}, \frac{1}{0.02} \right) = \text{diag}(0.5, \ 5, \ 50)$$

The update becomes:

$$\Delta \boldsymbol{\theta} = -\alpha M \nabla f = \alpha(1, \ 1, \ 1)$$

So all coordinates increase at the same rate. Let $\theta_1 = \theta_2 = \theta_3 = c$. Then:

$$\mathbf{x} \cdot \boldsymbol{\theta} = c(1 + 0.1 + 0.01) = 1.11c = 1 \quad \Rightarrow \quad c = \frac{1}{1.11}$$

Thus,

$$\boldsymbol{\theta}^* = \left(\frac{1}{1.11}, \frac{1}{1.11}, \frac{1}{1.11} \right)$$

(c) Feature Rescaling + SGD

Rescale each feature to have unit second moment:

- Feature 1: $1^2 = 1 \rightarrow$ scale factor = 1
- Feature 2: $0.1^2 = 0.01 \rightarrow$ scale factor = 10
- Feature 3: $0.01^2 = 0.0001 \rightarrow$ scale factor = 100

Rescaled input:

$$\mathbf{x}' = [1,\ 1,\ 1]$$

New model:

$$\hat{y} = \boldsymbol{\theta}' \cdot \mathbf{x}' = \theta'_1 + \theta'_2 + \theta'_3$$

SGD converges to the minimum norm solution:

$$\boldsymbol{\theta}'^* = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$$

Convert back to original coordinates:

$$\theta_i = \frac{\theta'_i}{\text{scale factor}_i}$$

So:

$$\theta_1 = \frac{1/3}{1} = \frac{1}{3}, \quad \theta_2 = \frac{1/3}{0.1} = \frac{10}{3}, \quad \theta_3 = \frac{1/3}{0.01} = \frac{100}{3}$$

Thus,

$$\boldsymbol{\theta}^* = \left(\frac{1}{3}, \frac{10}{3}, \frac{100}{3}\right)$$

---

3. Coding Question: Initialization and Optimizers

Based on the mean of gradient norm plot:

1. **He Initialization**: Shows relatively stable and moderate gradient norms throughout training. The gradients are well-preserved, allowing effective learning. This is because He initialization properly scales weights based on layer dimensions, maintaining gradient flow in deep networks.
2. **Random Initialization**: Exhibits very small gradient norms that remain low throughout training. This indicates vanishing gradients, where weights aren't updating effectively due to improper scaling. The small random weights cause activations to shrink layer by layer, resulting in minimal gradient signals.
3. **Zero Initialization**: Shows near-zero gradient norms from the start. This is expected since zero initialization breaks symmetry - all neurons in a layer learn the same features. The lack of diversity in gradients prevents meaningful learning, as all units update identically.

Explanation: Proper initialization is crucial for maintaining gradient flow in deep networks. He initialization preserves gradient magnitudes by scaling weights according to the network architecture (fan-in), while random and zero initializations fail to maintain gradient propagation, leading to poor learning dynamics. The gradient norms directly reflect these initialization effects on the training process.

---

4. Visualizing features from local linearization of neural nets

Visualizing features from local linearization of neural nets

```
!pip install ipympl torchviz
!pip install torch

import torch
import torch.nn as nn
import matplotlib.pyplot as plt
import numpy as np
import copy
import time
from torchvision.models.feature_extraction import
create_feature_extractor
from ipywidgets import fixed, interactive, widgets
%matplotlib inline

def to_torch(x):
    return torch.from_numpy(x).float()


def to_numpy(x):
    return x.detach().cpu().numpy()


def plot_data(X, y, X_test, y_test):
    clip_bound = 2.5
    plt.xlim(0, 1)
    plt.ylim(-clip_bound, clip_bound)
    plt.scatter(X[:, 0], y, c='darkorange', s=40.0, label='training
data points')
    plt.plot(X_test, y_test, '--', color='royalblue', linewidth=2.0,
label='Ground truth')


def plot_relu(bias, slope):
    plt.scatter([-bias / slope], 0, c='darkgrey', s=40.0)
    if slope > 0 and bias < 0:
        plt.plot([0, -bias / slope, 1], [0, 0, slope * (1 - bias)],
':')
    elif slope < 0 and bias > 0:
        plt.plot([0, -bias / slope, 1], [-bias * slope, 0, 0], ':')


def plot_relus(params):
    slopes = to_numpy(params[0]).ravel()
    biases = to_numpy(params[1])
```

```
    for relu in range(biases.size):
        plot_relu(biases[relu], slopes[relu])


def plot_function(X_test, net):
    y_pred = net(to_torch(X_test))
    plt.plot(X_test, to_numpy(y_pred), '-', color='forestgreen',
label='prediction')


def plot_update(X, y, X_test, y_test, net, state=None):
    if state is not None:
        net.load_state_dict(state)
    plt.figure(figsize=(10, 7))
    plot_relus(list(net.parameters()))
    plot_function(X_test, net)
    plot_data(X, y, X_test, y_test)
    plt.legend()
    plt.show();


def train_network(X, y, X_test, y_test, net, optim, n_steps,
save_every, device="cpu", initial_weights=None, verbose=False):
    loss = torch.nn.MSELoss()
    y_train = to_torch(y.reshape(-1, 1)).to(device=device)
    X_train = to_torch(X).to(device=device)

    y_test = to_torch(y_test.reshape(-1, 1)).to(device=device)
    X_test = to_torch(X_test).to(device=device)
    if initial_weights is not None:
        net.load_state_dict(initial_weights)
    history = {}
    for s in range(n_steps):
        perm = torch.randperm(y.size, device=device)
        subsample = perm[:y.size // 5]
        step_loss = loss(y_train[subsample],
net(X_train[subsample, :]))
        optim.zero_grad()
        step_loss.backward()
        optim.step()
        if (s + 1) % save_every == 0 or s == 0:
            history[s + 1] = {}
            history[s + 1]['state'] = copy.deepcopy(net.state_dict())
            with torch.no_grad():
                test_loss = loss(y_test, net(X_test))
            history[s + 1]['train_error'] = to_numpy(step_loss).item()
            history[s + 1]['test_error'] = to_numpy(test_loss).item()
            if verbose:
                print("SGD Iteration %d" % (s + 1))
                print("\tTrain Loss: %.3f" %
```

```
to_numpy(step_loss).item())
                print("\tTest Loss: %.3f" %
to_numpy(test_loss).item())
            else:
                # Print update every 10th save point
                if (s + 1) % (save_every * 10) == 0:
                    print("SGD Iteration %d" % (s + 1))

    return history


def plot_test_train_errors(history):
    sample_points = np.array(list(history.keys()))
    etrain = [history[s]['train_error'] for s in history]
    etest = [history[s]['test_error'] for s in history]
    plt.plot(sample_points / 1e3, etrain, label='Train Error')
    plt.plot(sample_points / 1e3, etest, label='Test Error')
    plt.xlabel("Iterations (1000's)")
    plt.ylabel("MSE")
    plt.yscale('log')
    plt.legend()
    plt.show();


def make_iter_slider(iters):
    return widgets.SelectionSlider(
        options=iters,
        value=1,
        description='SGD Iterations: ',
        disabled=False
    )


def history_interactive(history, idx, X, y, X_test, y_test, net):
    plot_update(X, y, X_test, y_test, net, state=history[idx]
['state'])
    plt.show()
    print("Train Error: %.3f" % history[idx]['train_error'])
    print("Test Error: %.3f" % history[idx]['test_error'])


def make_history_interactive(history, X, y, X_test, y_test, net):
    sample_points = list(history.keys())
    return interactive(history_interactive,
                       history=fixed(history),
                       idx=make_iter_slider(sample_points),
                       X=fixed(X),
                       y=fixed(y),
                       X_test=fixed(X_test),
                       y_test=fixed(y_test),
```

```
                        net=fixed(net))


%matplotlib inline
```

Generate Training and Test Data

We are using piecewise linear function. Our training data has added noise $y=f(x)+\epsilon, \epsilon \sim N(0, \sigma^2)$. The test data is noise free.

Once you have gone through the discussion once you may wish to adjust the number of training samples and noise variance to see how gradient descent behaves under the new conditions.

```
f_type = 'piecewise_linear'

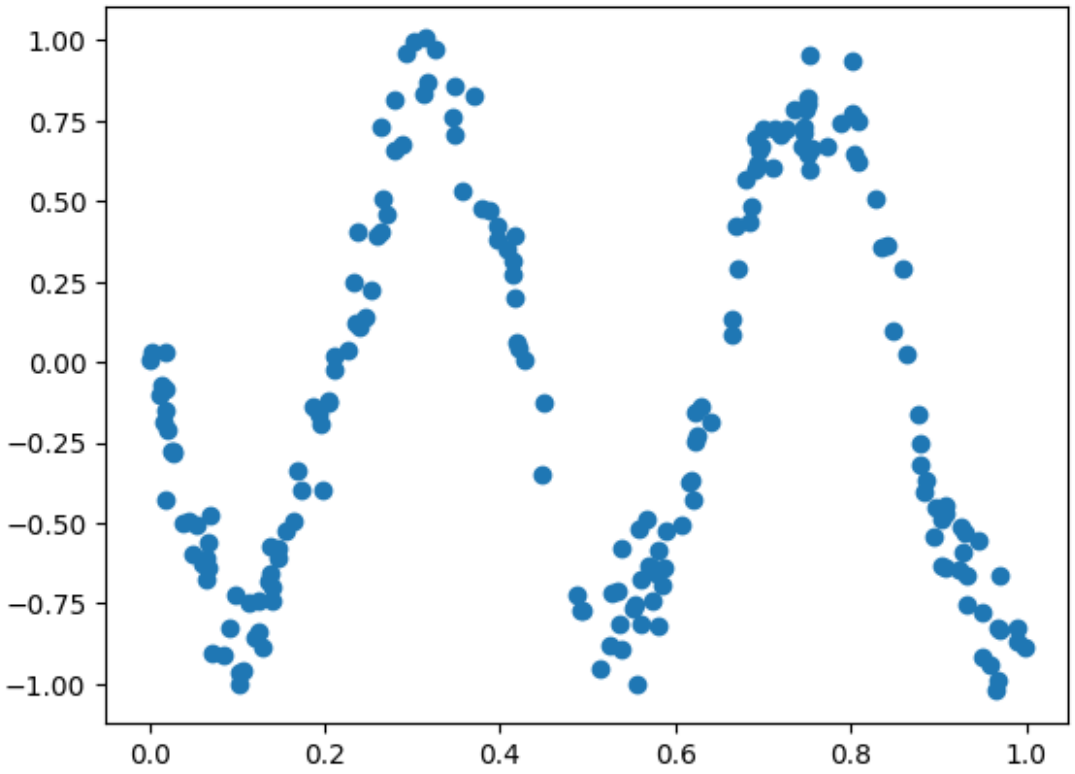
def f_true(X, f_type):
    if f_type == 'sin(20x)':
        return np.sin(20 * X[:,0])
    else:
        TenX = 10 * X[:,0]
        _ = 12345
        return (TenX - np.floor(TenX)) * np.sin(_ * np.ceil(TenX)) -
(TenX - np.ceil(TenX)) * np.sin(_ * np.floor(TenX))

n_features = 1
n_samples = 200
sigma = 0.1
rng = np.random.RandomState(1)

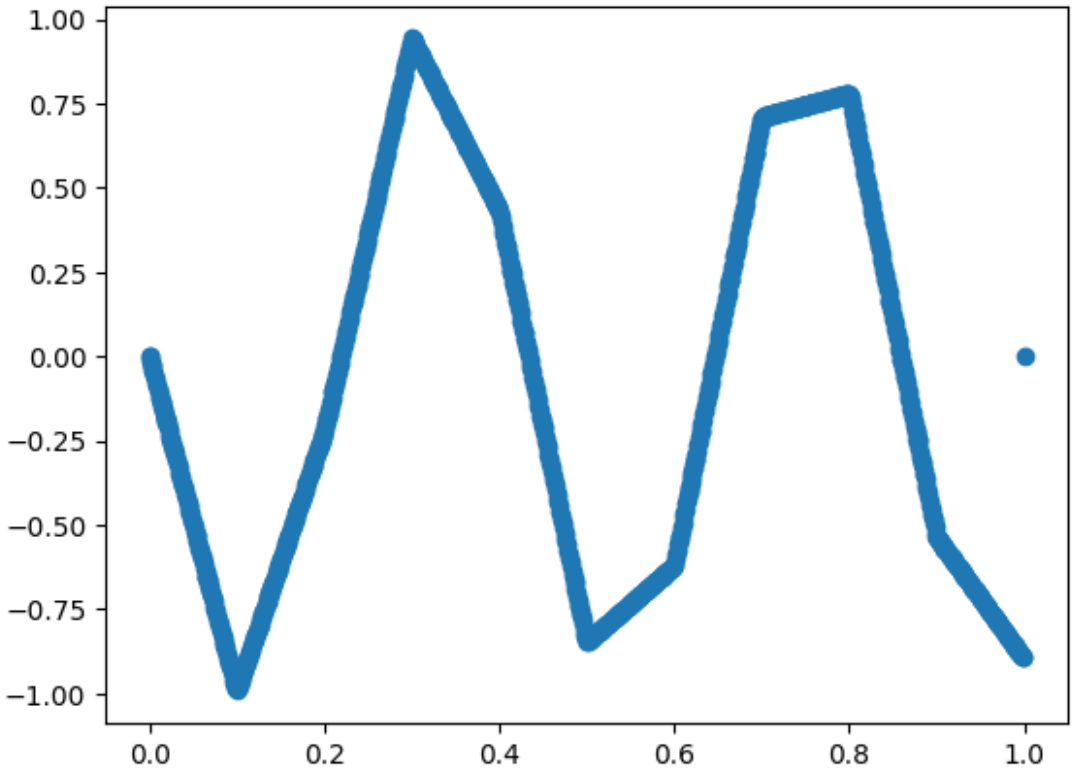
# Generate train data
X = np.sort(rng.rand(n_samples, n_features), axis=0)
y = f_true(X, f_type) + rng.randn(n_samples) * sigma

# Generate NOISELESS test data
X_test = np.concatenate([X.copy(), np.expand_dims(np.linspace(0., 1.,
1000), axis=1)])
X_test = np.sort(X_test, axis=0)
y_test = f_true(X_test, f_type)

plt.scatter(X, y)
plt.show()
```




```
plt.scatter(X_test, y_test)
plt.show()
```



Define the Neural Networks

We will learn the piecewise linear target function using a simple 1-hidden layer neural network with ReLU non-linearity, defined by

$$\hat{y} = W^{(2)} \Phi\left(W^{(1)} x + b^{(1)}\right) + b^{(2)}$$

where $\Phi(x) = ReLU(x)$ and superscripts refer to indices, not the power operator.

We will also create two SGD optimizers to allow us to choose whether to train all parameters or only the linear output layer's parameters. Note that we use separate learning rates for the two version of training. There is too much variance in the gradients when training all layers to use a large learning rate, so we have to decrease it.

We will modify the default initialization of the biases so that the ReLU elbows are all inside the region we are interested in.

We create several versions of this network with varying widths to explore how hidden layer width impacts learning performance.

Once you have gone through the discussion once you may wish to train networks with even larger widths to see how they behave under the three different training paradigms in this notebook.

```
# Don't rerun this cell after training or you will lose all your work
nets_by_size = {}

widths = [10, 20, 40]
for width in widths:
    # Define a 1-hidden layer ReLU nonlinearity network
    net = nn.Sequential(nn.Linear(1, width),
                        nn.ReLU(),
                        nn.Linear(width, 1))
    loss = nn.MSELoss()
    # Get trainable parameters
    weights_all = list(net.parameters())
    # Get the output weights alone
    weights_out = weights_all[2:]
    # Adjust initial biases so elbows are in [0,1]
    elbows = np.sort(np.random.rand(width))
    new_biases = -elbows * to_numpy(weights_all[0]).ravel()
    weights_all[1].data = to_torch(new_biases)
    # Create SGD optimizers for outputs alone and for all weights
    lr_out = 0.2
    lr_all = 0.02
    opt_all = torch.optim.SGD(params=weights_all, lr=lr_all)
    opt_out = torch.optim.SGD(params=weights_out, lr=lr_out)
    # Save initial state for comparisons
    initial_weights = copy.deepcopy(net.state_dict())
    # print("Initial Weights", initial_weights)
    nets_by_size[width] = {'net': net, 'opt_all': opt_all,
                           'opt_out': opt_out, 'init':
initial_weights}

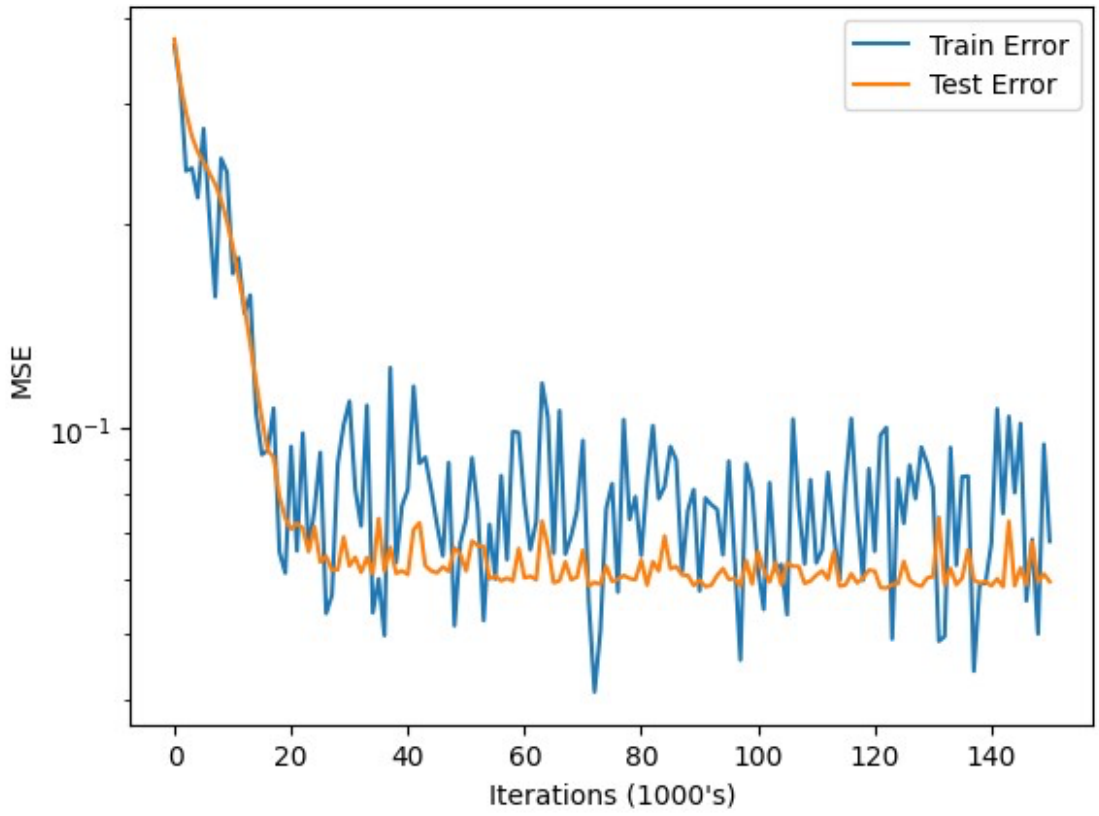
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
for width, net in nets_by_size.items():
  net['net'].to(device=device)
```

Train the neural networks

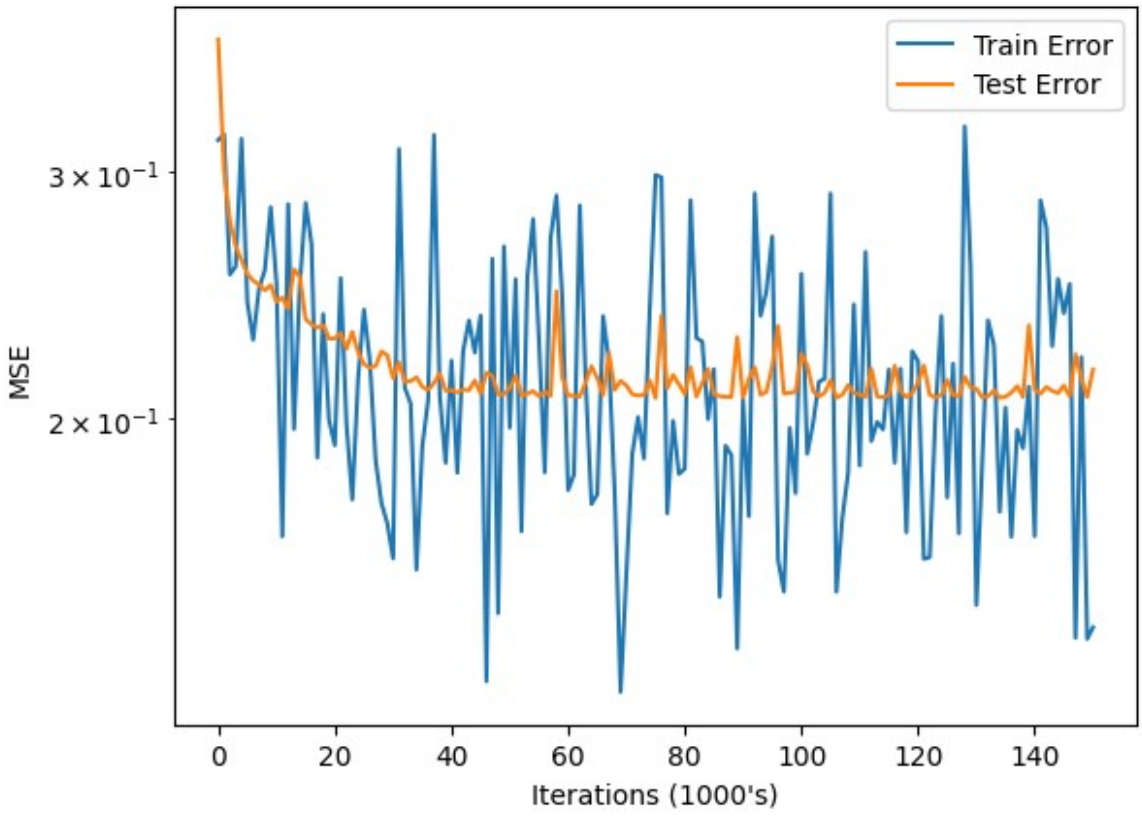
```
n_steps = 150000
save_every = 1000
t0 = time.time()
for w in widths:
    print("-"*40)
    print("Width", w)
    new_net = nn.Sequential(nn.Linear(1, w),
                        nn.ReLU(),
                        nn.Linear(w, 1))
    new_net.load_state_dict(nets_by_size[w]
```

```
['net'].state_dict().copy())
    new_net.to(device=device)
    opt_all = torch.optim.SGD(params=new_net.parameters(), lr=lr_all)
    initial_weights = nets_by_size[w]['init']
    history_all = train_network(X, y, X_test, y_test,
                                new_net, optim=opt_all,
                                n_steps=n_steps, save_every=save_every,
                                initial_weights=initial_weights,
                                verbose=False, device=device)
    nets_by_size[w]['trained_net'] = new_net
    nets_by_size[w]['hist_all'] = history_all
    print("Width", w)
    plot_test_train_errors(history_all)
t1 = time.time()
print("-"*40)
print("Trained all layers in %.1f minutes" % ((t1 - t0) / 60))
```

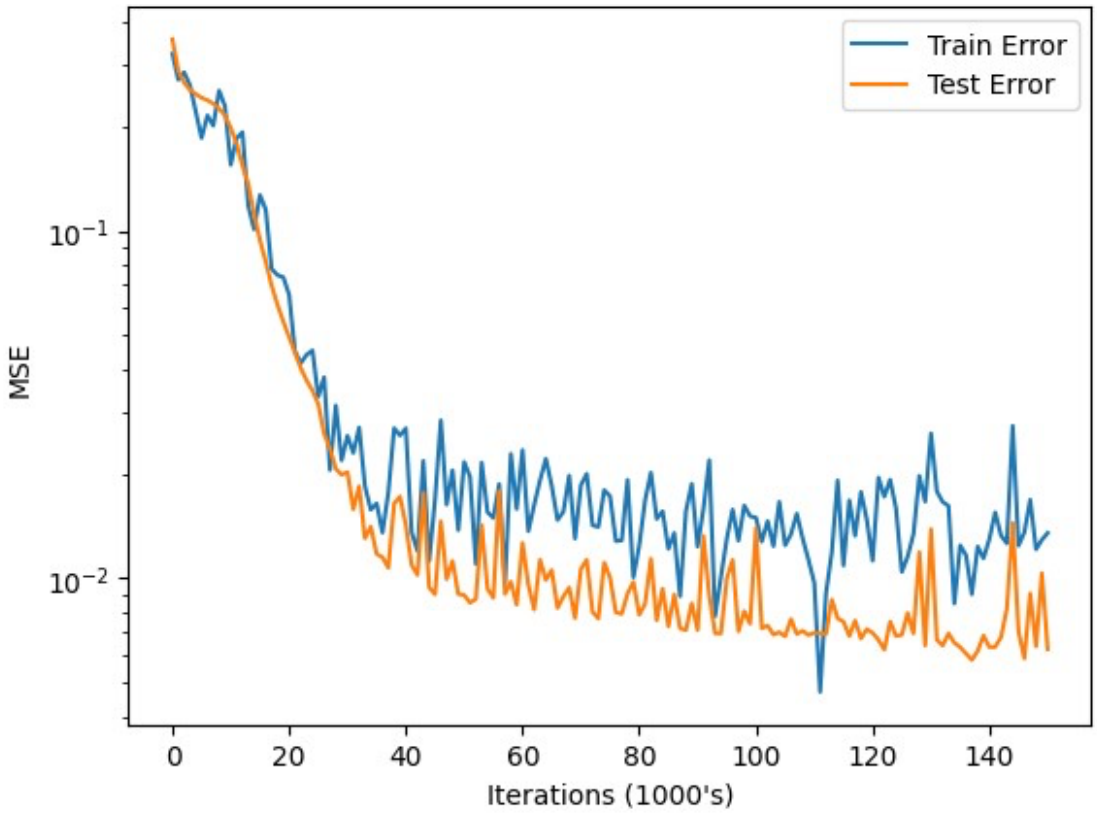
```
----------------------------------------
Width 10
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Width 10
```




```
----------------------------------------
Width 20
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Width 20
```




```
----------------------------------------
Width 40
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Width 40
```




```
----------------------------------------
Trained all layers in 7.8 minutes
```

(a) Visualize Gradients

Visualize the features corresponding to $\frac{\partial}{\partial w_i^{[1]}} y(x)$ and $\frac{\partial}{\partial b_i^{[1]}} y(x)$ where $w_i^{[1]}$ are the first hidden layer's weights and the $b_i^{[1]}$ are the first hidden layer's biases. These derivatives should be evaluated at at least both the random initialization and the final trained network. When visualizing these features, plot them as a function of the scalar input x, the same way that the notebook plots the constituent "elbow" features that are the outputs of the penultimate layer.

```
def backward_and_plot_grad(X, model, vis_name='all', title='',
legend=False):
    """
    Run backpropagation on `model` using `X` as the input
    to compute the gradient w.r.t. parameters of `y`,
    and then visualize collected gradients according to `vis_name`
    """
    width = model[0].out_features  # the width is the number of hidden
units.
```

```
    gradients = np.zeros((width, X.shape[0]))
    num_pts = 0
    gradient_collect, vis_collect = { }, { }
    for x in X:
        y = model(to_torch(x).to(device=device))


##########################################################################
##
        # TODO: Complete the following part to run backpropagation. (2
lines)
        # Hint: Remember to set grad to zero before backpropagation

##########################################################################
##
        model.zero_grad()
        y.backward()

##########################################################################
##

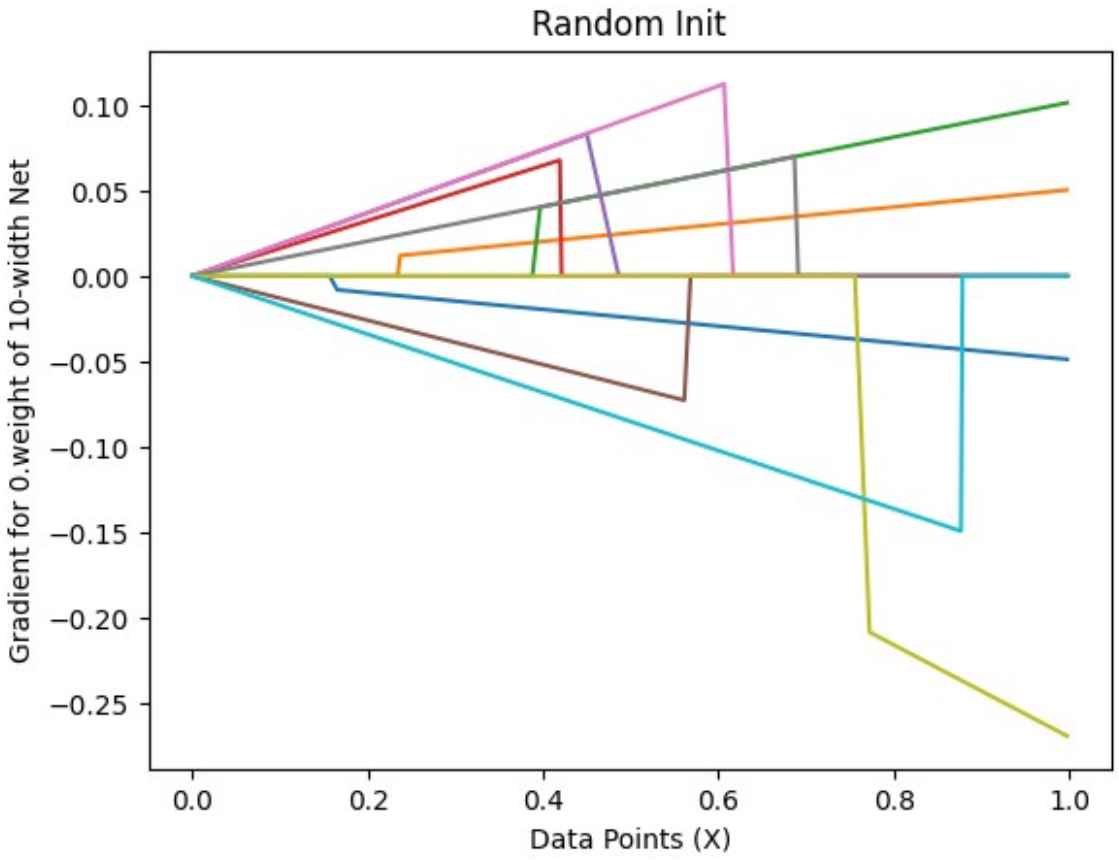
        # collect gradients from `p.grad.data`
        for n, p in model.named_parameters():
            for w_idx, w_grad in enumerate( p.grad.data.reshape(-1) ):
                if f'{n}.{w_idx}' not in gradient_collect:
                    gradient_collect[ f'{n}.{w_idx}' ] = {'x':[], 'y':
[]}
                if vis_name == 'all' or vis_name == n:
                    if f'{n}.{w_idx}' not in vis_collect:
                        vis_collect[f'{n}.{w_idx}'] = True
                gradient_collect[ f'{n}.{w_idx}' ]
['y'].append( w_grad.item() )
                gradient_collect[ f'{n}.{w_idx}' ]['x'].append( x )

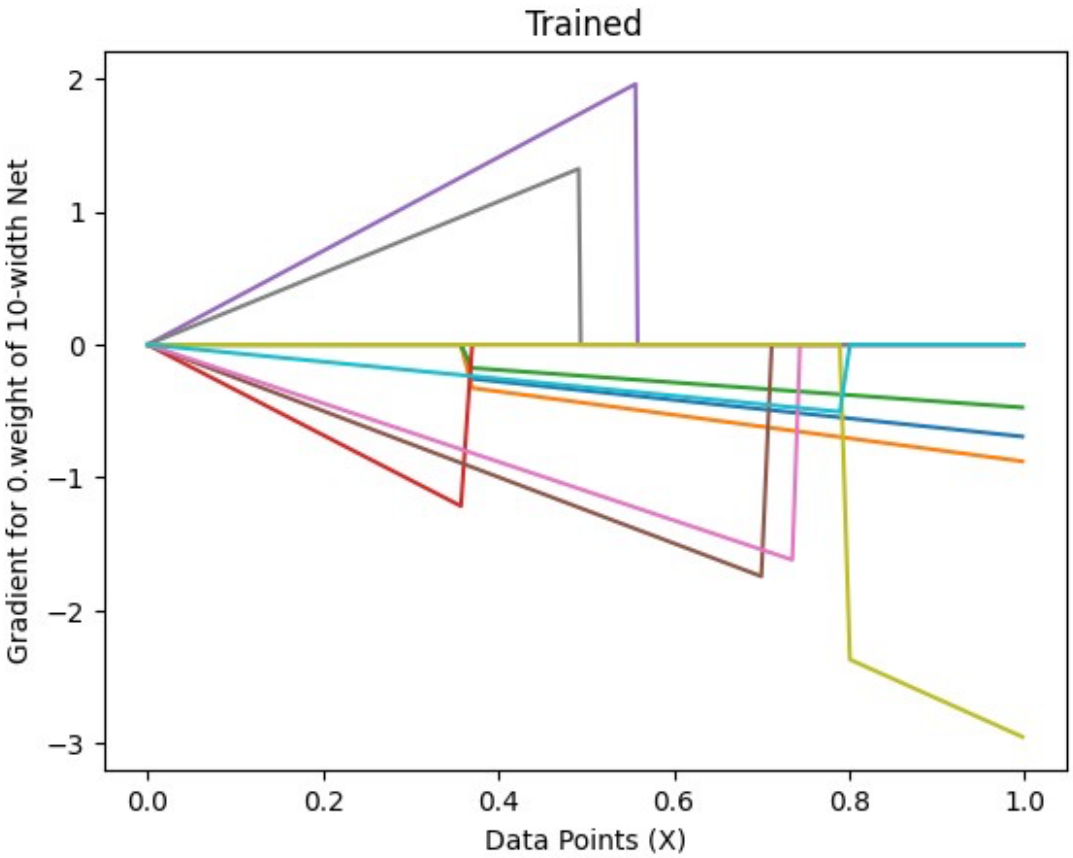
    for w_n in vis_collect:
        # we assume that X is sorted, so we use line plot
        plt.plot( X, gradient_collect[w_n]['y'], label=w_n )

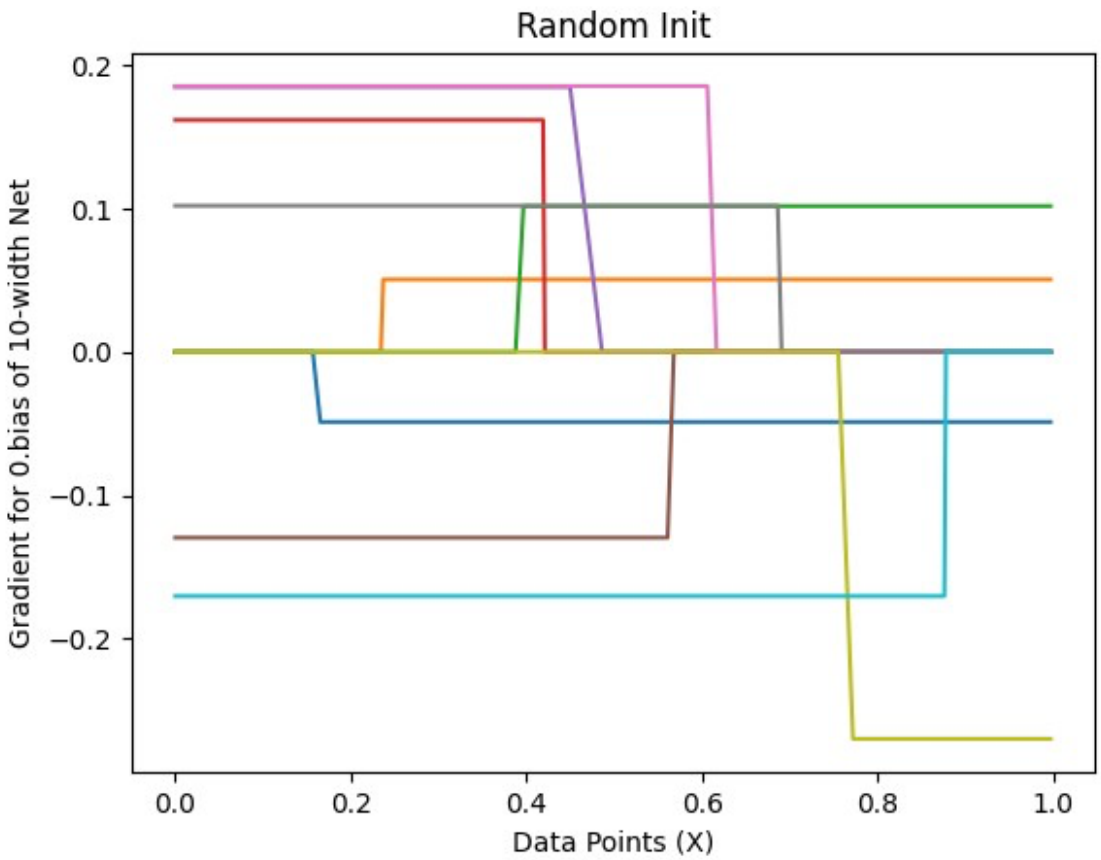
    plt.xlabel('Data Points (X)')
    plt.ylabel(f'Gradient for {vis_name} of {width}-width Net')
    if legend:
        plt.legend()
    plt.title(title)
    plt.show()

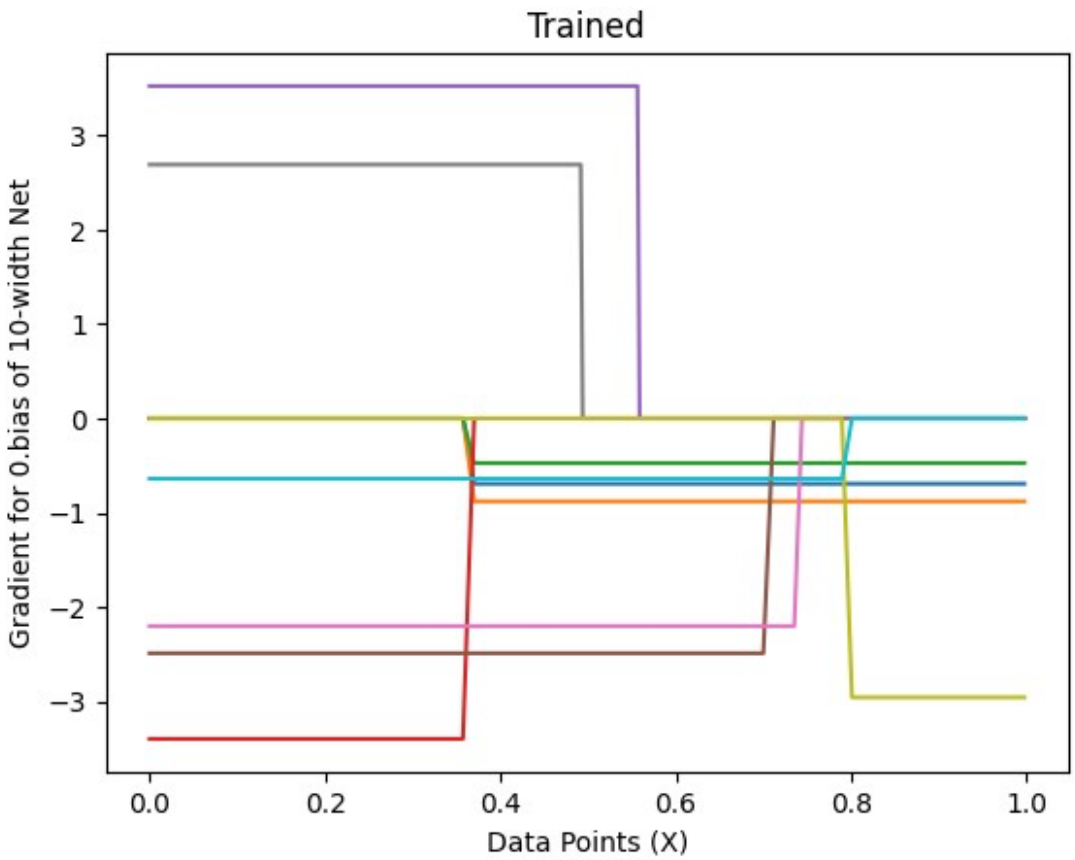

for width in nets_by_size:
    backward_and_plot_grad(X, nets_by_size[width]['net'], '0.weight',
'Random Init')
    backward_and_plot_grad(X, nets_by_size[width]['trained_net'],
```

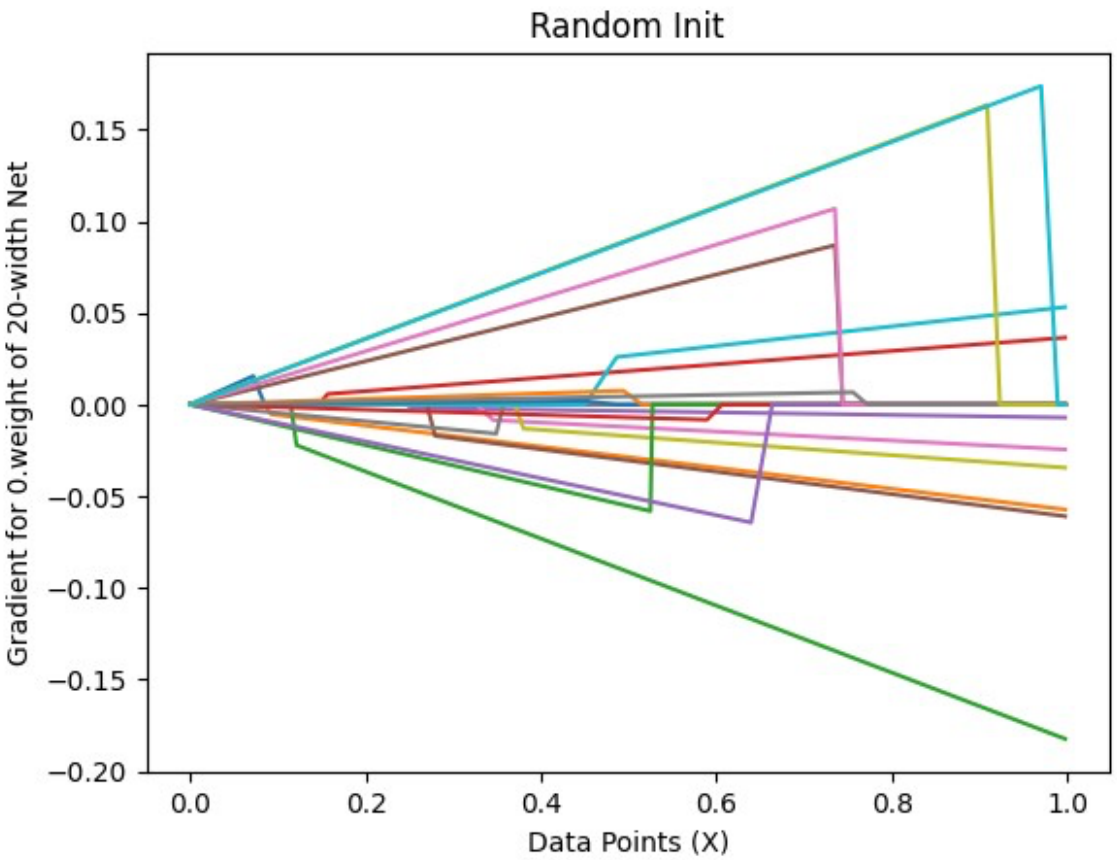
```
'0.weight', 'Trained')
    backward_and_plot_grad(X, nets_by_size[width]['net'], '0.bias',
'Random Init')
    backward_and_plot_grad(X, nets_by_size[width]['trained_net'],
'0.bias', 'Trained')
```

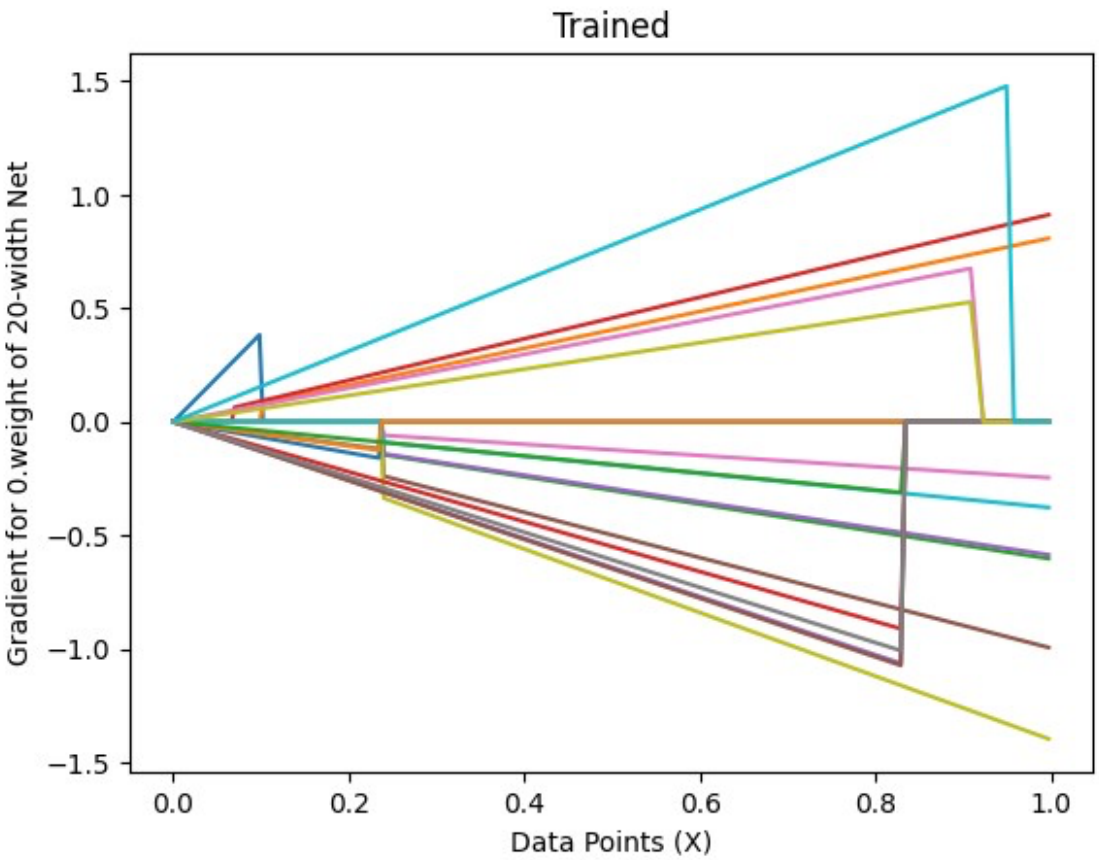


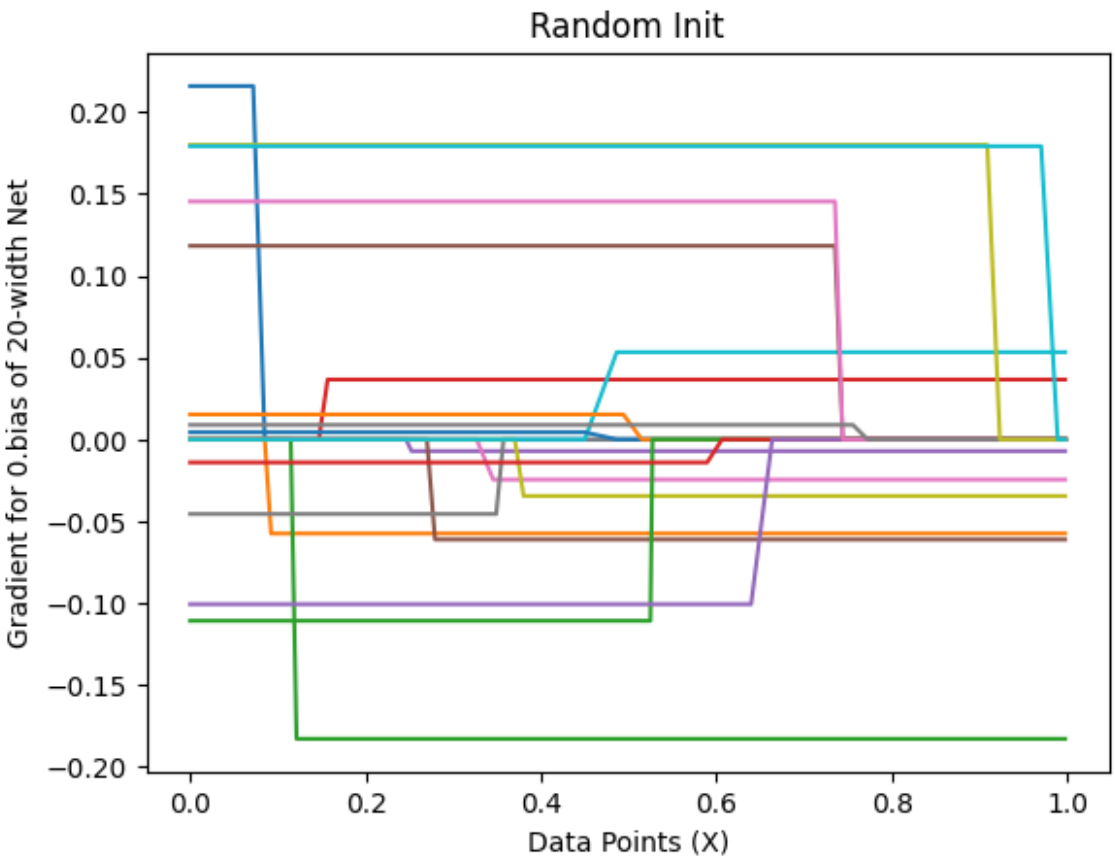

Trained
Gradient for 0.weight of 10-width Net
Data Points (X)
2
1
0
−1
−2
−3
0.0
0.2
0.4
0.6
0.8
1.0

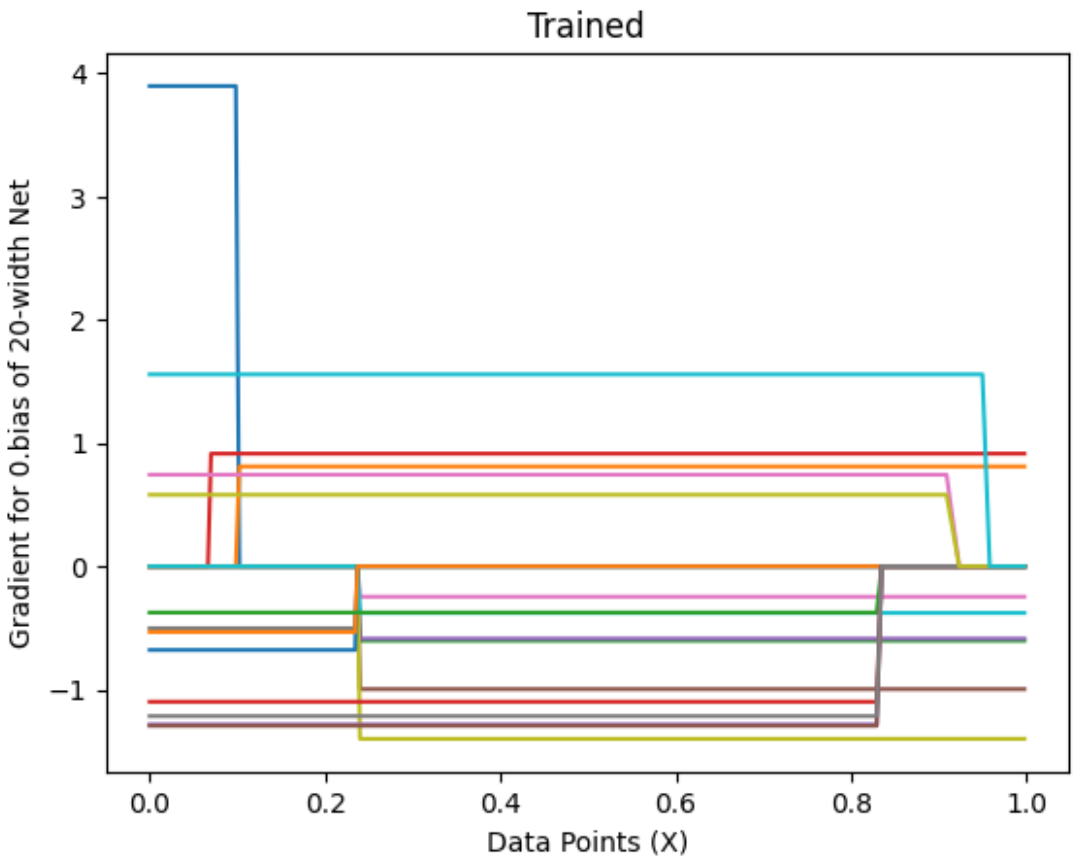

Random Init
0.2
0.1
0.0
−0.1
−0.2
Gradient for 0.bias of 10-width Net
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

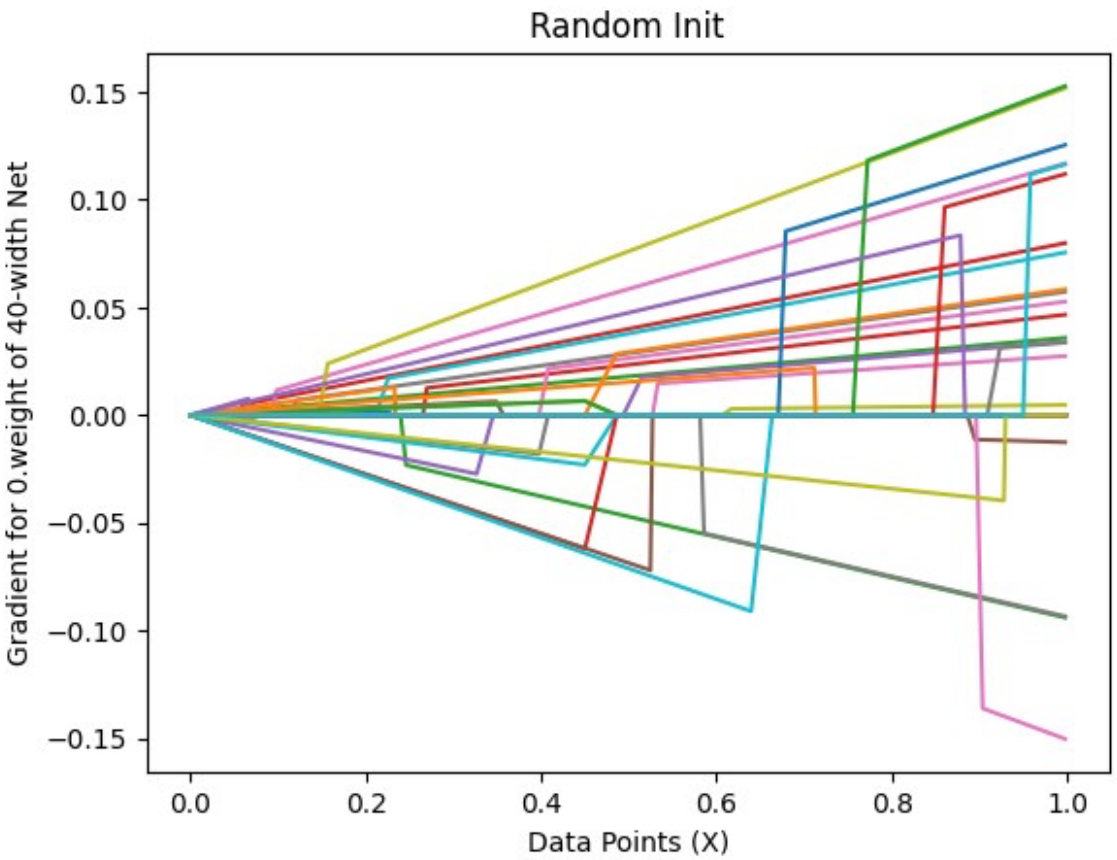

Trained
Gradient for 0.bias of 10-width Net
3
2
1
0
−1
−2
−3
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

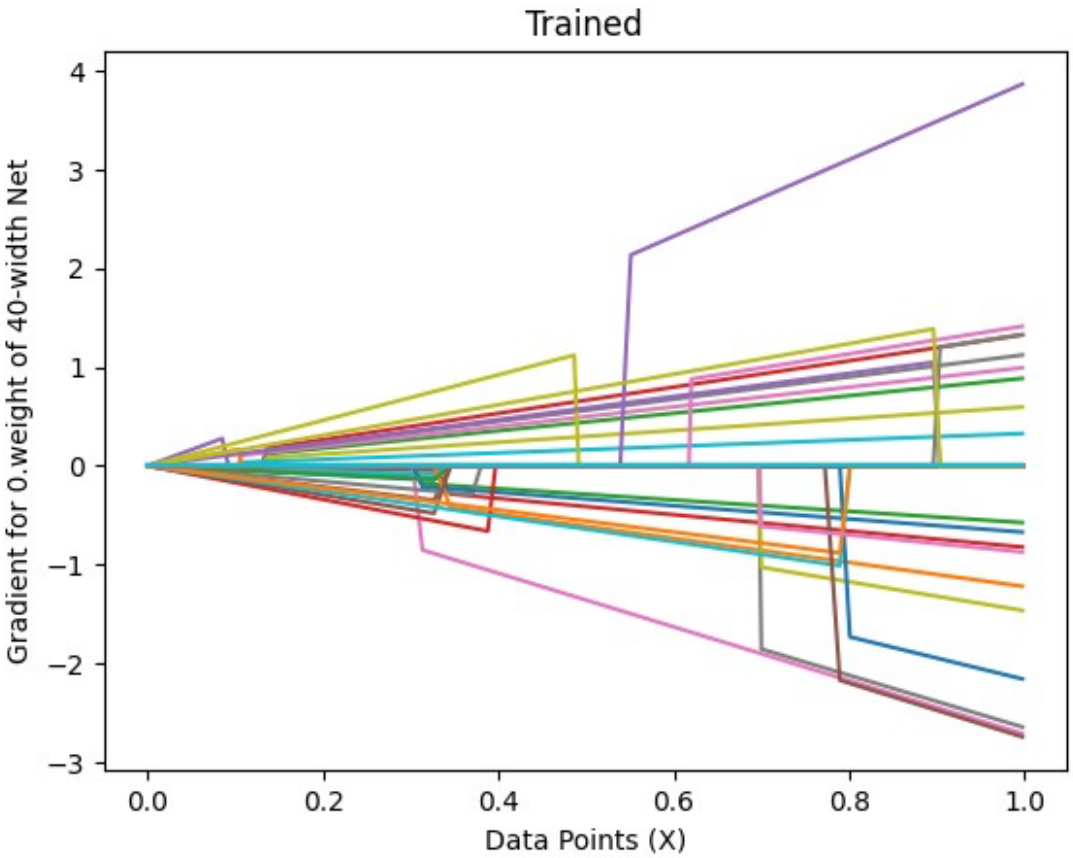

Random Init
Gradient for 0.weight of 20-width Net
Data Points (X)
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
−0.20
0.0
0.2
0.4
0.6
0.8
1.0

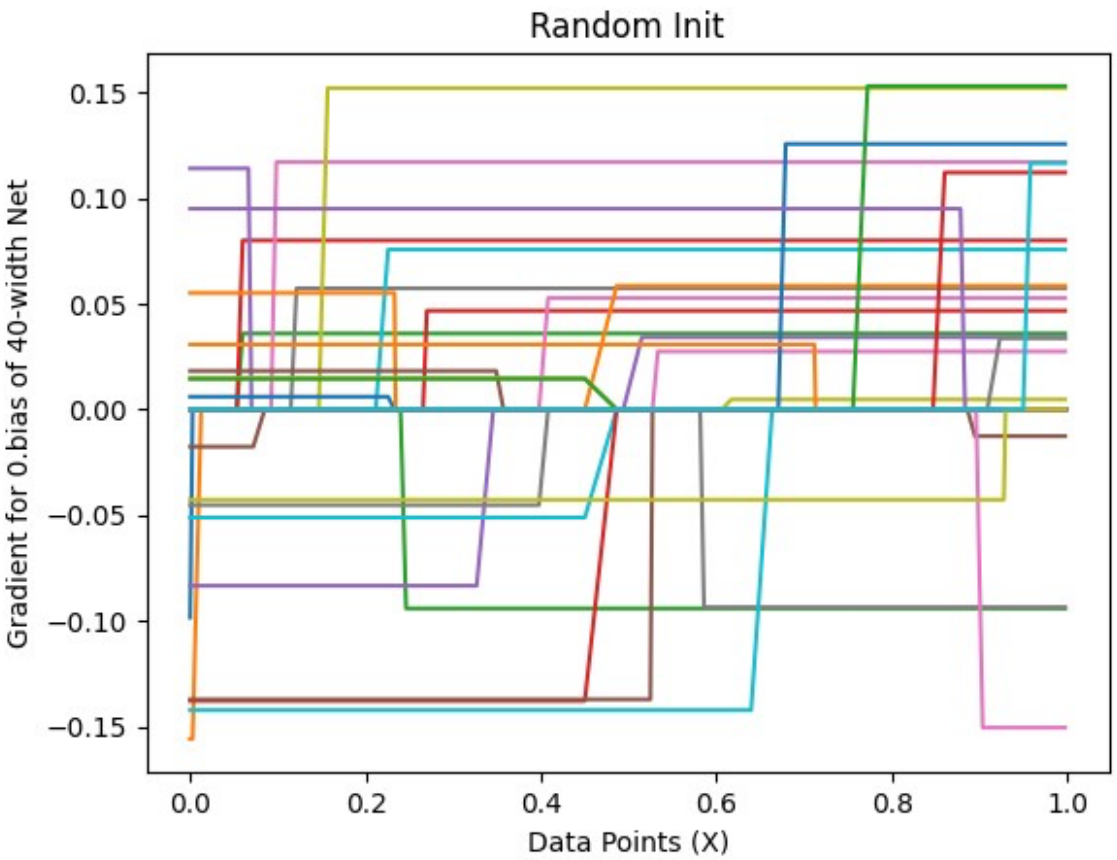

Trained
Gradient for 0.weight of 20-width Net
Data Points (X)
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.0
0.2
0.4
0.6
0.8
1.0

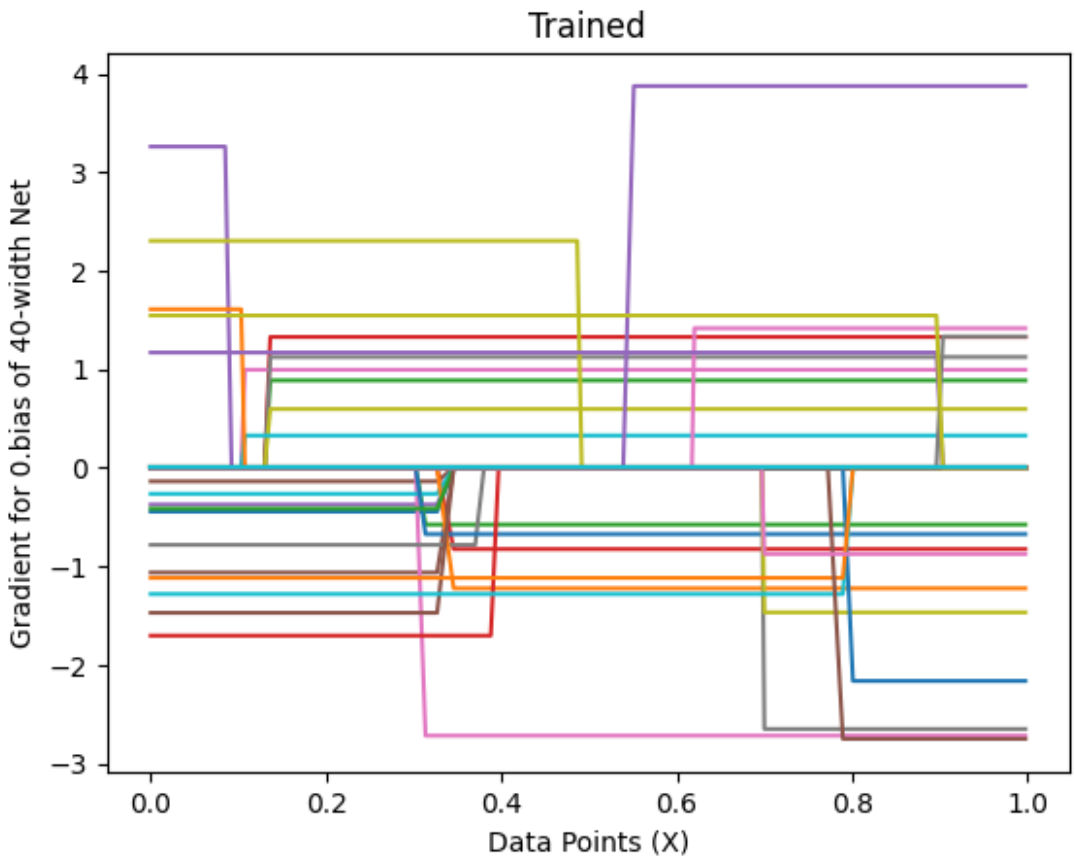

Random Init
Gradient for 0.bias of 20-width Net
Data Points (X)
0.20
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
−0.20
0.0
0.2
0.4
0.6
0.8
1.0


Trained
Gradient for 0.bias of 20-width Net
4
3
2
1
0
−1
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)


Random Init
Gradient for 0.weight of 40-width Net
Data Points (X)
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0


Trained
Gradient for 0.weight of 40-width Net
Data Points (X)
4
3
2
1
0
−1
−2
−3
0.0
0.2
0.4
0.6
0.8
1.0


Random Init
Gradient for 0.bias of 40-width Net
Data Points (X)
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0



(b) SVD for feature matrix

During training, we can imagine that we have a generalized linear model with a feature matrix corresponding to the linearized features corresponding to each learnable parameter. We know from our analysis of gradient descent, that the singular values and singular vectors corresponding to this feature matrix are important.

Use the SVD of this feature matrix to plot both the singular values and visualize the “principle features” that correspond to the d-dimensional singular vectors multiplied by all the features corresponding to the parameters

(HINT: Remember that the feature matrix whose SVD you are taking has n rows where each row corresponds to one training point and d columns where each column corresponds to each of the learnable features. Meanwhile, you are going to be plotting/visualizing the "principle features" as functions of x even at places where you don't have training points.)

```
def compute_svd_plot_features(X, y, X_test, y_test, model):
    width = model[0].out_features  # the width is the number of hidden
units.
    gradients = np.zeros((width, X.shape[0]))
    num_pts = 0
```

```
    gradient_collect, vis_collect = { }, { }
    for x in X:
        y = model(to_torch(x).to(device=device))


#########################################################################
##
        # TODO: Complete the following part to run backpropagation. (2
lines)
        # Hint: The same as part (a)

#########################################################################
##
        model.zero_grad()
        y.backward()

#########################################################################
##

        for n, p in model.named_parameters():
            for w_idx, w_grad in enumerate( p.grad.view(-1).data ):
                if f'{n}.{w_idx}' not in gradient_collect:
                    gradient_collect[ f'{n}.{w_idx}' ] = {'x':[], 'y':
[]}
                gradient_collect[ f'{n}.{w_idx}' ]
['y'].append( w_grad.item() )
                gradient_collect[ f'{n}.{w_idx}' ]['x'].append( x )

    feature_matrix = []
    for w_n in gradient_collect:
        feature_matrix.append( gradient_collect[w_n]['y'] )
    feature_matrix = np.array( feature_matrix ).T


#########################################################################
######
    # TODO: Complete the following part to SVD-decompose the feature
matrix.
    #       (1 line)
    # Hint: the shape of u, s, vh should be [n, d], [d], and [d, d]
    #       respectively

#########################################################################
######
    u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)

#########################################################################
######

    plt.scatter(np.arange(s.shape[0]), s, c='darkorange', s=40.0,
```

```
label='singular values')
    plt.legend()
    plt.show()

    # Construct more training matrix

#########################################################################
######
    # TODO: Complete the following part to compute the pricipal
feature
    #       (1 line)

#########################################################################
######
    princple_feature = u @ np.diag(s)

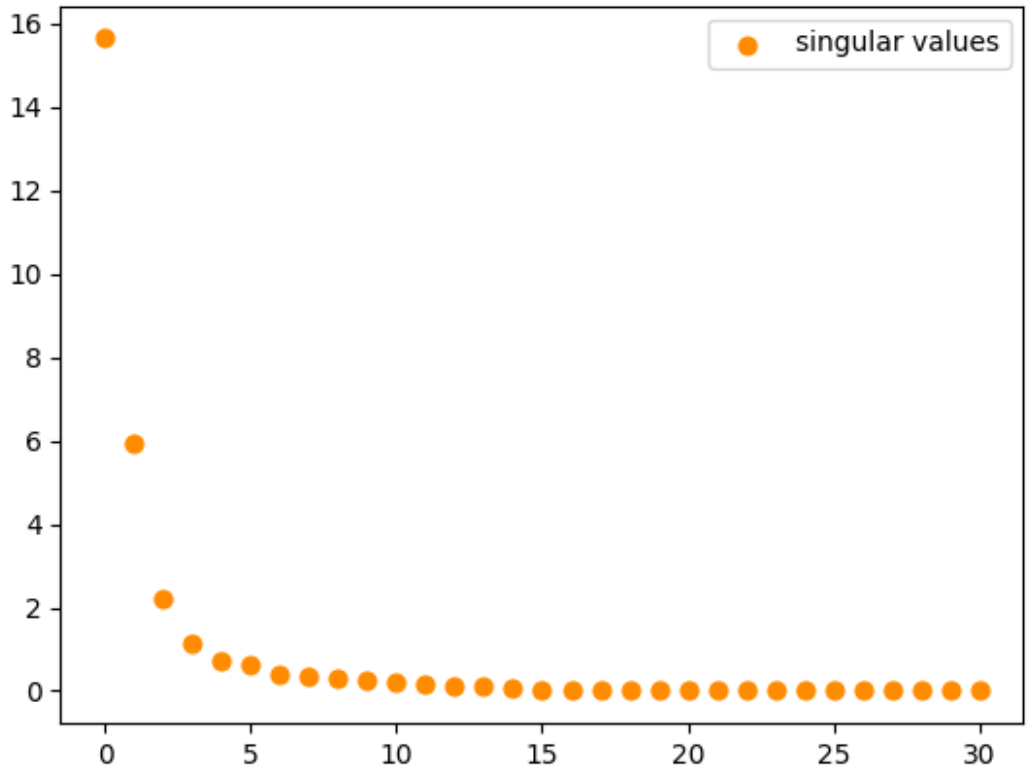
#########################################################################
######

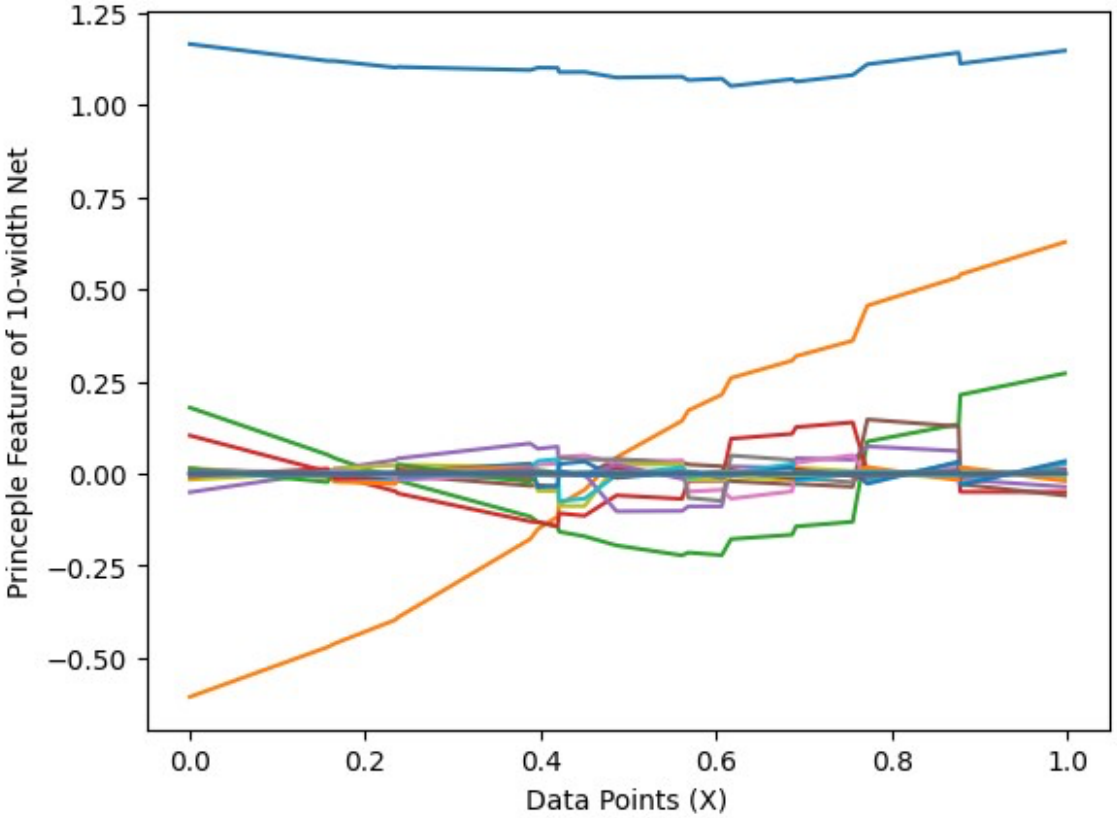

    for w_idx in range(feature_matrix.shape[1]):
        plt.plot( X, princple_feature.T[w_idx] )

    plt.xlabel('Data Points (X)')
    plt.ylabel(f'Princeple Feature of {width}-width Net')
    plt.show()

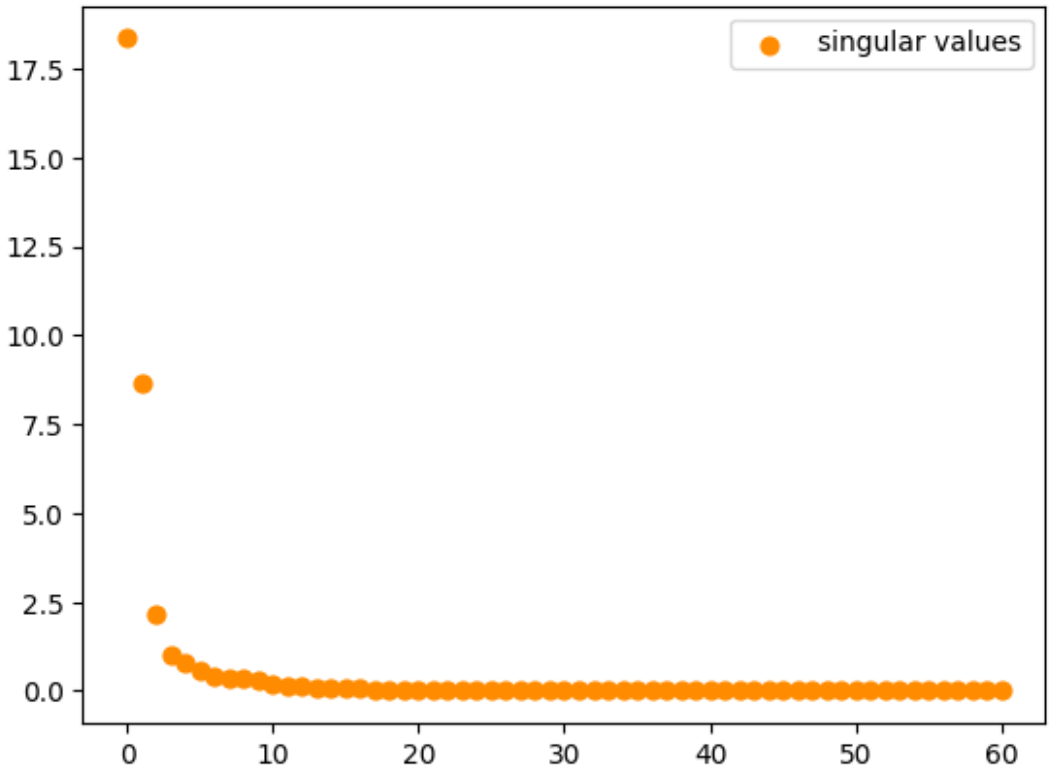
for w in widths:
    net = nets_by_size[w]['net']
    print("Width", w)
    compute_svd_plot_features(X, y, X_test, y_test, net)
```

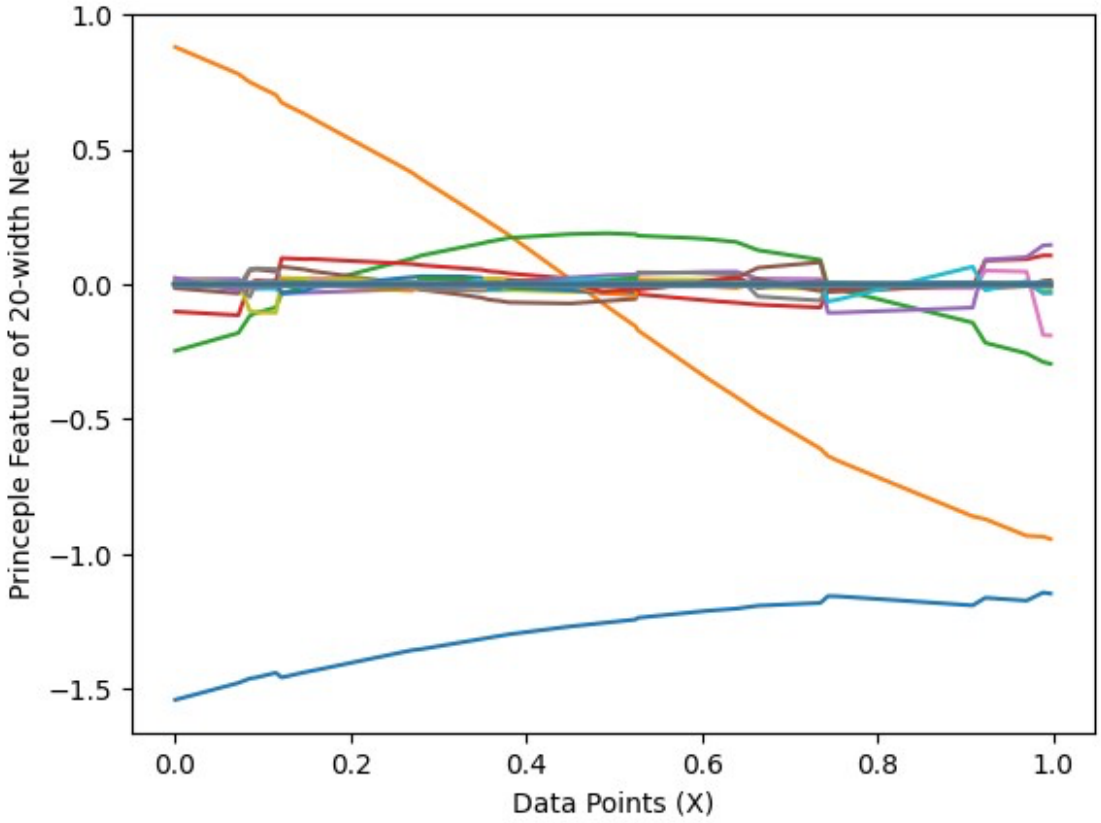
```
Width 10
```


singular values
16
14
12
10
8
6
4
2
0
0
5
10
15
20
25
30

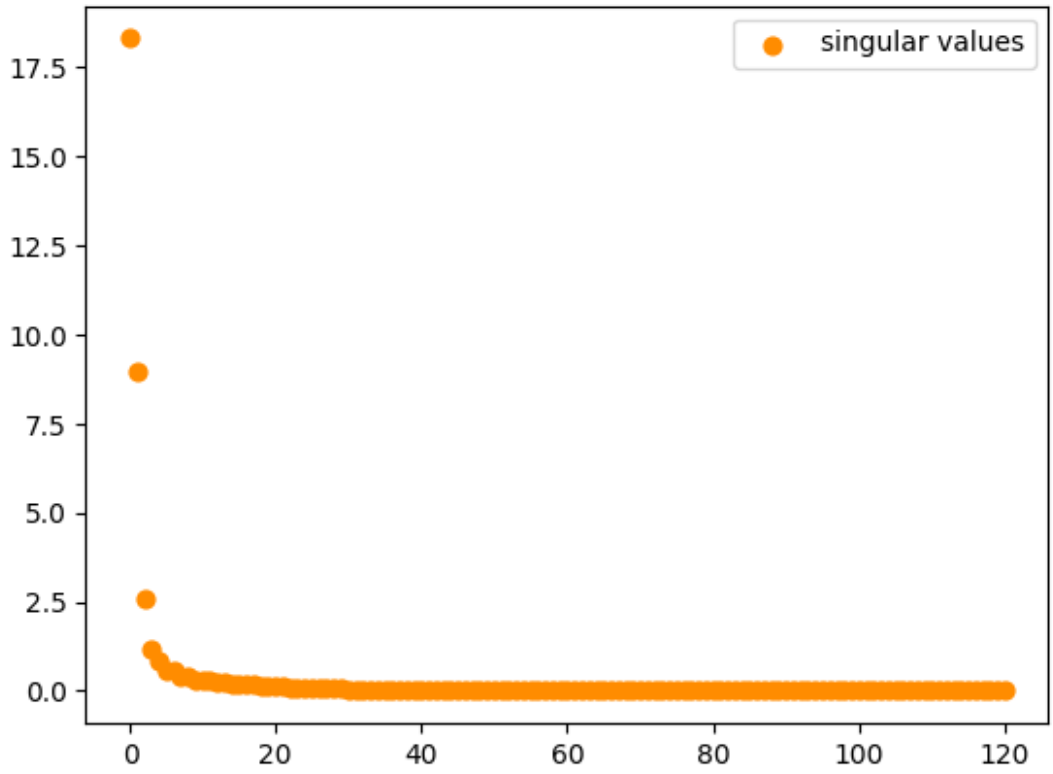

Princeple Feature of 10-width Net
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

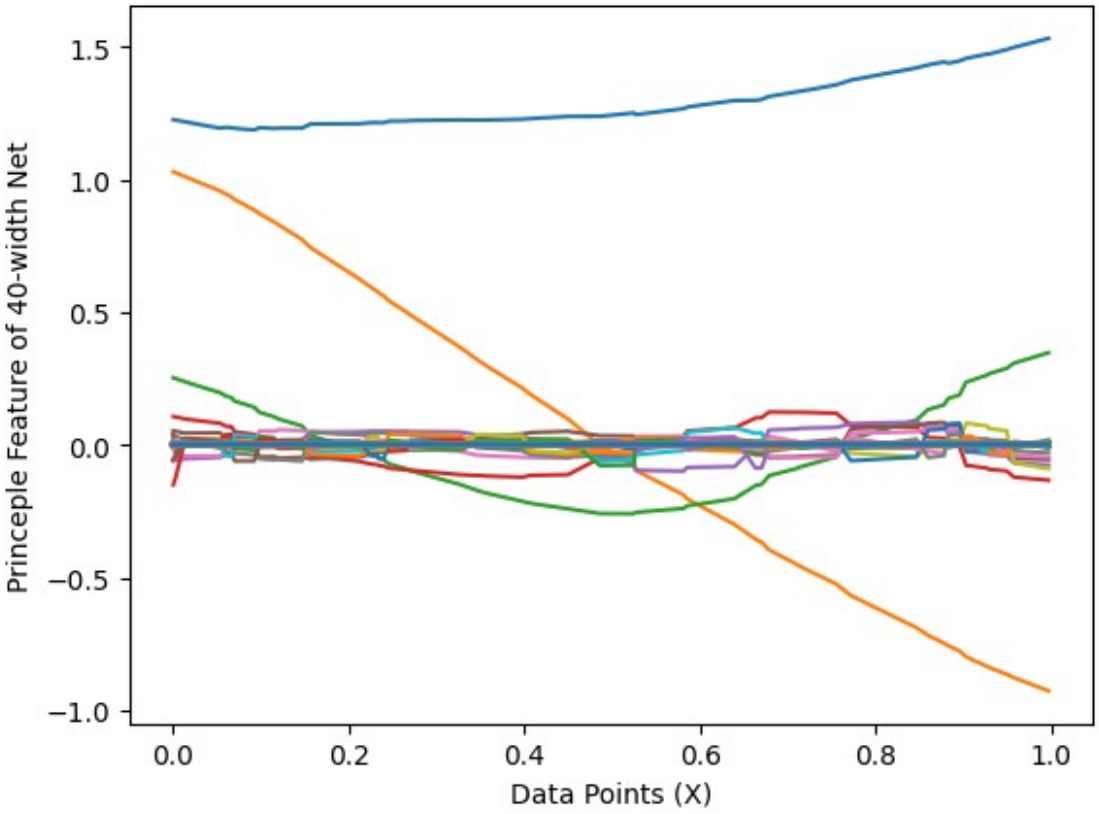
```
Width 20
```




1.0
0.5
0.0
−0.5
−1.0
−1.5
Princeple Feature of 20-width Net
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

```
Width 40
```


singular values
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
0
20
40
60
80
100
120


Princeple Feature of 40-width Net
1.5
1.0
0.5
0.0
−0.5
−1.0
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

(c) Two-layer Network

Augment the jupyter notebook to add a second hidden layer of the same size as the first hidden layer, fully connected to the first hidden layer.

Allow the visualization of the features corresponding to the parameters in both hidden layers, as well as the "principle features" and the singular values.

```
##########################################################################
##########
# (YOUR CODE HERE)
# Hint: You can reuse most of the code from above. You should change
the network
# definition and visualization function 'backward_and_plot_grad' such
that an
# additional hidden layer is added.
##########################################################################
##########
# Define two-layer neural networks
nets_two_layer = {}

for width in widths:
    # Define 2-hidden layer neural network
    net = nn.Sequential(
        nn.Linear(1, width),      # First hidden layer
        nn.ReLU(),
        nn.Linear(width, width),  # Second hidden layer
        nn.ReLU(),
        nn.Linear(width, 1)       # Output layer
    )

    # Get all trainable parameters
    weights_all = list(net.parameters())

    # Adjust first layer biases to position ReLU elbows in [0,1] range
    elbows = np.sort(np.random.rand(width))
    new_biases_1 = -elbows * to_numpy(weights_all[0]).ravel()
    weights_all[1].data = to_torch(new_biases_1)

    # Use Xavier initialization for second layer weights and biases
    torch.nn.init.xavier_uniform_(weights_all[2])
    torch.nn.init.normal_(weights_all[3], mean=0, std=0.1)

    # Create optimizer with appropriate learning rate for deeper
network
    lr_all = 0.01
    opt_all = torch.optim.SGD(params=weights_all, lr=lr_all)

    # Save initial state
    initial_weights = copy.deepcopy(net.state_dict())
```

```
    nets_two_layer[width] = {
        'net': net,
        'opt_all': opt_all,
        'init': initial_weights
    }

# Move networks to device
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
for width, net_dict in nets_two_layer.items():
    net_dict['net'].to(device=device)

# Train two-layer networks
n_steps = 150000
save_every = 1000

for w in widths:
    print("-"*40)
    print("Two-Layer Width", w)

    # Create new network instance and load initial weights
    new_net = nn.Sequential(
        nn.Linear(1, w),
        nn.ReLU(),
        nn.Linear(w, w),
        nn.ReLU(),
        nn.Linear(w, 1)
    )
    new_net.load_state_dict(nets_two_layer[w]
['net'].state_dict().copy())
    new_net.to(device=device)

    # Create optimizer
    opt_all = torch.optim.SGD(params=new_net.parameters(), lr=0.01)
    initial_weights = nets_two_layer[w]['init']

    # Train network
    history_all = train_network(X, y, X_test, y_test,
                                new_net, optim=opt_all,
                                n_steps=n_steps, save_every=save_every,
                                initial_weights=initial_weights,
                                verbose=False, device=device)

    # Save training results
    nets_two_layer[w]['trained_net'] = new_net
    nets_two_layer[w]['hist_all'] = history_all

    print("Two-Layer Width", w)
    plot_test_train_errors(history_all)
```

```
# Enhanced gradient visualization function supporting multi-layer
networks
def backward_and_plot_grad_enhanced(X, model, layer_patterns=None,
title='', legend=False):
    """
    Enhanced gradient visualization function supporting multi-layer
networks
    layer_patterns: List of layer patterns to visualize, e.g., ['0.',
'2.'] for first and third layers
    """
    if layer_patterns is None:
        layer_patterns = ['all']

    gradient_collect = {}

    for x in X:
        y_pred = model(to_torch(x).to(device=device))

        # Clear gradients and run backpropagation
        model.zero_grad()
        y_pred.backward()

        # Collect gradients
        for name, param in model.named_parameters():
            if param.grad is not None:
                # Check if parameter is in specified layers
                include_param = 'all' in layer_patterns
                if not include_param:
                    for pattern in layer_patterns:
                        if name.startswith(pattern):
                            include_param = True
                            break

                if include_param:
                    for idx, grad_val in enumerate(param.grad.view(-
1).data):
                        param_name = f'{name}.{idx}'
                        if param_name not in gradient_collect:
                            gradient_collect[param_name] = {'x': [],
'y': []}
                        gradient_collect[param_name]
['y'].append(grad_val.item())
                        gradient_collect[param_name]['x'].append(x)

    # Plot gradients
    for param_name, data in gradient_collect.items():
        plt.plot(data['x'], data['y'], label=param_name)

    plt.xlabel('Data Points (X)')
    plt.ylabel('Gradient Values')
```

```
    if legend:
        plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
    plt.title(title)
    plt.tight_layout()
    plt.show()

# Visualize gradients for two-layer networks
for width in widths:
    net = nets_two_layer[width]['net']
    trained_net = nets_two_layer[width]['trained_net']

    print(f"=== Two-Layer Network Width {width} ===")

    # First layer weights and biases
    backward_and_plot_grad_enhanced(X, net, ['0.weight'], f'Random
Init - Layer 1 Weights (Width {width})')
    backward_and_plot_grad_enhanced(X, trained_net, ['0.weight'],
f'Trained - Layer 1 Weights (Width {width})')
    backward_and_plot_grad_enhanced(X, net, ['0.bias'], f'Random Init
- Layer 1 Bias (Width {width})')
    backward_and_plot_grad_enhanced(X, trained_net, ['0.bias'],
f'Trained - Layer 1 Bias (Width {width})')

    # Second layer weights and biases
    backward_and_plot_grad_enhanced(X, net, ['2.weight'], f'Random
Init - Layer 2 Weights (Width {width})')
    backward_and_plot_grad_enhanced(X, trained_net, ['2.weight'],
f'Trained - Layer 2 Weights (Width {width})')
    backward_and_plot_grad_enhanced(X, net, ['2.bias'], f'Random Init
- Layer 2 Bias (Width {width})')
    backward_and_plot_grad_enhanced(X, trained_net, ['2.bias'],
f'Trained - Layer 2 Bias (Width {width})')

# Enhanced SVD analysis function supporting multi-layer networks
def compute_svd_plot_features_enhanced(X, y, X_test, y_test, model,
title_suffix=''):
    """
    Enhanced SVD analysis function supporting multi-layer networks
    """
    gradient_collect = {}

    for x in X:
        y_pred = model(to_torch(x).to(device=device))

        # Clear gradients and run backpropagation
        model.zero_grad()
        y_pred.backward()

        # Collect gradients from all parameters
        for name, param in model.named_parameters():
```

```
            if param.grad is not None:
                for idx, grad_val in enumerate(param.grad.view(-
1).data):
                    param_name = f'{name}.{idx}'
                    if param_name not in gradient_collect:
                        gradient_collect[param_name] = {'x': [], 'y':
[]}
                    gradient_collect[param_name]
['y'].append(grad_val.item())
                    gradient_collect[param_name]['x'].append(x)

    # Build feature matrix
    feature_matrix = []
    for param_name in gradient_collect:
        feature_matrix.append(gradient_collect[param_name]['y'])

    feature_matrix = np.array(feature_matrix).T

    # Perform SVD decomposition
    u, s, vh = np.linalg.svd(feature_matrix, full_matrices=False)

    # Plot singular values
    plt.figure(figsize=(10, 5))
    plt.scatter(np.arange(s.shape[0]), s, c='darkorange', s=40.0,
label='Singular Values')
    plt.xlabel('Component Index')
    plt.ylabel('Singular Value')
    plt.title(f'Singular Values {title_suffix}')
    plt.yscale('log')
    plt.legend()
    plt.show()

    # Compute principal features
    principal_features = u @ np.diag(s)

    # Plot principal features
    plt.figure(figsize=(12, 8))
    n_components_to_plot = min(10, feature_matrix.shape[1])
    for idx in range(n_components_to_plot):
        plt.plot(X, principal_features[:, idx], label=f'Component
{idx+1}')

    plt.xlabel('Data Points (X)')
    plt.ylabel('Principal Feature Value')
    plt.title(f'Principal Features {title_suffix}')
    plt.legend()
    plt.show()

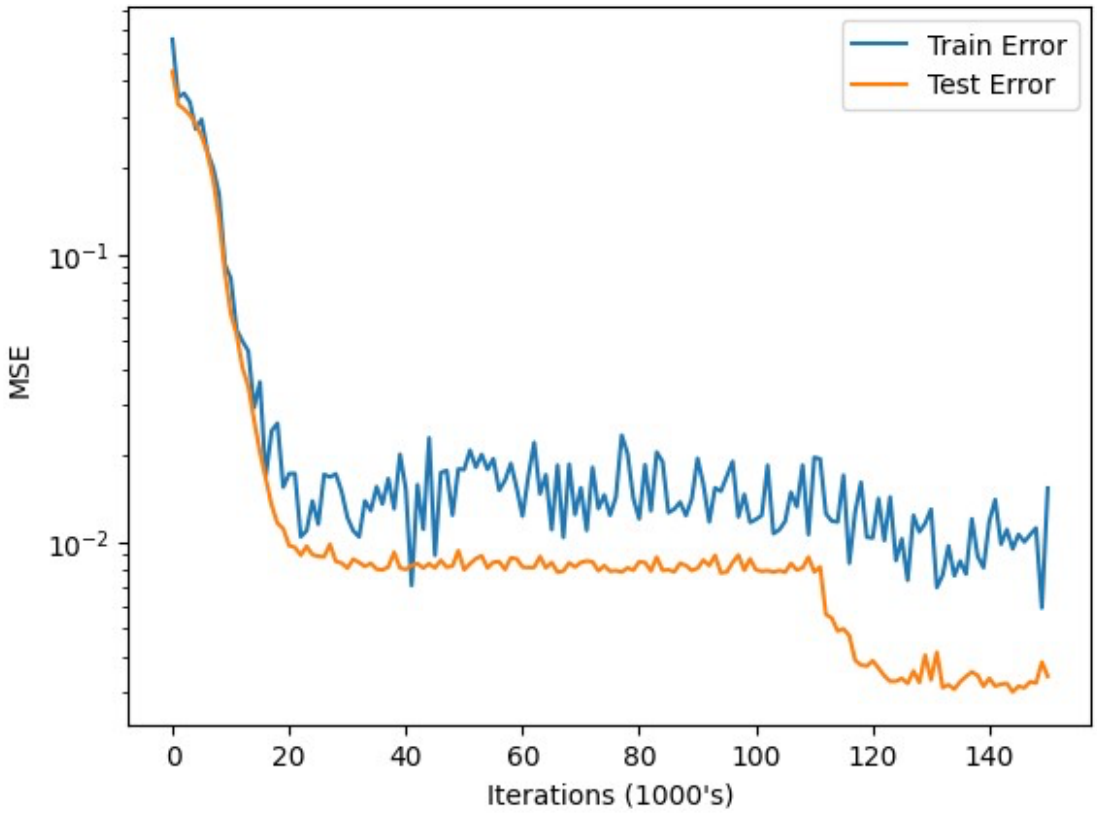
# Perform SVD analysis on two-layer networks
for w in widths:
```

```
    net = nets_two_layer[w]['net']
    trained_net = nets_two_layer[w]['trained_net']

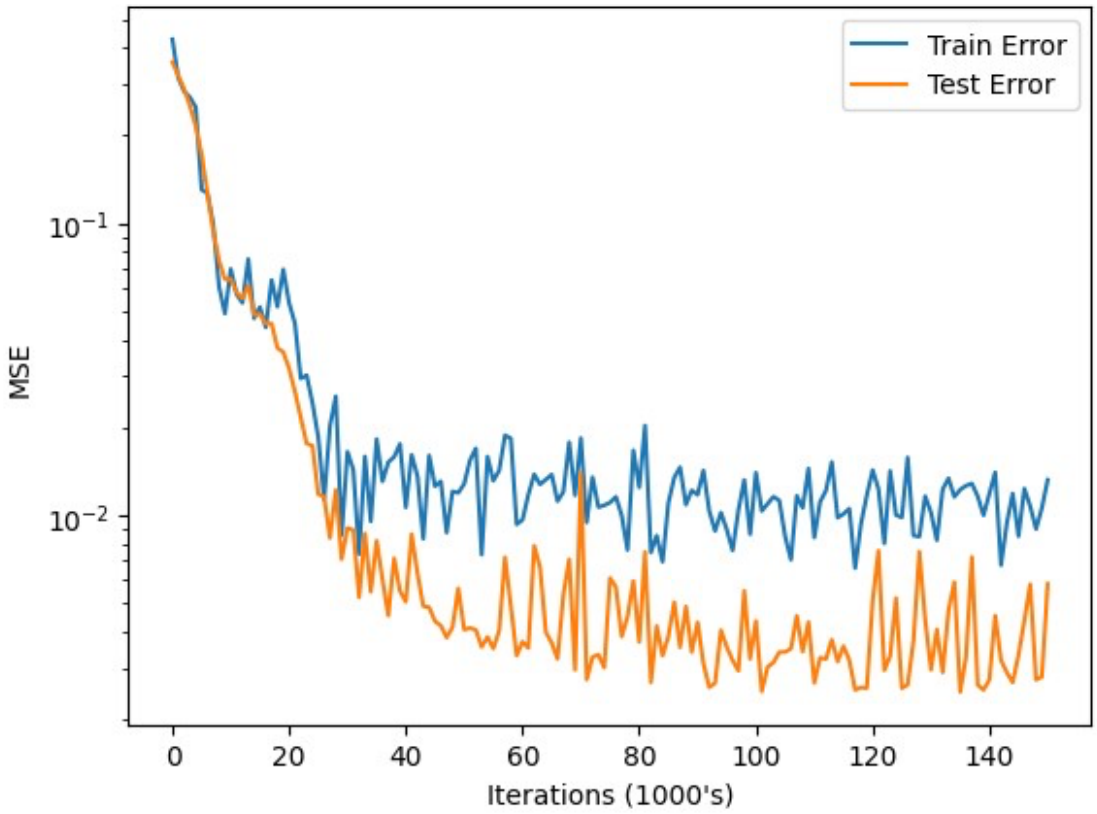
    print(f"=== Two-Layer Network Width {w} - Random Initialization
===")
    compute_svd_plot_features_enhanced(X, y, X_test, y_test, net,
f'(Width {w}, Random Init)')

    print(f"=== Two-Layer Network Width {w} - After Training ===")
    compute_svd_plot_features_enhanced(X, y, X_test, y_test,
trained_net, f'(Width {w}, Trained)')
```

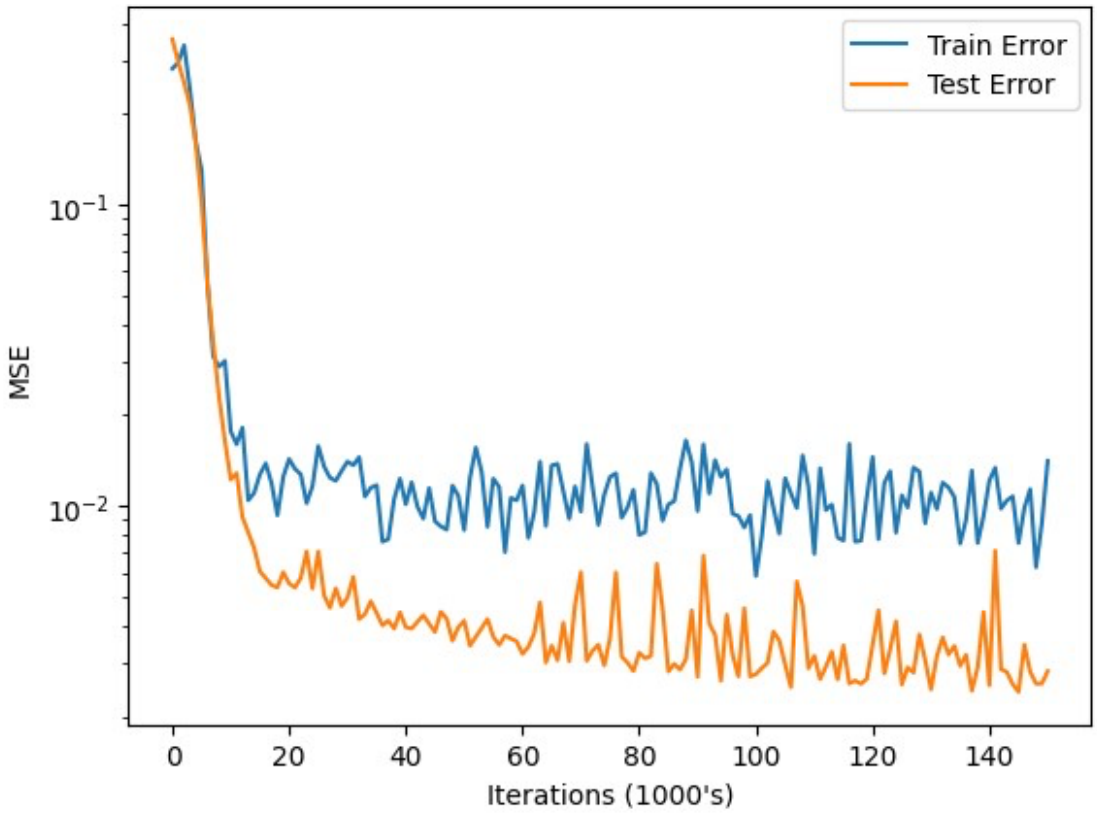
```
----------------------------------------
Two-Layer Width 10
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Two-Layer Width 10
```



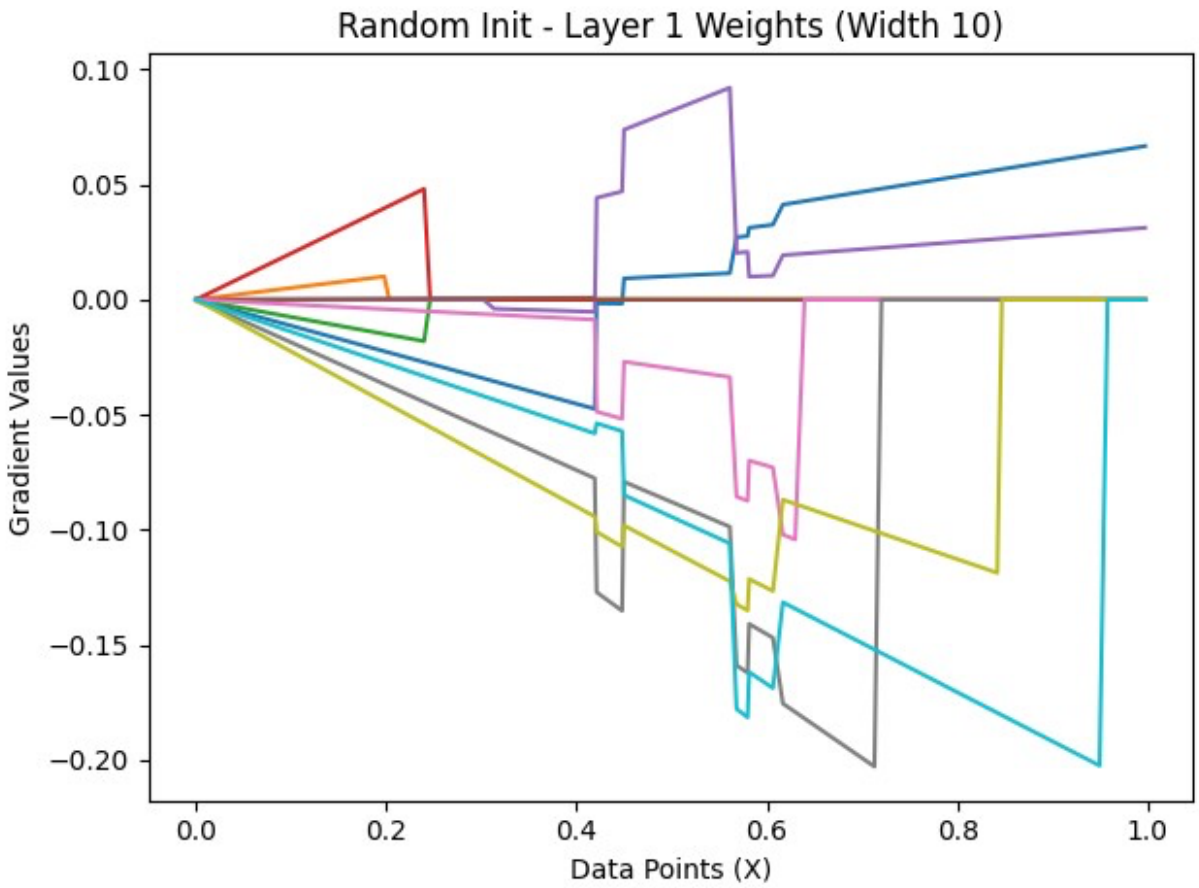

```
----------------------------------------
Two-Layer Width 20
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Two-Layer Width 20
```

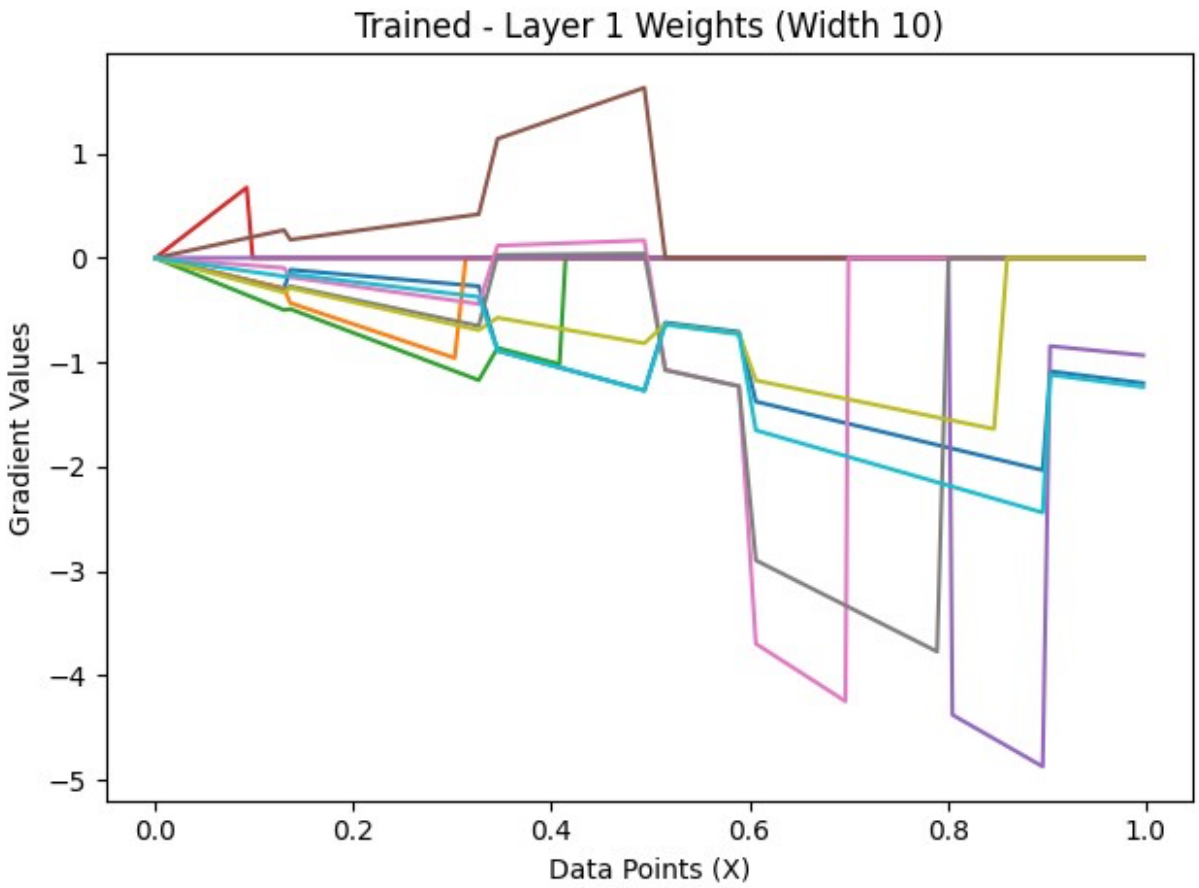


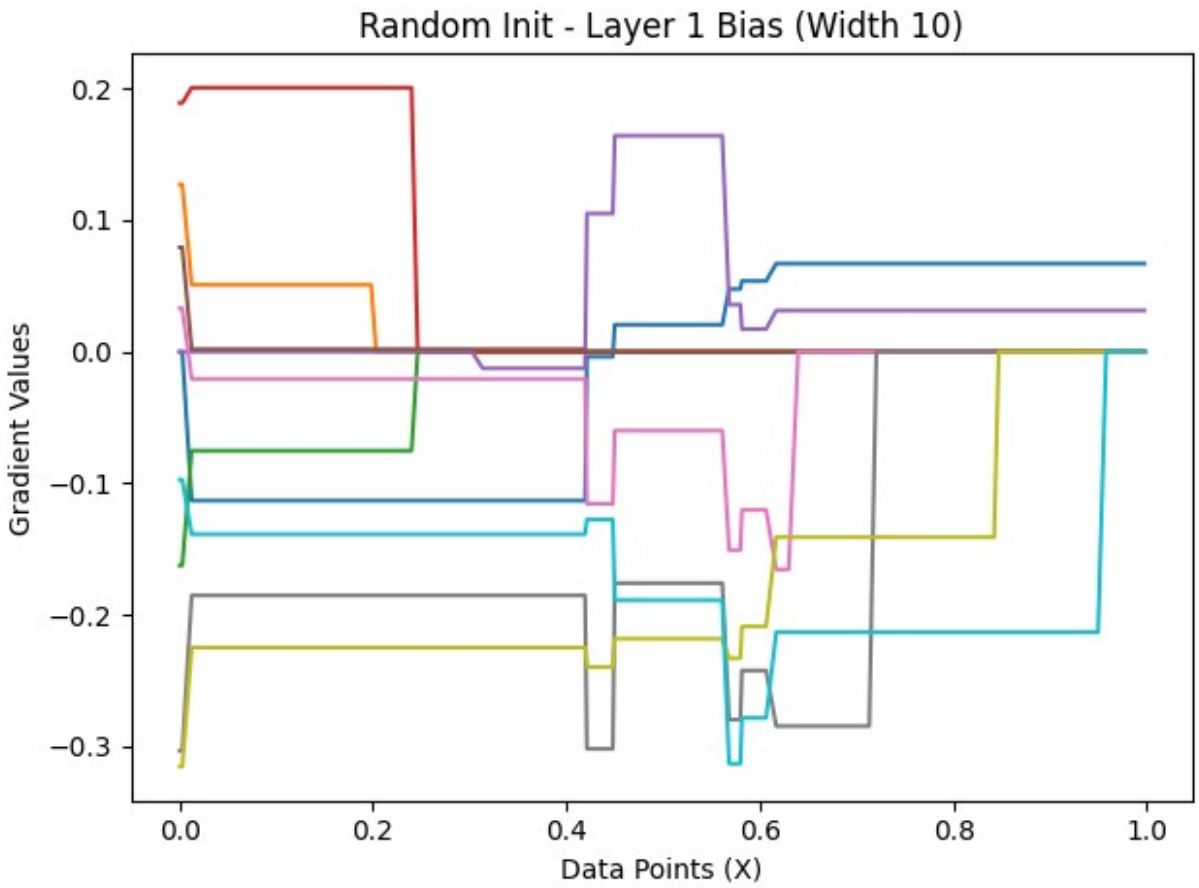

```
----------------------------------------
Two-Layer Width 40
SGD Iteration 10000
SGD Iteration 20000
SGD Iteration 30000
SGD Iteration 40000
SGD Iteration 50000
SGD Iteration 60000
SGD Iteration 70000
SGD Iteration 80000
SGD Iteration 90000
SGD Iteration 100000
SGD Iteration 110000
SGD Iteration 120000
SGD Iteration 130000
SGD Iteration 140000
SGD Iteration 150000
Two-Layer Width 40
```

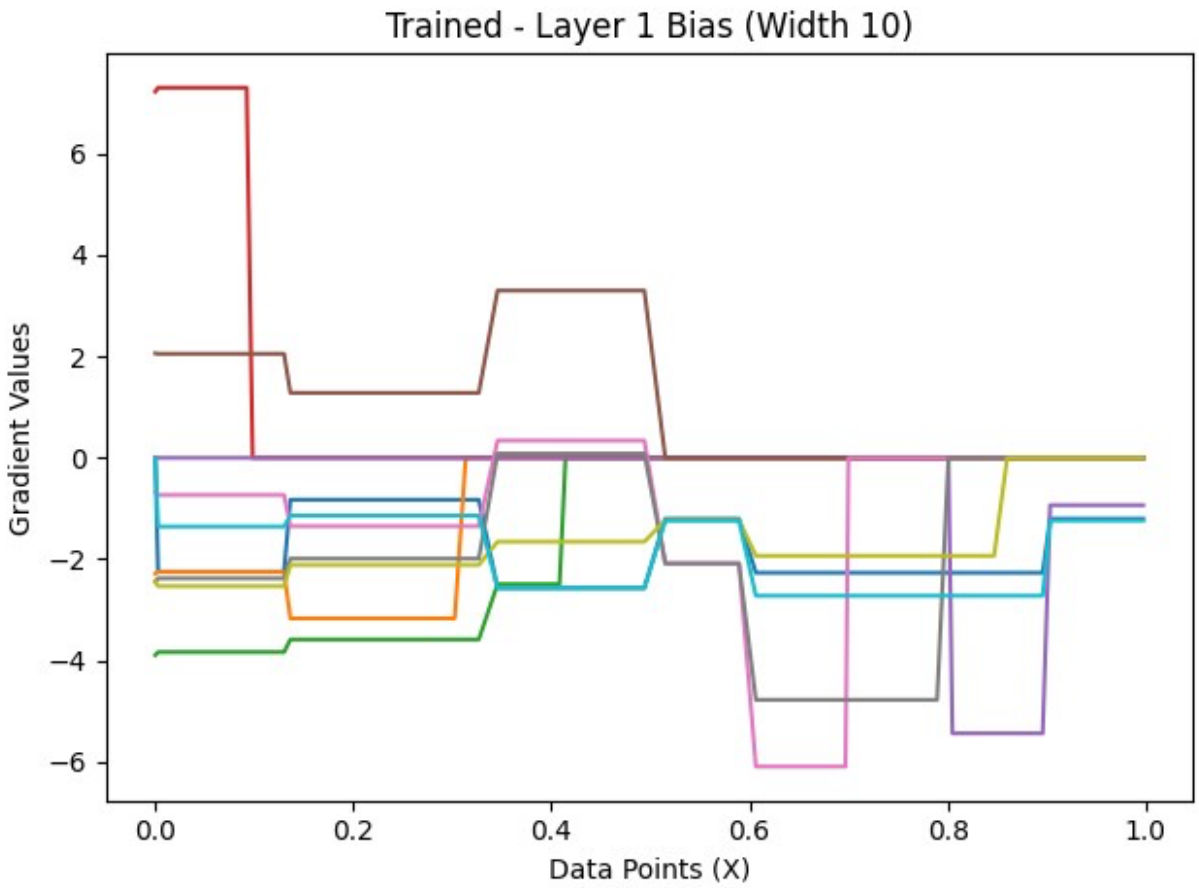


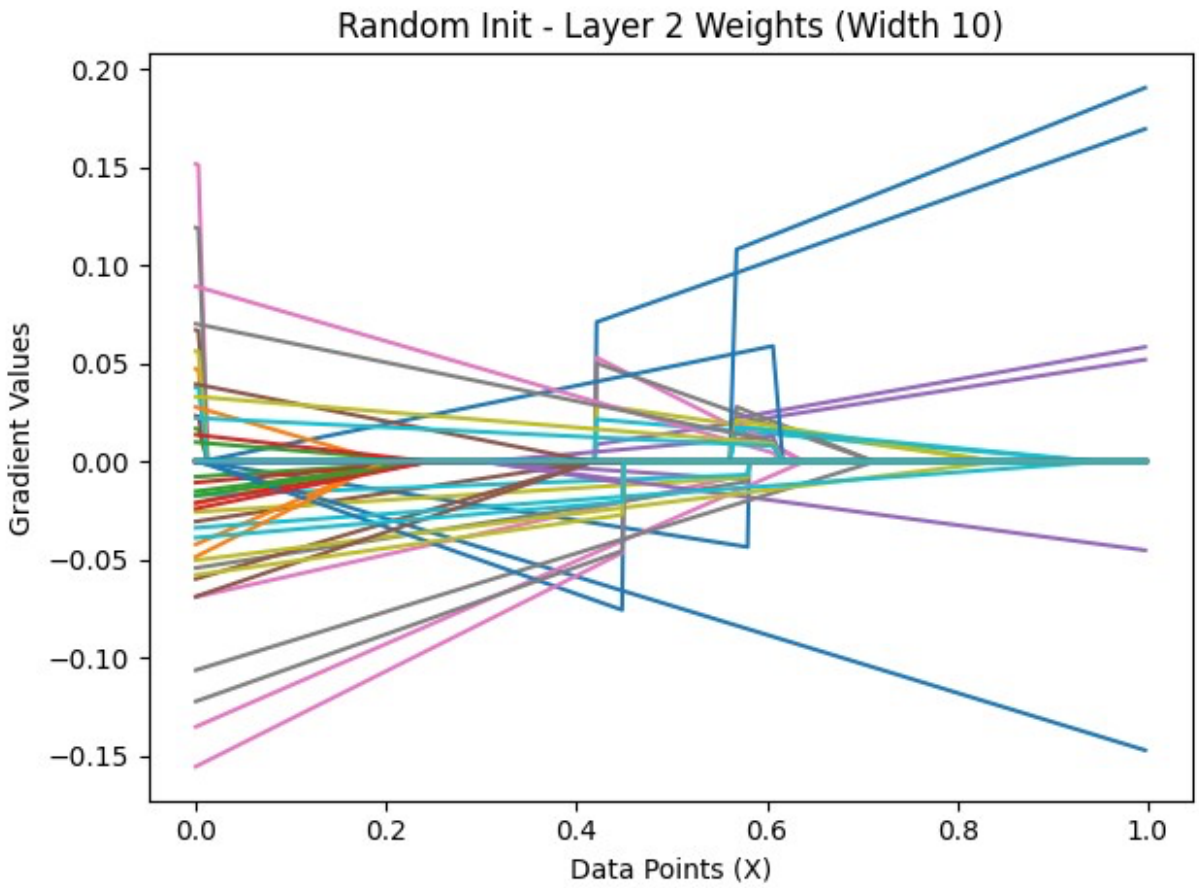

```
=== Two-Layer Network Width 10 ===
```

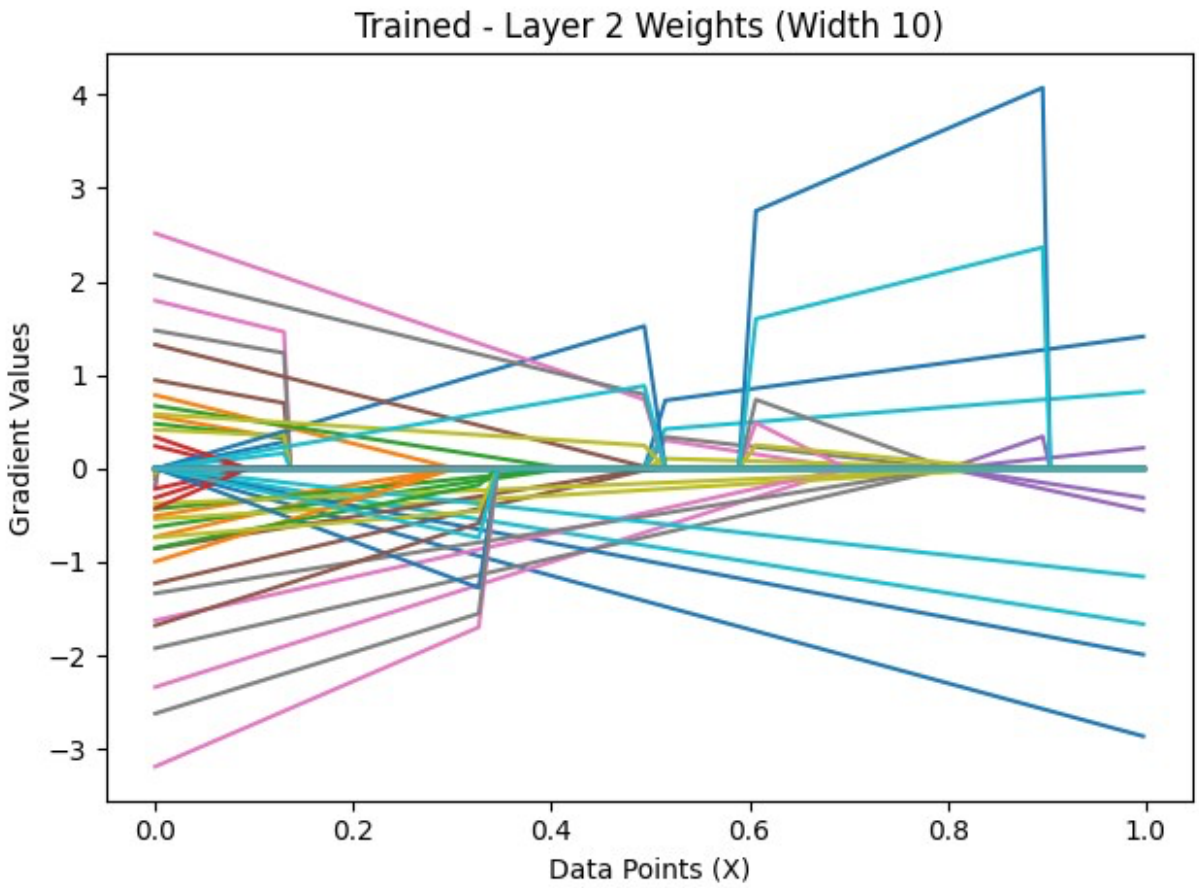

Random Init - Layer 1 Weights (Width 10)
Gradient Values
Data Points (X)
0.10
0.05
0.00
−0.05
−0.10
−0.15
−0.20
0.0
0.2
0.4
0.6
0.8
1.0

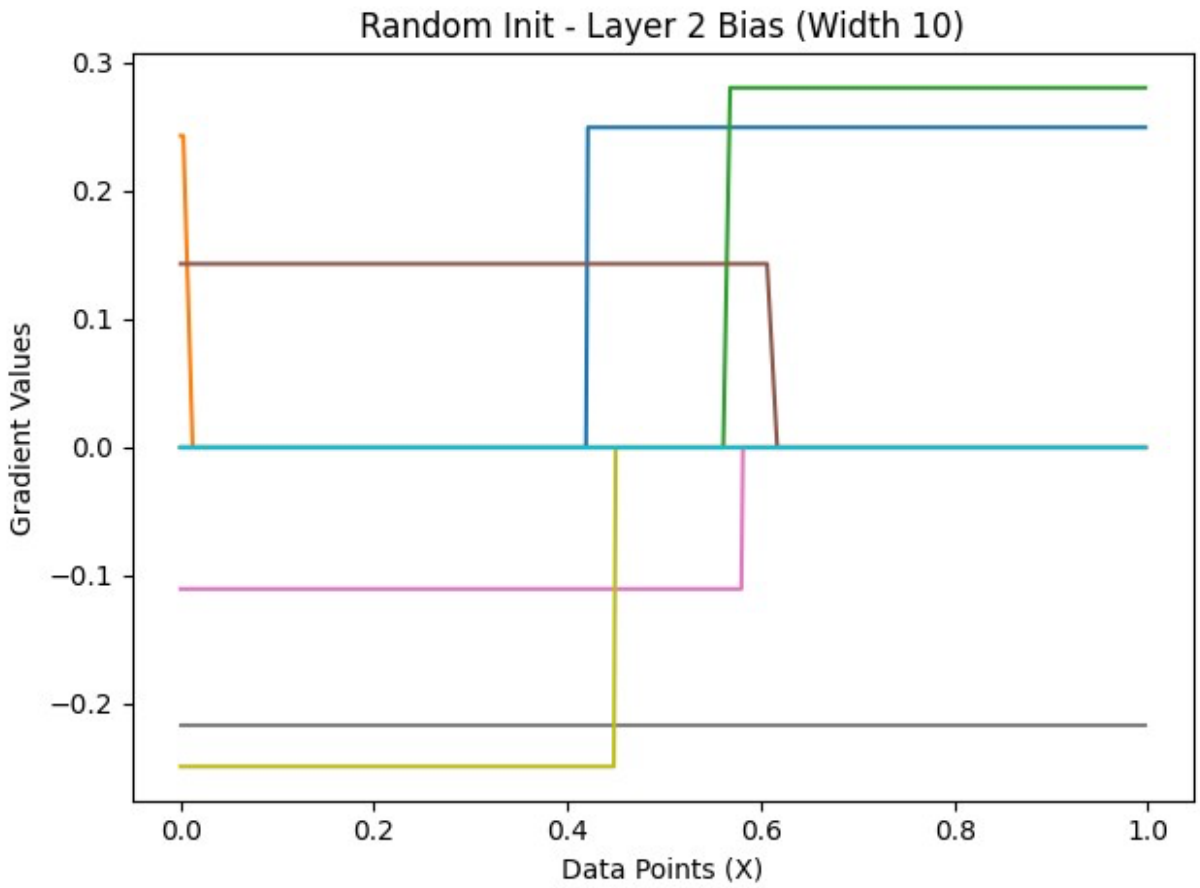

Trained - Layer 1 Weights (Width 10)
Gradient Values
1
0
−1
−2
−3
−4
−5
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

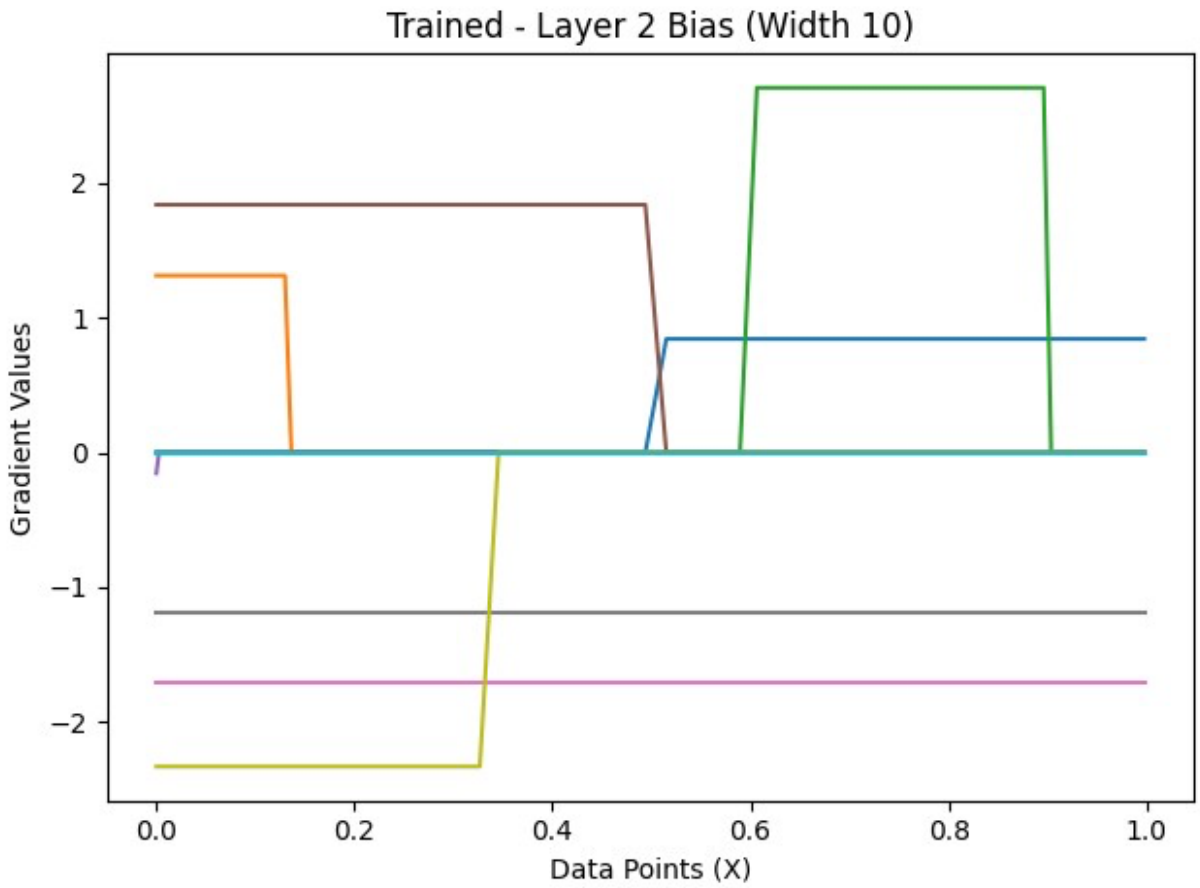

Random Init - Layer 1 Bias (Width 10)
Gradient Values
Data Points (X)
0.2
0.1
0.0
−0.1
−0.2
−0.3
0.0
0.2
0.4
0.6
0.8
1.0


Trained - Layer 1 Bias (Width 10)
Gradient Values
6
4
2
0
−2
−4
−6
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

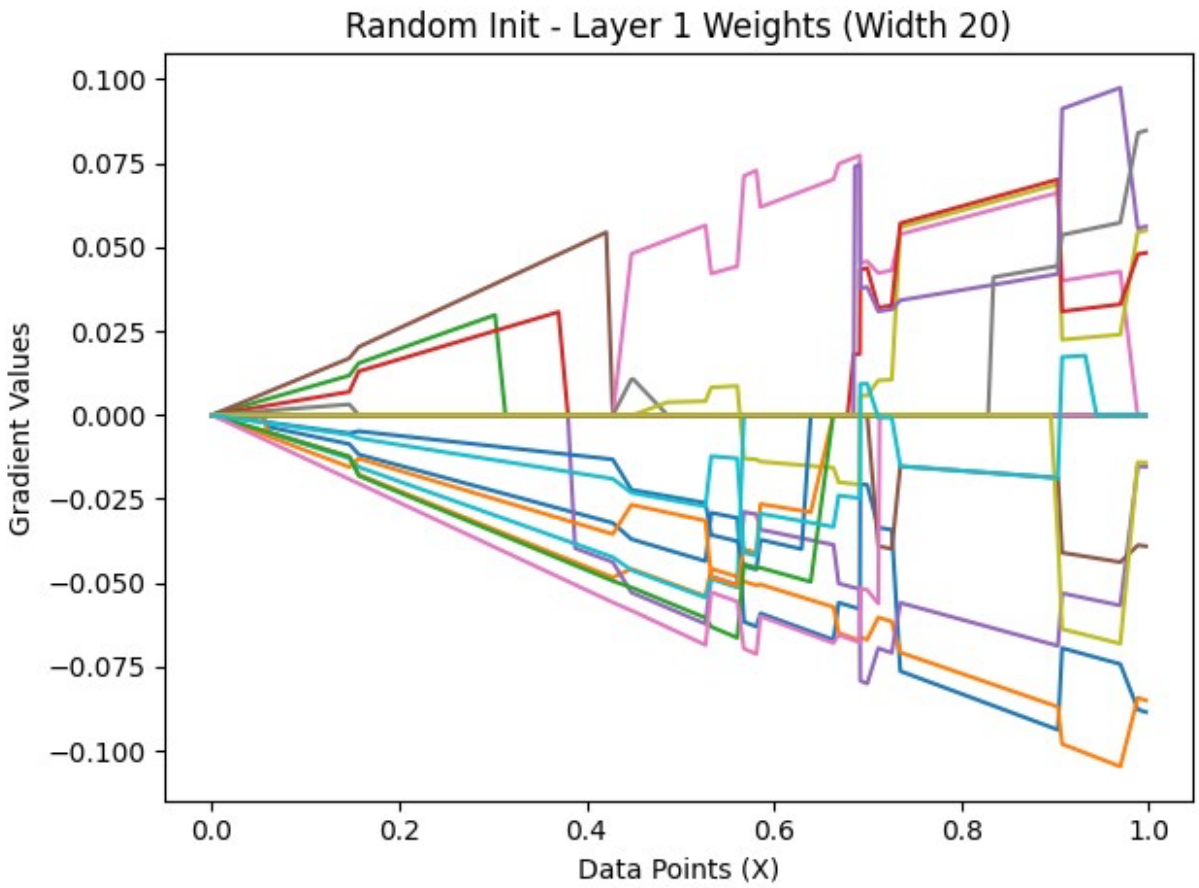

Random Init - Layer 2 Weights (Width 10)
Gradient Values
Data Points (X)
0.20
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0

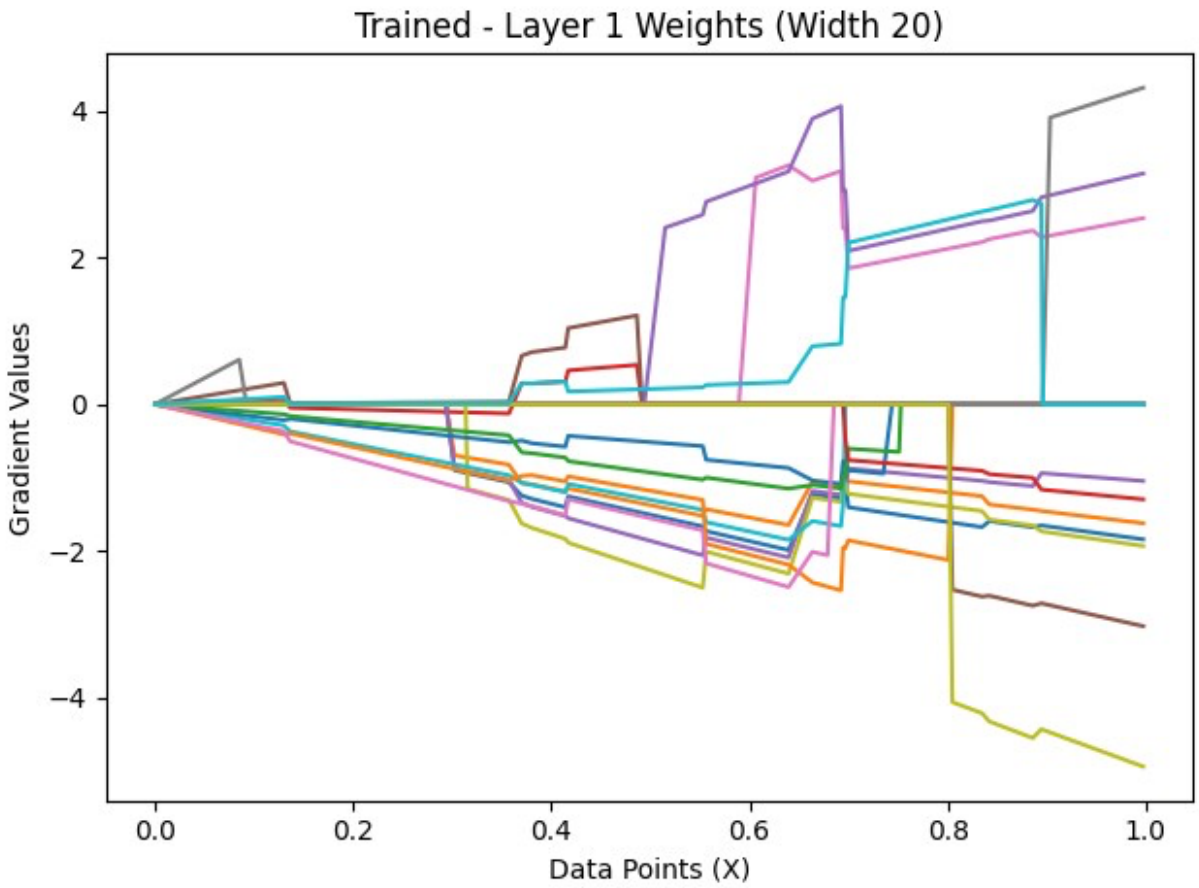

Trained - Layer 2 Weights (Width 10)
Gradient Values
Data Points (X)
4
3
2
1
0
−1
−2
−3
0.0
0.2
0.4
0.6
0.8
1.0

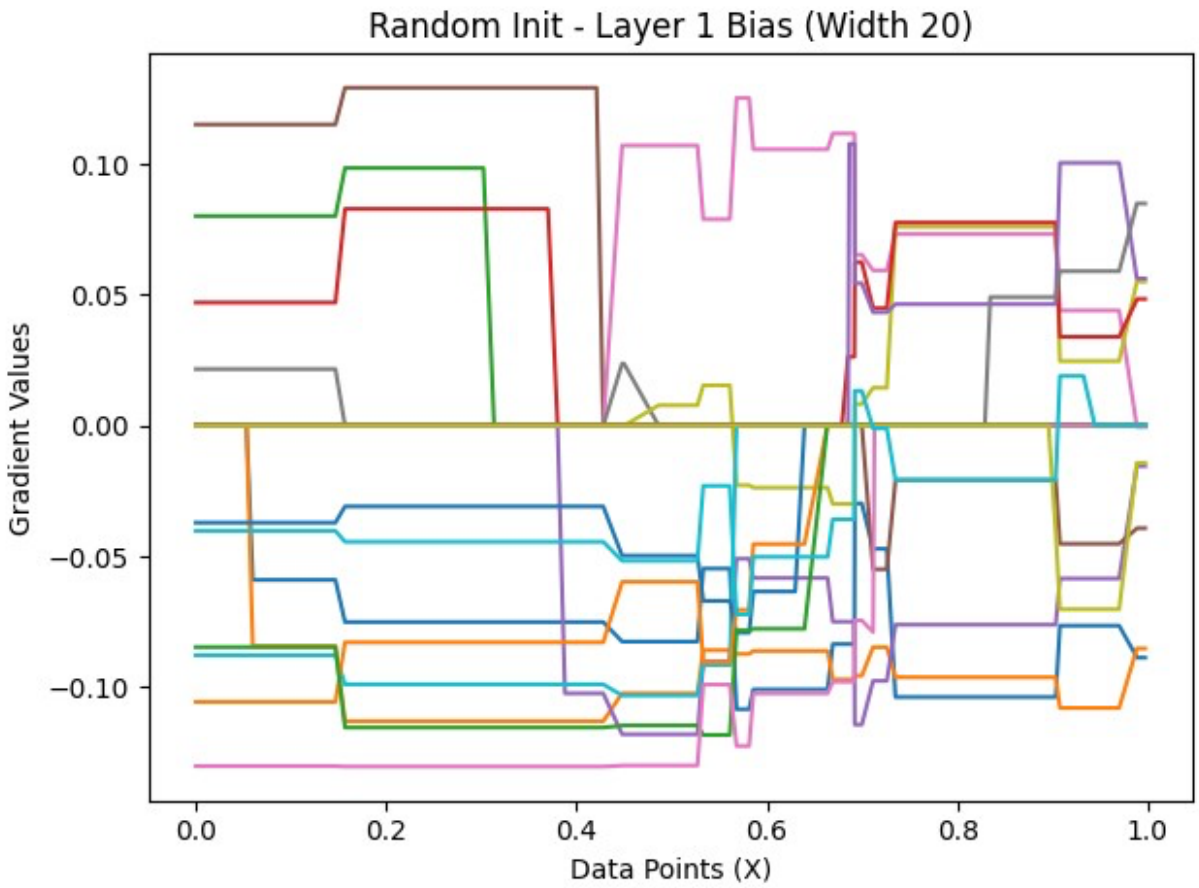

Random Init - Layer 2 Bias (Width 10)
Gradient Values
Data Points (X)
0.3
0.2
0.1
0.0
−0.1
−0.2
0.0
0.2
0.4
0.6
0.8
1.0

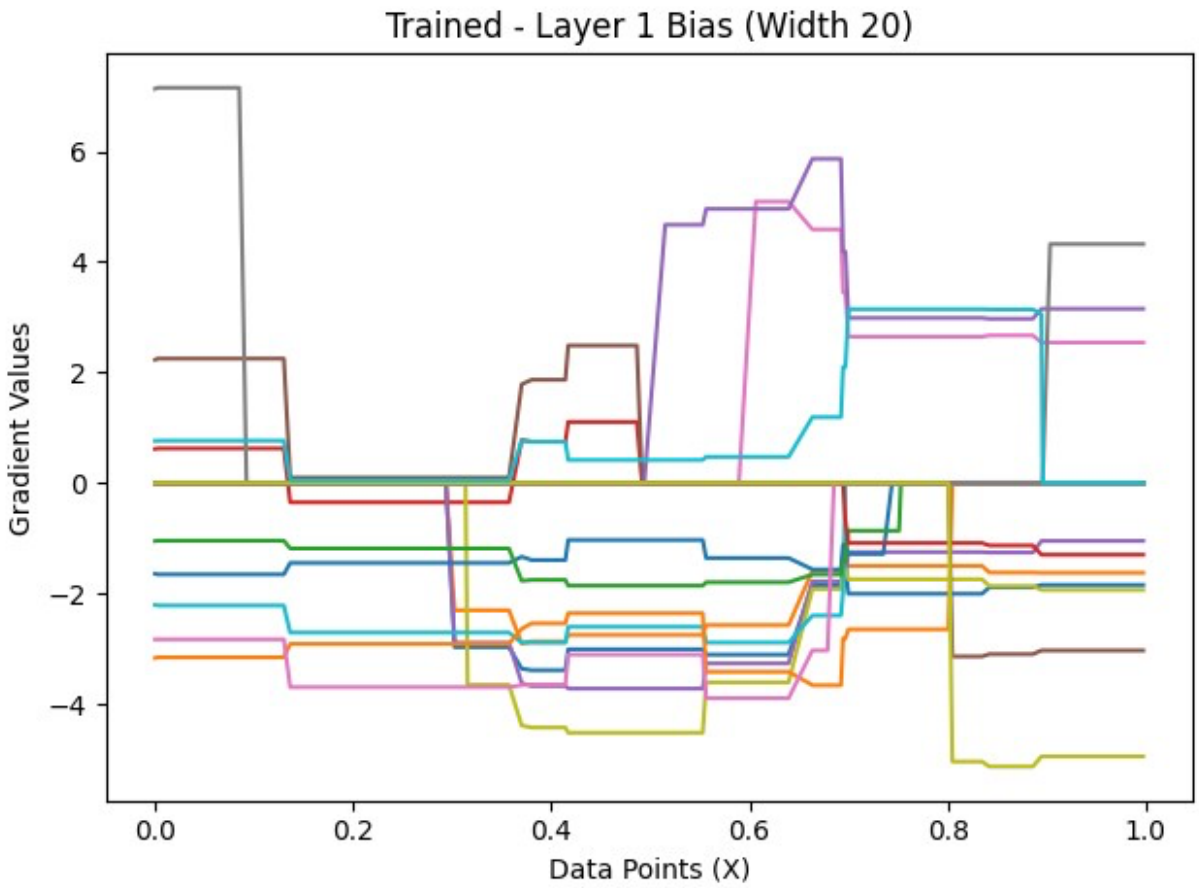


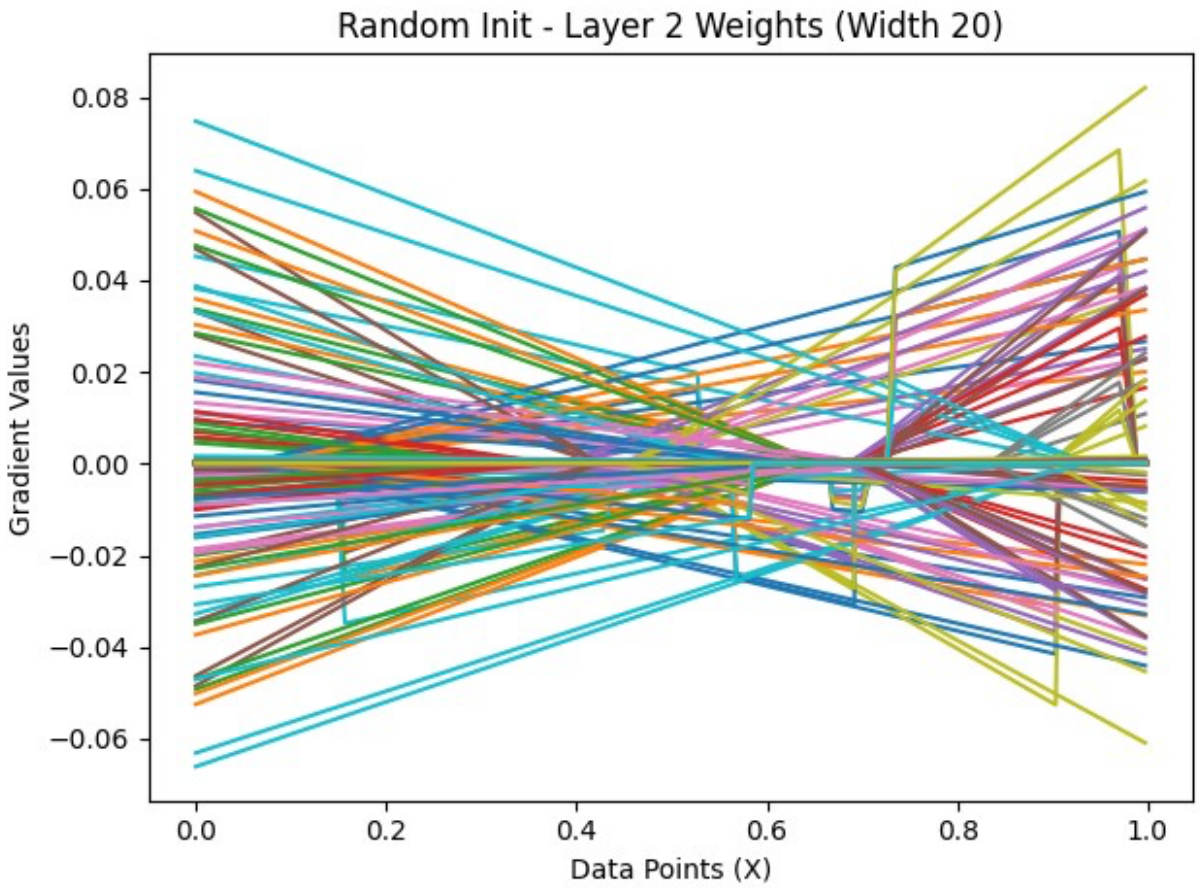

```
=== Two-Layer Network Width 20 ===
```

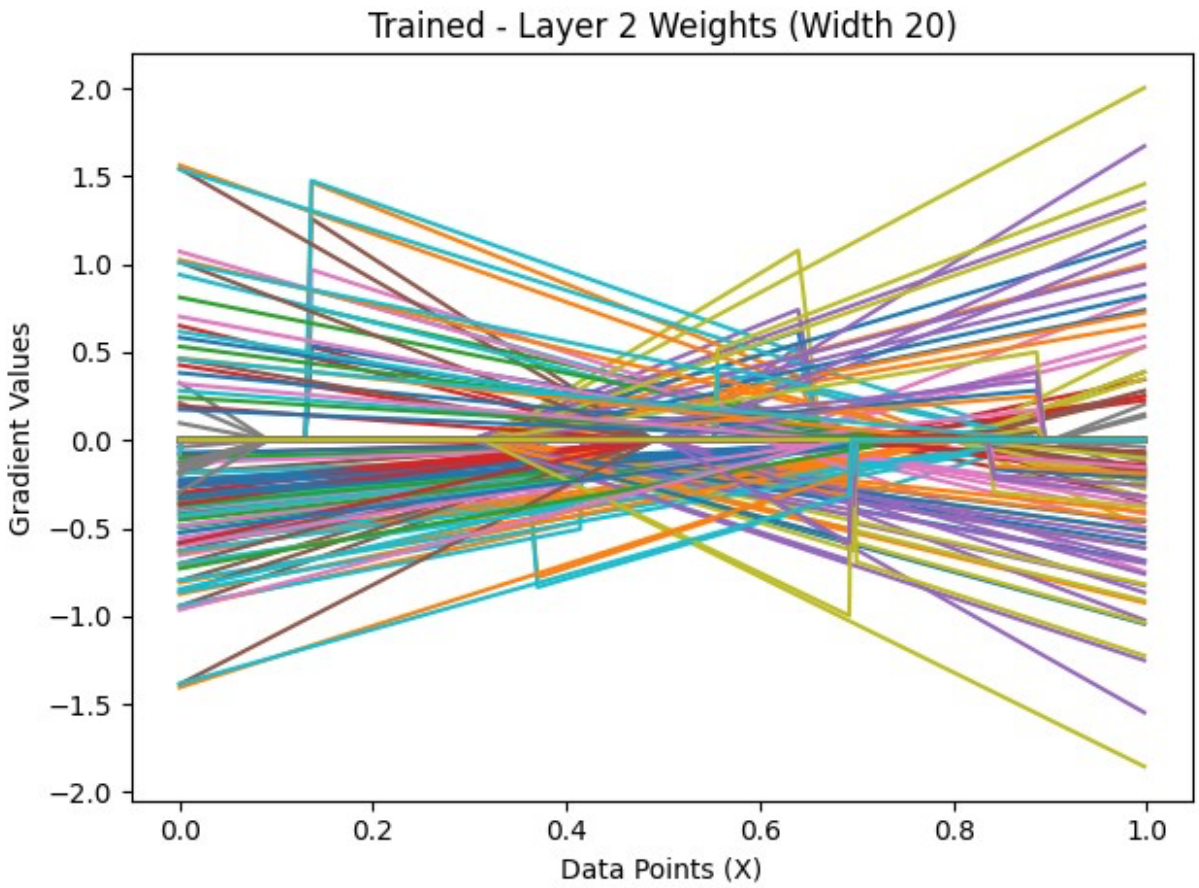

Random Init - Layer 1 Weights (Width 20)
Gradient Values
0.100
0.075
0.050
0.025
0.000
−0.025
−0.050
−0.075
−0.100
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

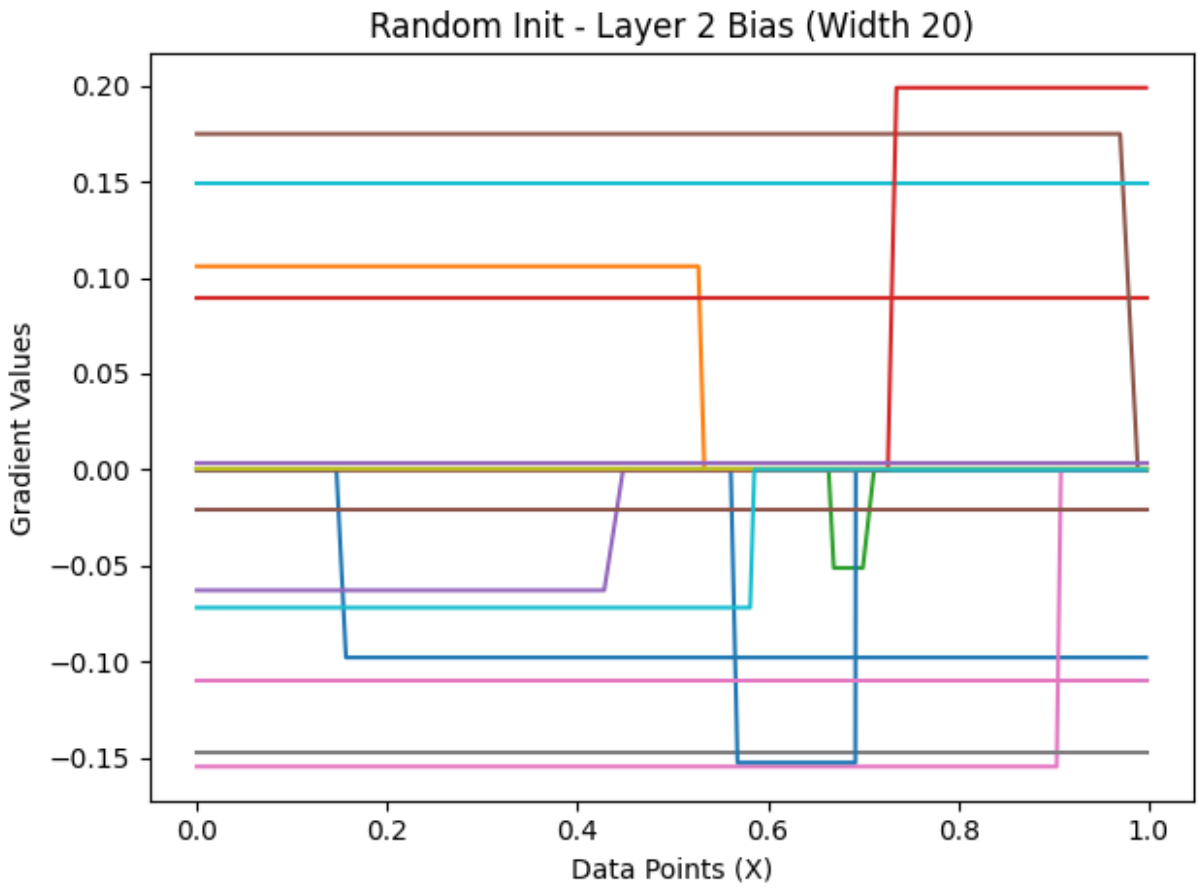

Trained - Layer 1 Weights (Width 20)
Gradient Values
4
2
0
−2
−4
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

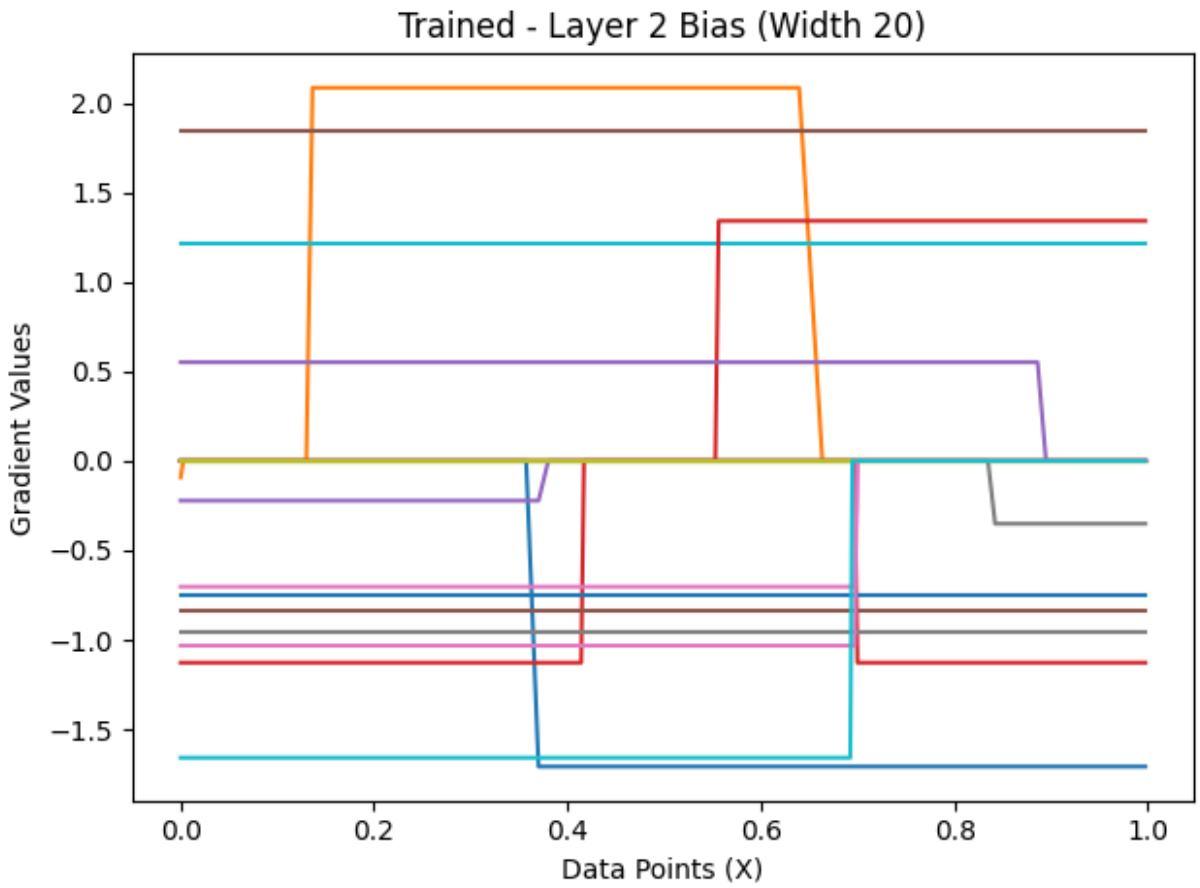

Random Init - Layer 1 Bias (Width 20)
Gradient Values
0.10
0.05
0.00
−0.05
−0.10
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)


Trained - Layer 1 Bias (Width 20)
Gradient Values
6
4
2
0
−2
−4
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

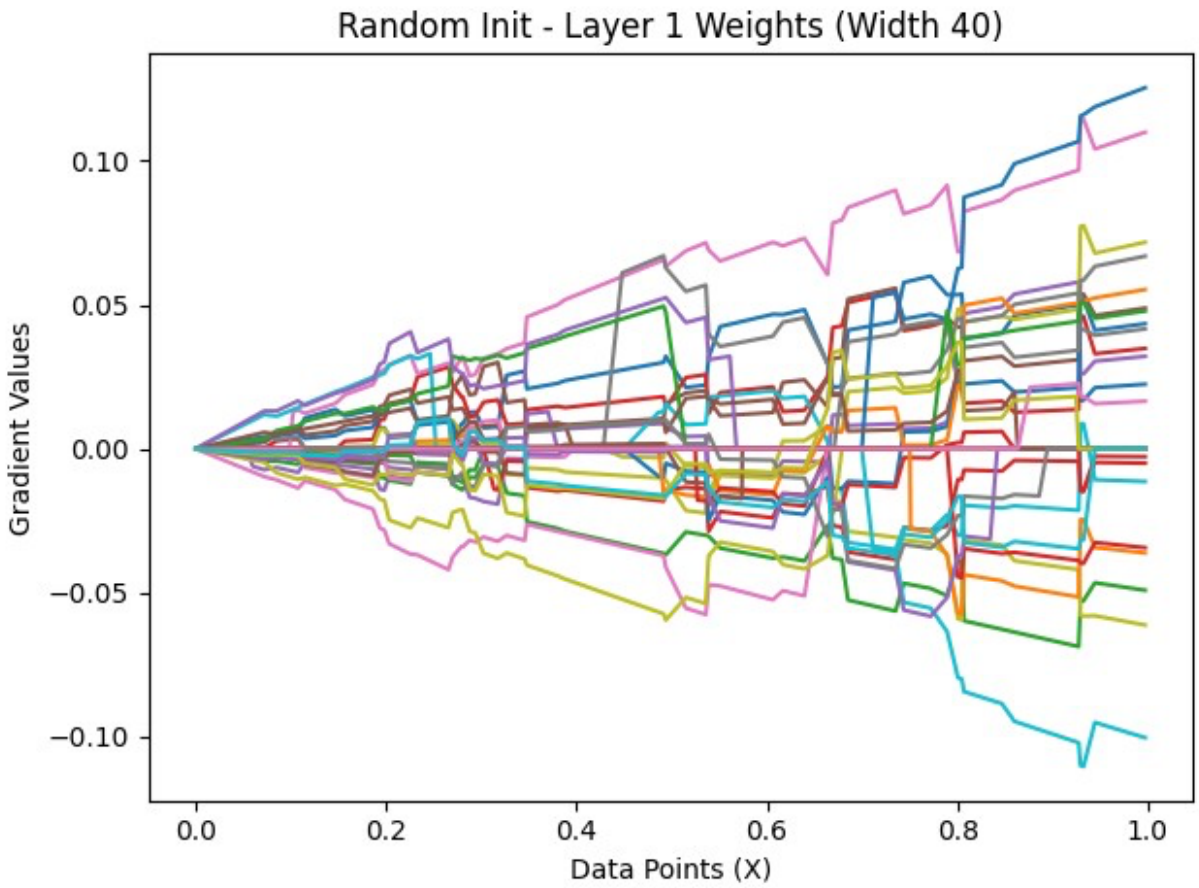

Random Init - Layer 2 Weights (Width 20)
Gradient Values
0.08
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

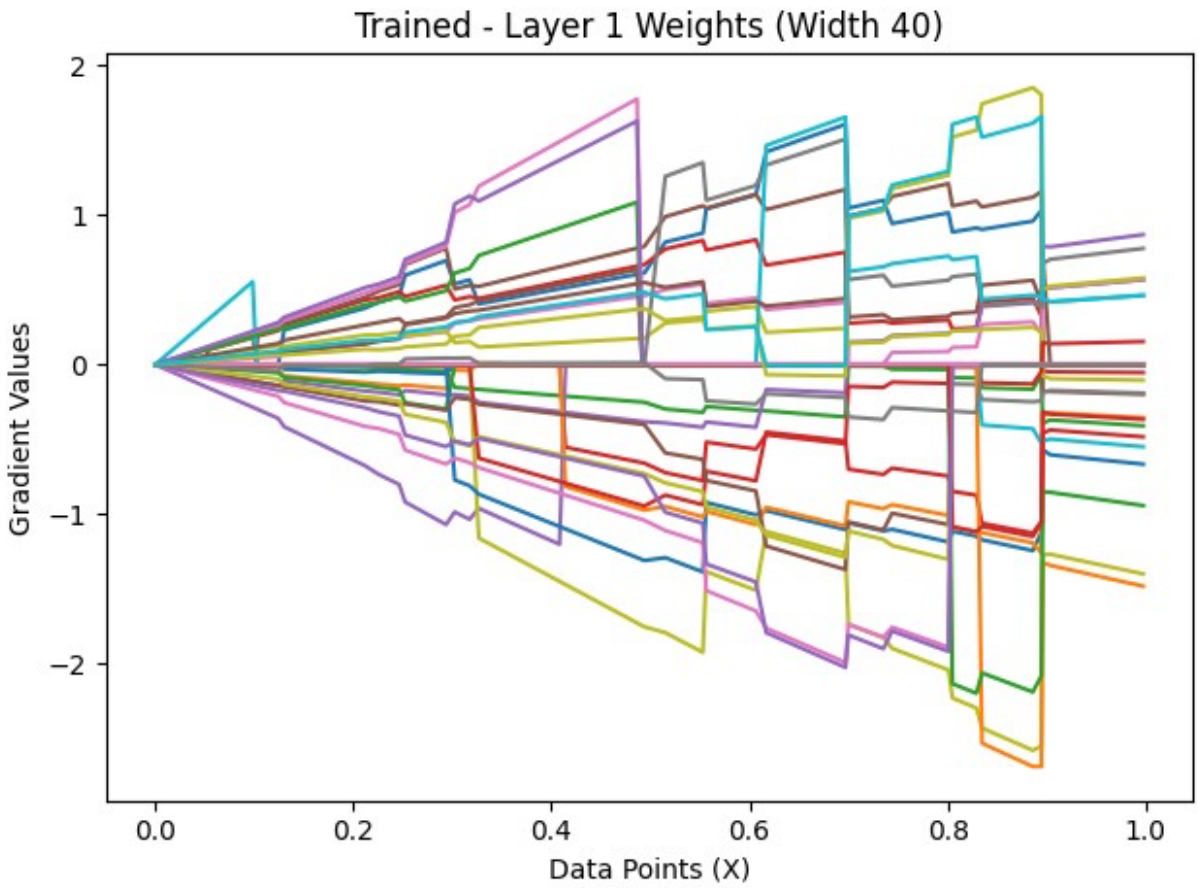

Trained - Layer 2 Weights (Width 20)
Gradient Values
Data Points (X)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
0.0
0.2
0.4
0.6
0.8
1.0

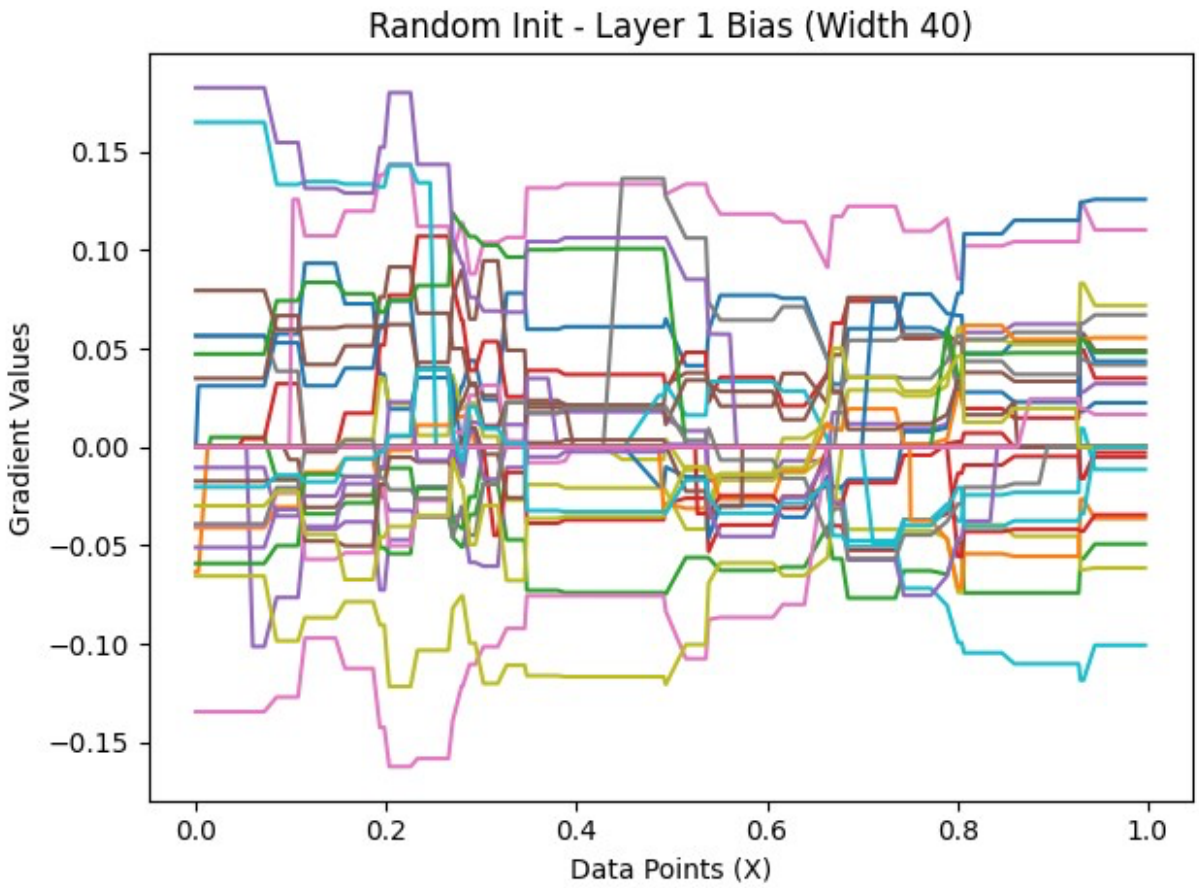

Random Init - Layer 2 Bias (Width 20)
Gradient Values
Data Points (X)
0.20
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0

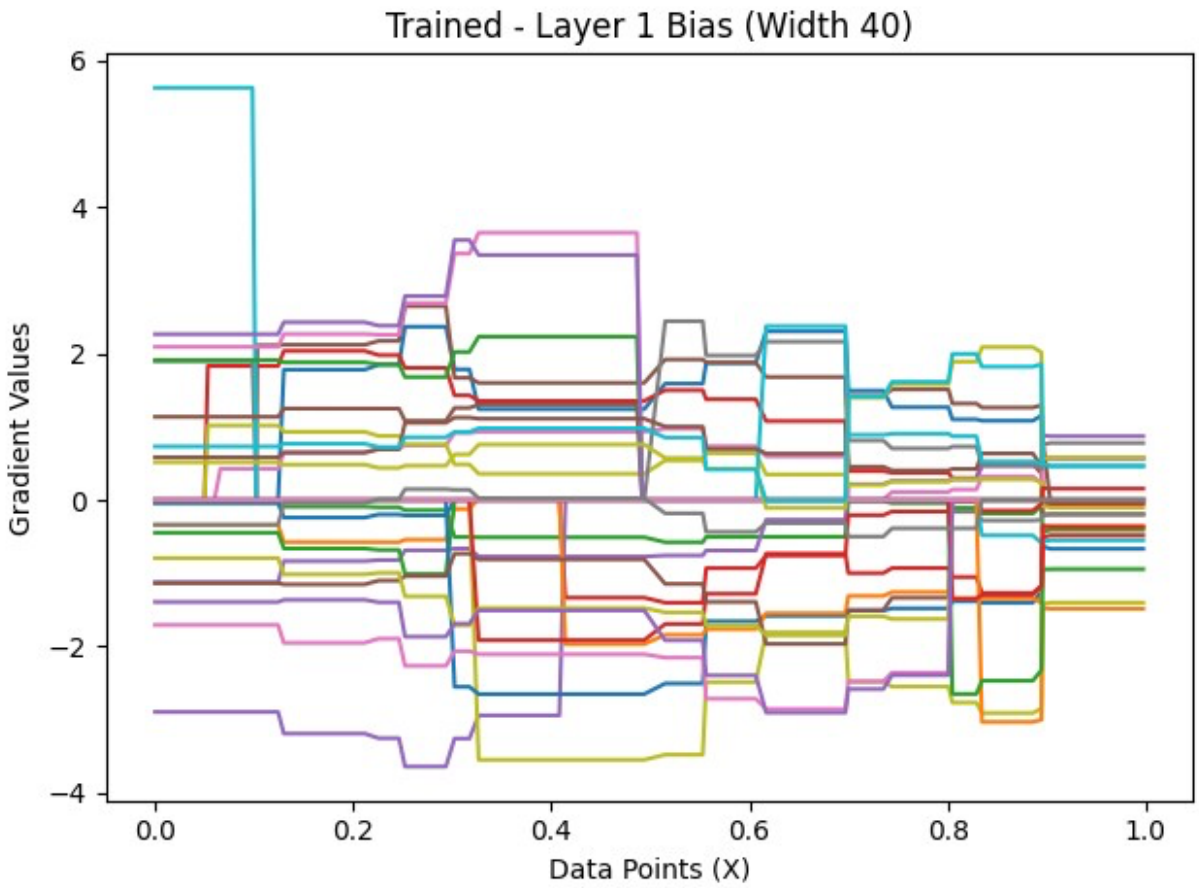


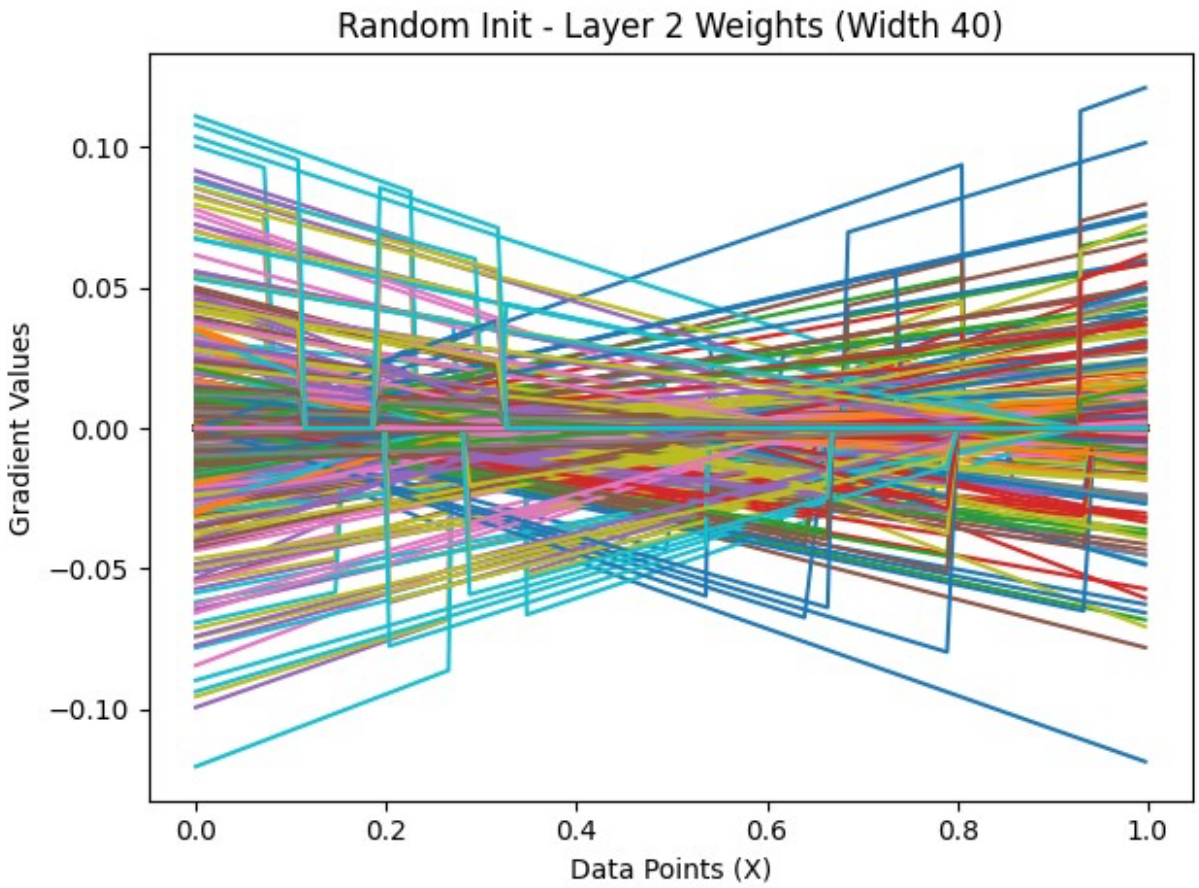

```
=== Two-Layer Network Width 40 ===
```

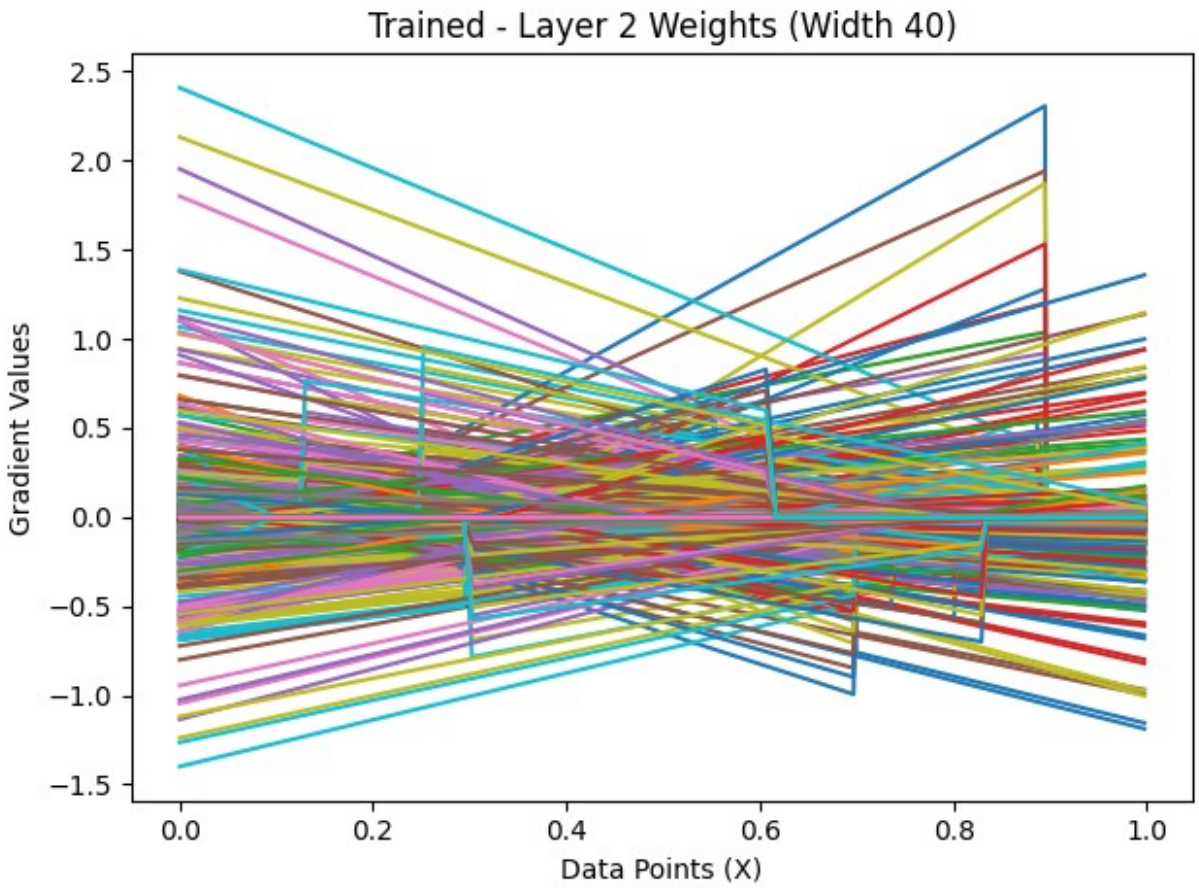

Random Init - Layer 1 Weights (Width 40)
Gradient Values
0.10
0.05
0.00
−0.05
−0.10
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

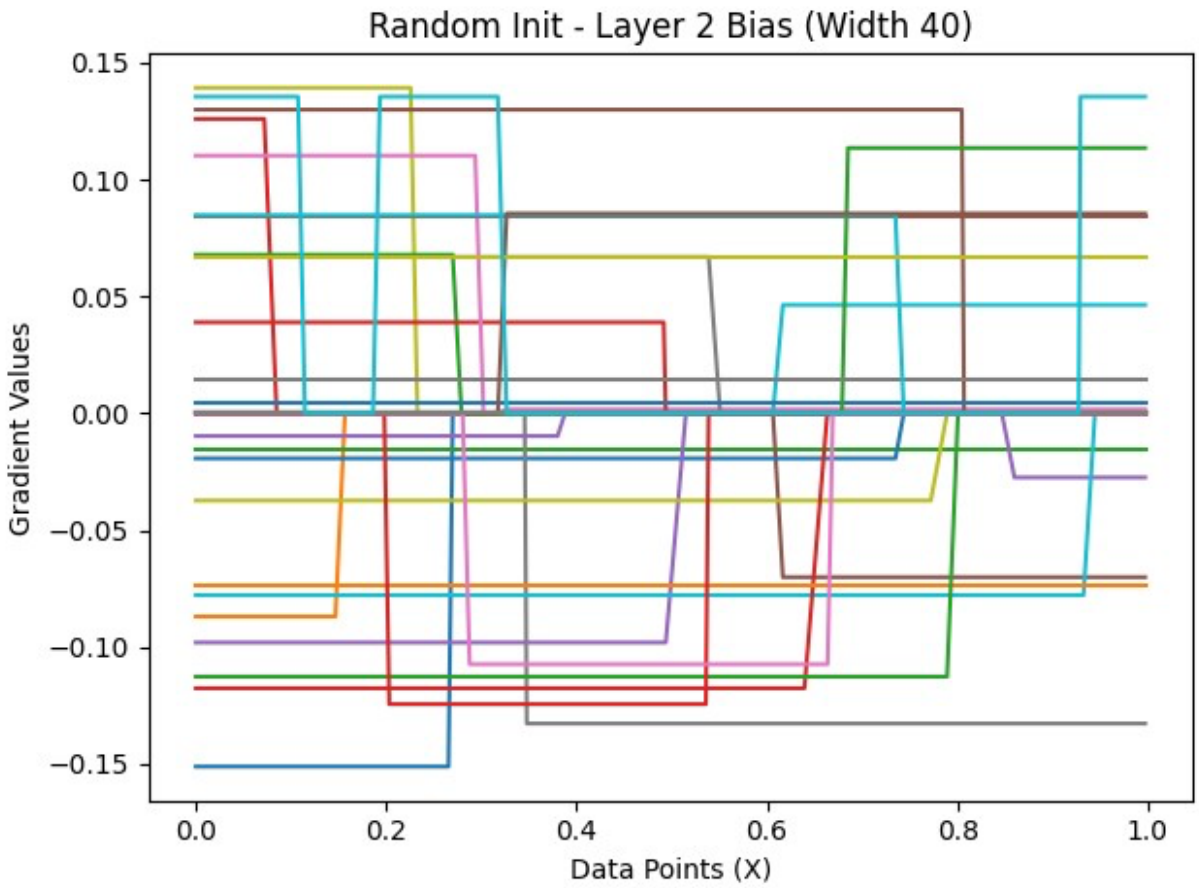

Trained - Layer 1 Weights (Width 40)
Gradient Values
Data Points (X)
2
1
0
−1
−2
0.0
0.2
0.4
0.6
0.8
1.0

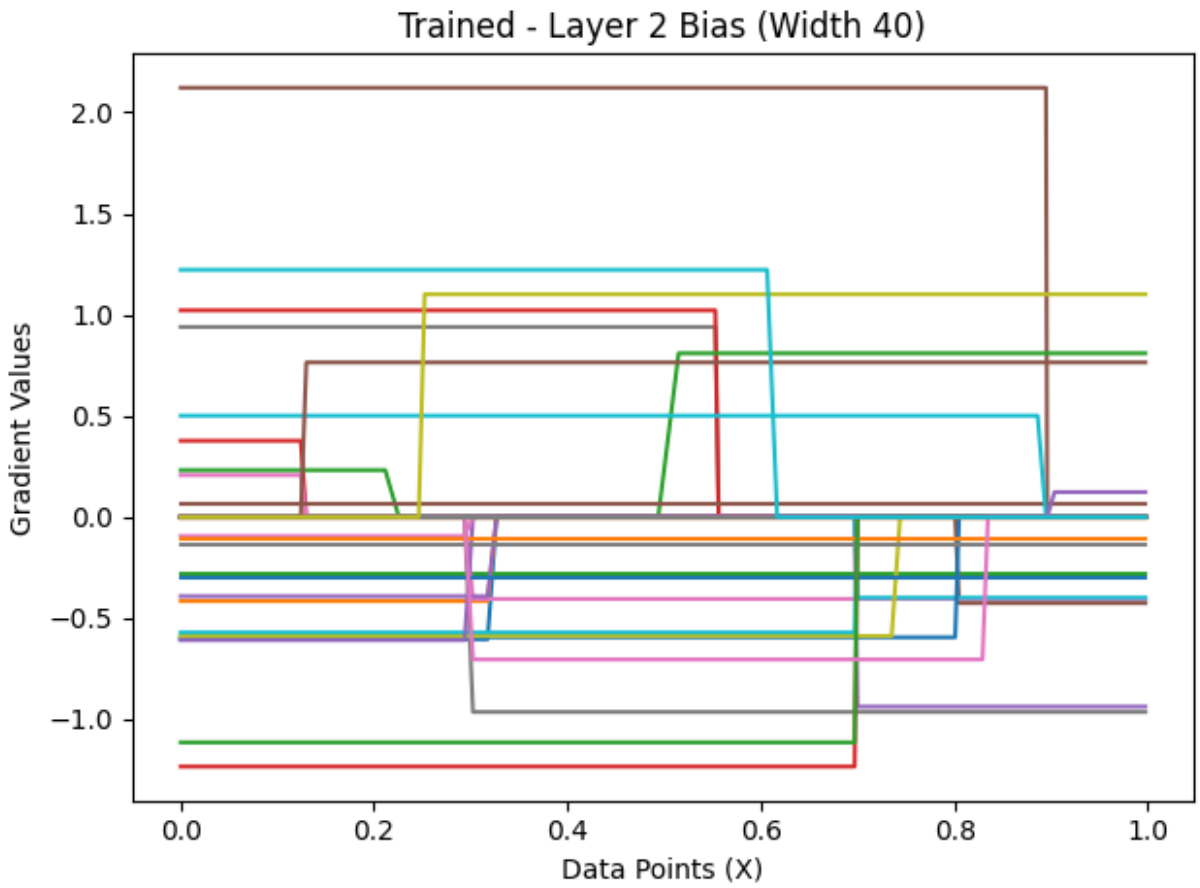

Random Init - Layer 1 Bias (Width 40)
Gradient Values
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)


Trained - Layer 1 Bias (Width 40)
Gradient Values
Data Points (X)
6
4
2
0
−2
−4
0.0
0.2
0.4
0.6
0.8
1.0

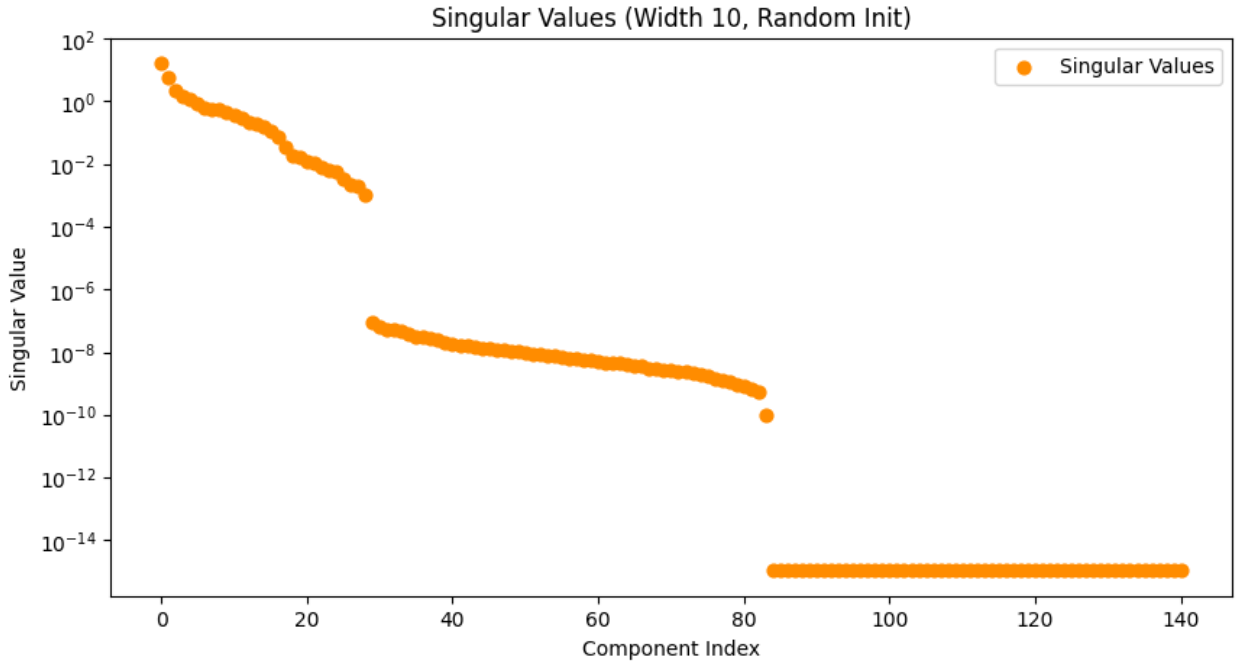

Random Init - Layer 2 Weights (Width 40)
Gradient Values
0.10
0.05
0.00
−0.05
−0.10
0.0
0.2
0.4
0.6
0.8
1.0
Data Points (X)

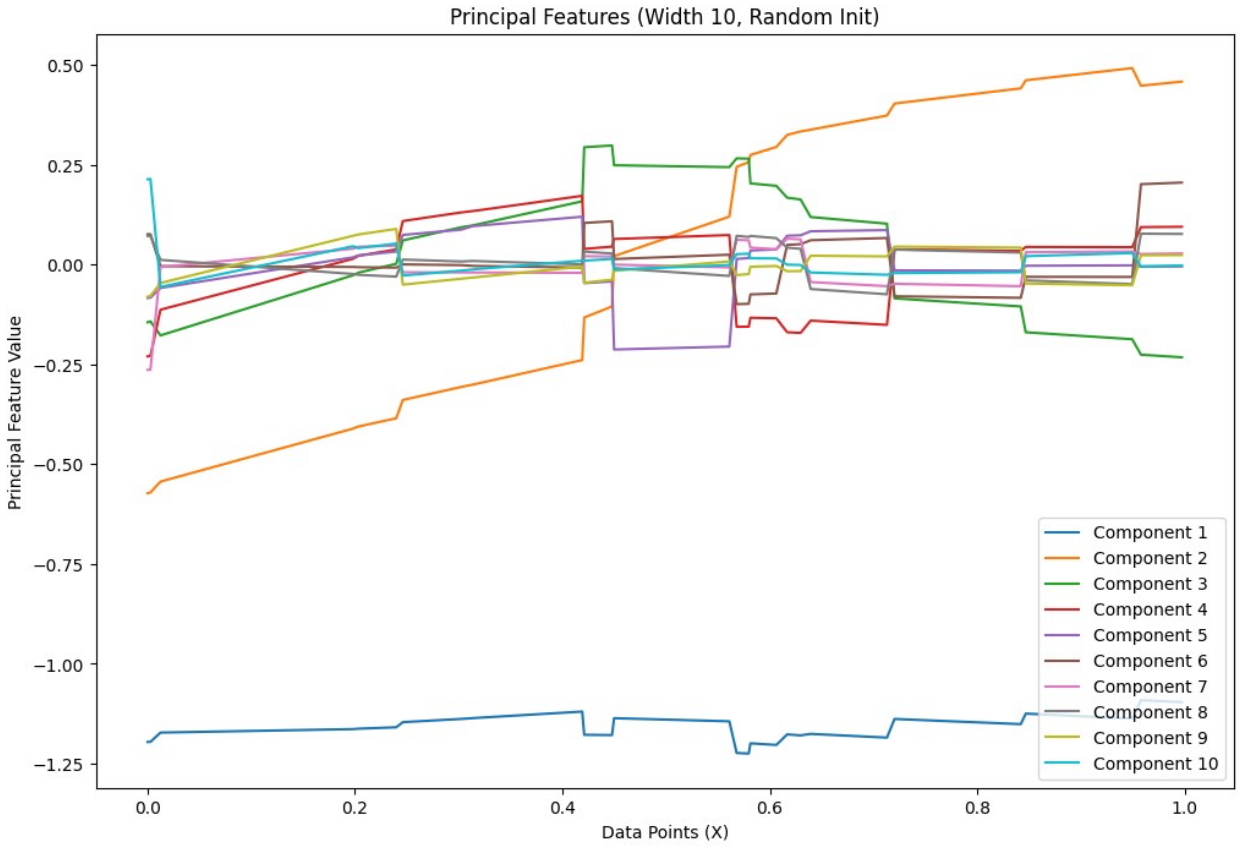

Trained - Layer 2 Weights (Width 40)
Gradient Values
Data Points (X)
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
0.0
0.2
0.4
0.6
0.8
1.0


Random Init - Layer 2 Bias (Width 40)
Gradient Values
Data Points (X)
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
0.0
0.2
0.4
0.6
0.8
1.0

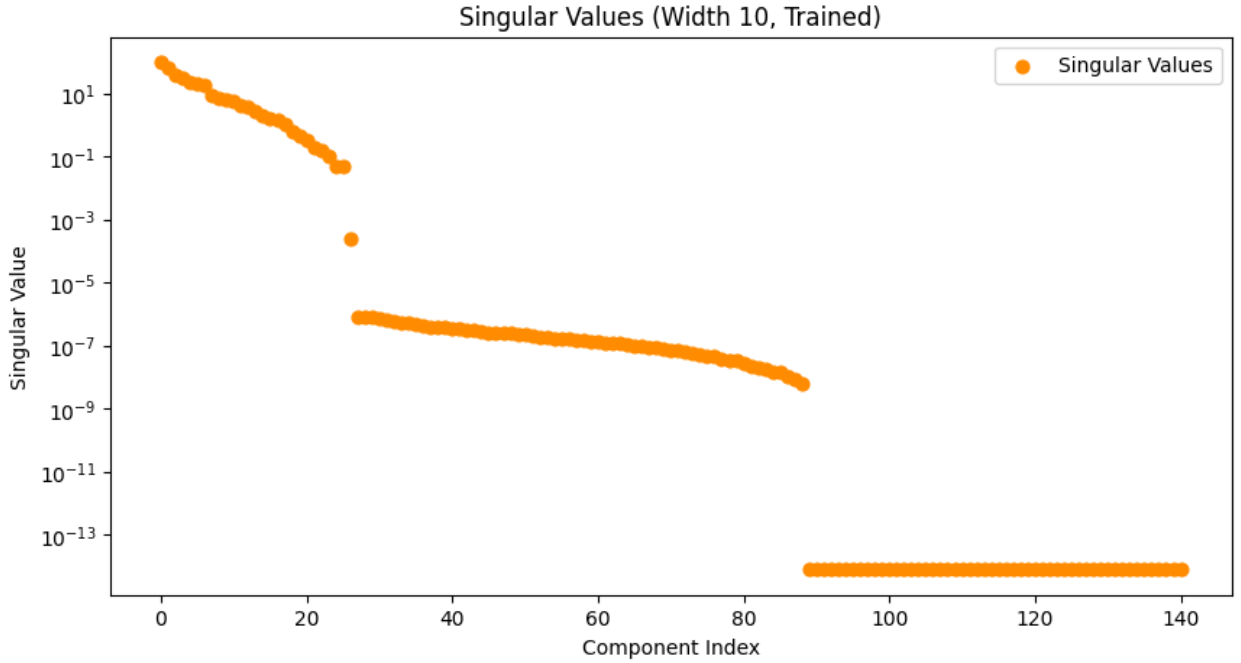


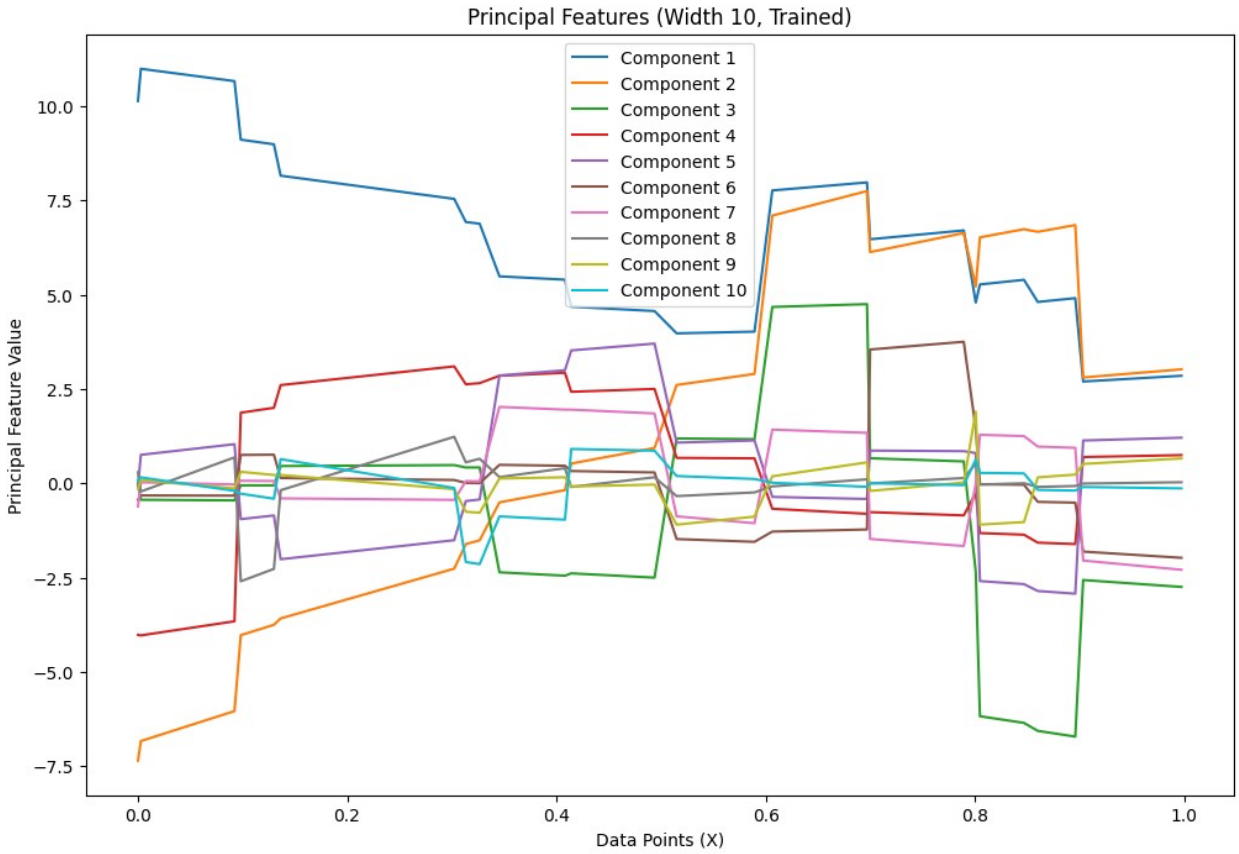

```
=== Two-Layer Network Width 10 - Random Initialization ===
```



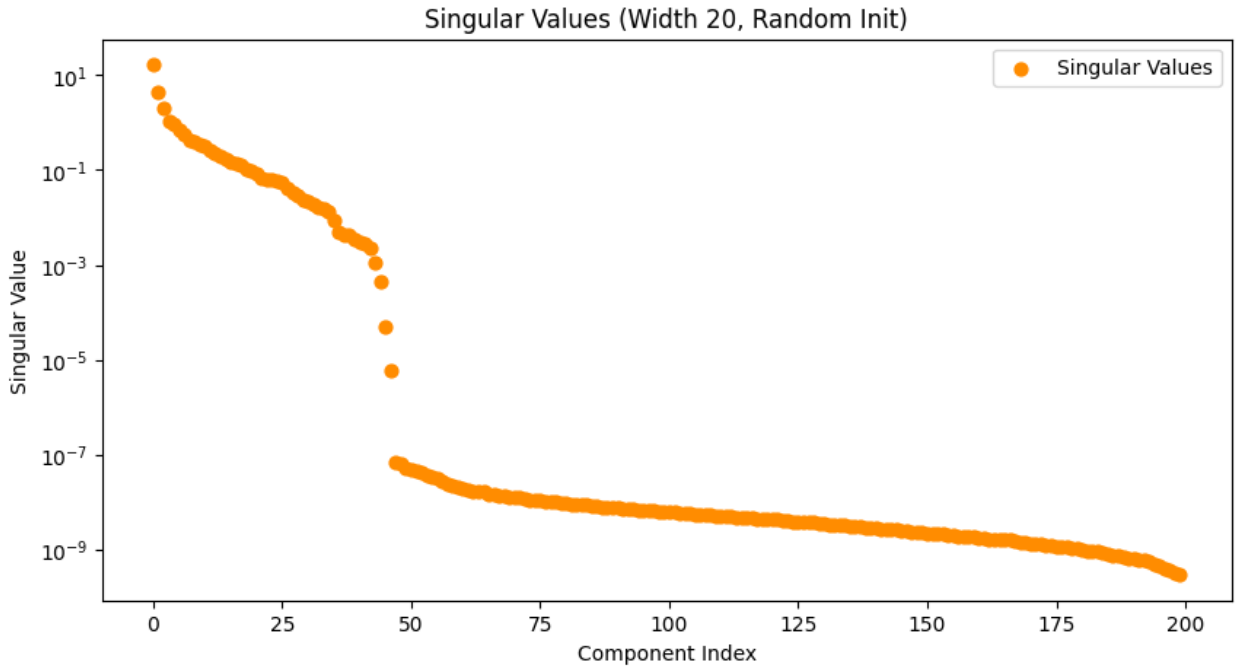


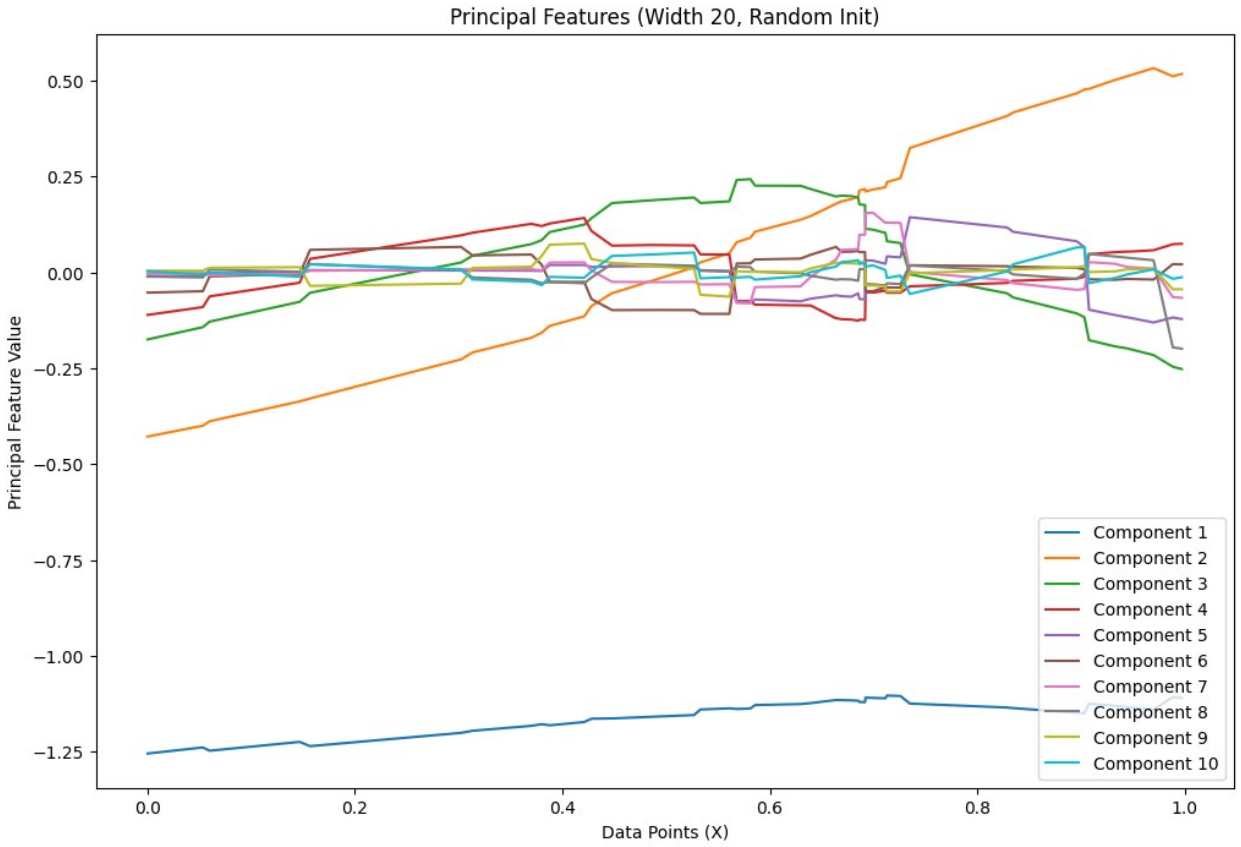

```
=== Two-Layer Network Width 10 - After Training ===
```



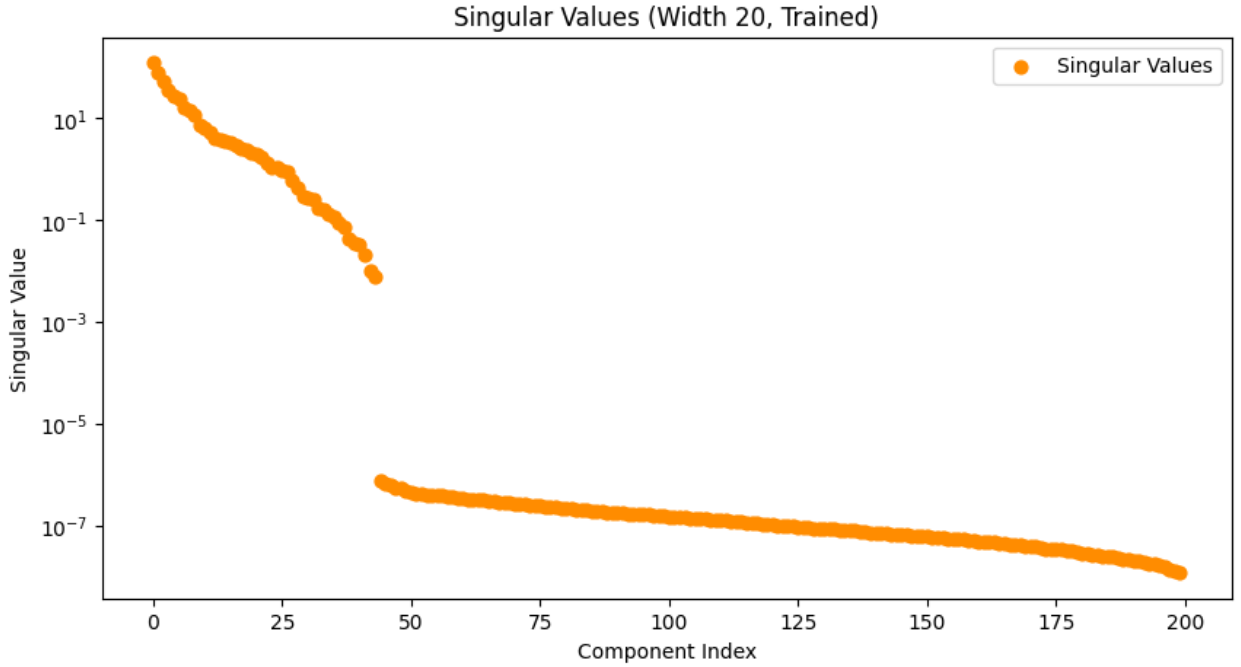


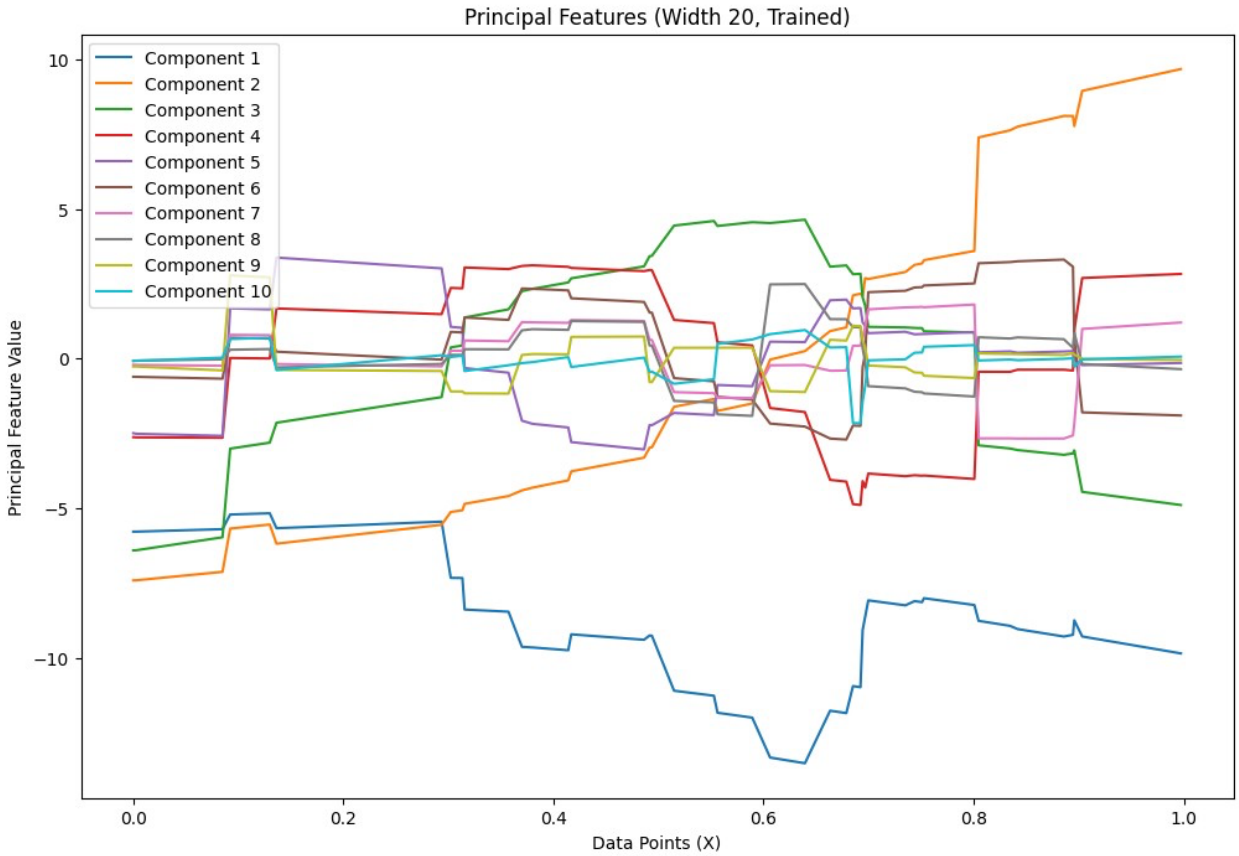

```
=== Two-Layer Network Width 20 - Random Initialization ===
```



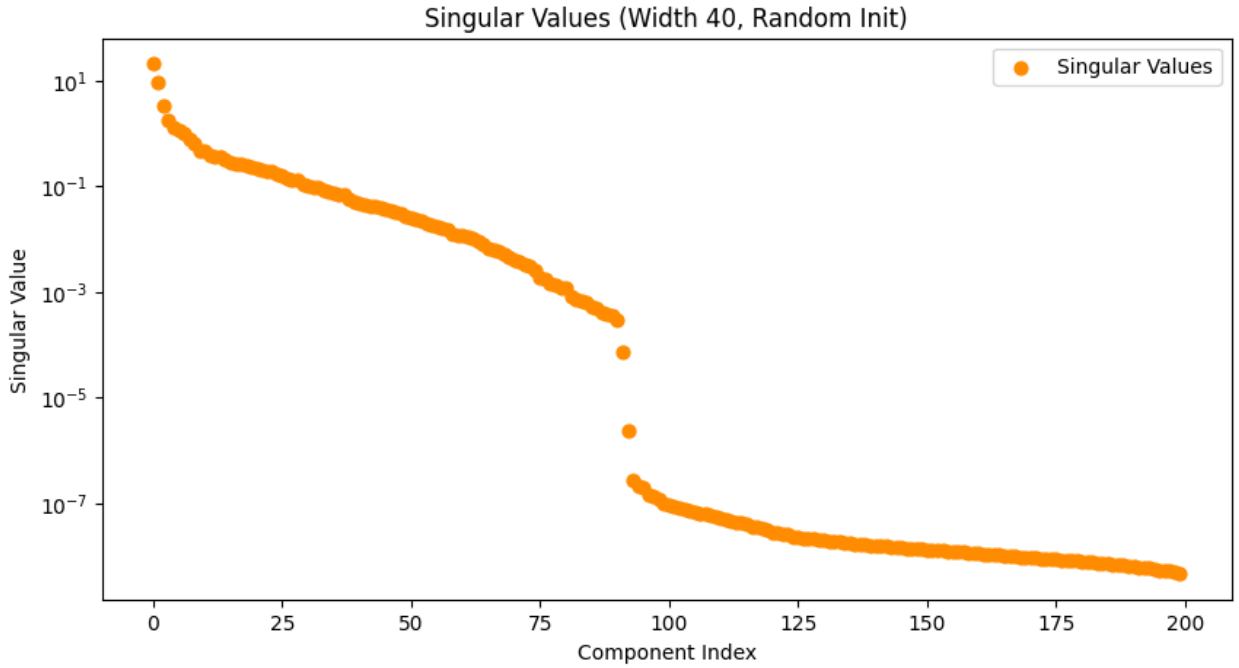


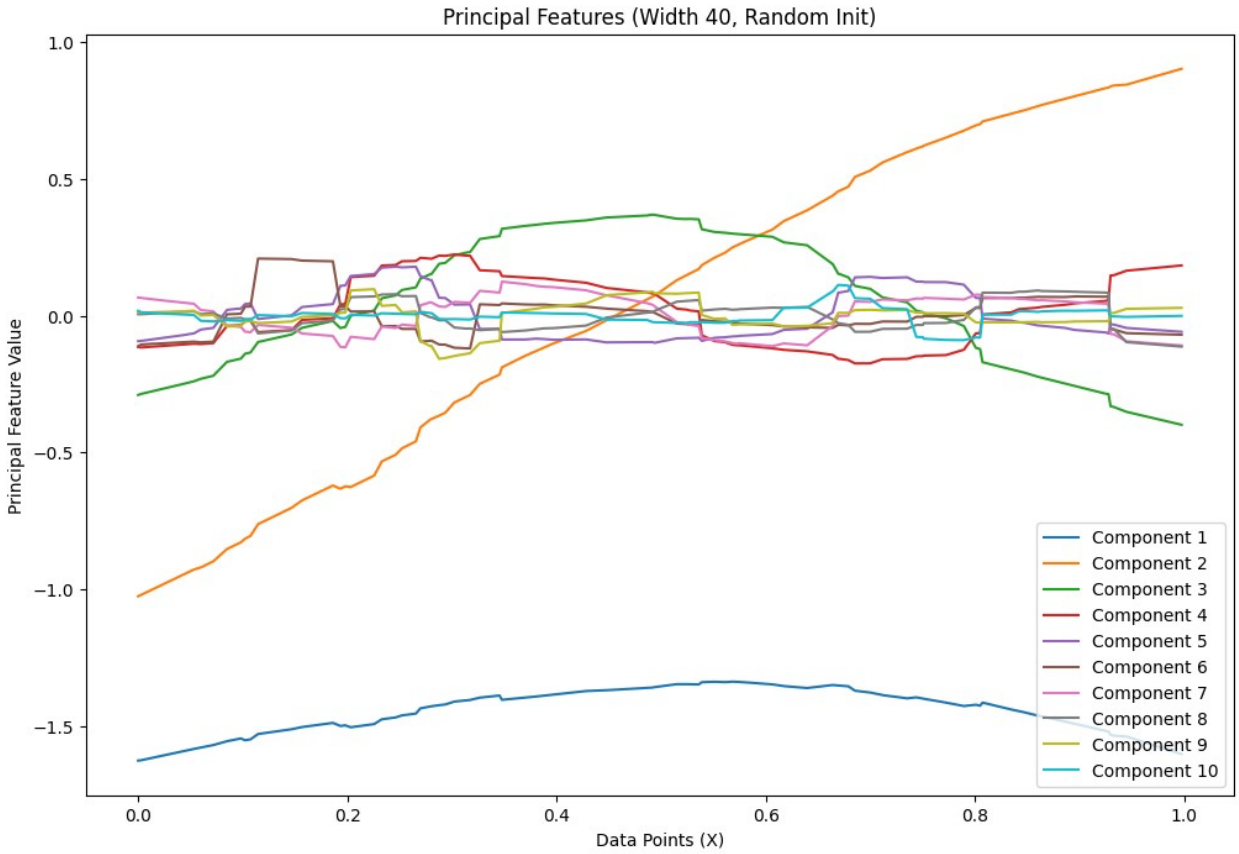

```
=== Two-Layer Network Width 20 - After Training ===
```



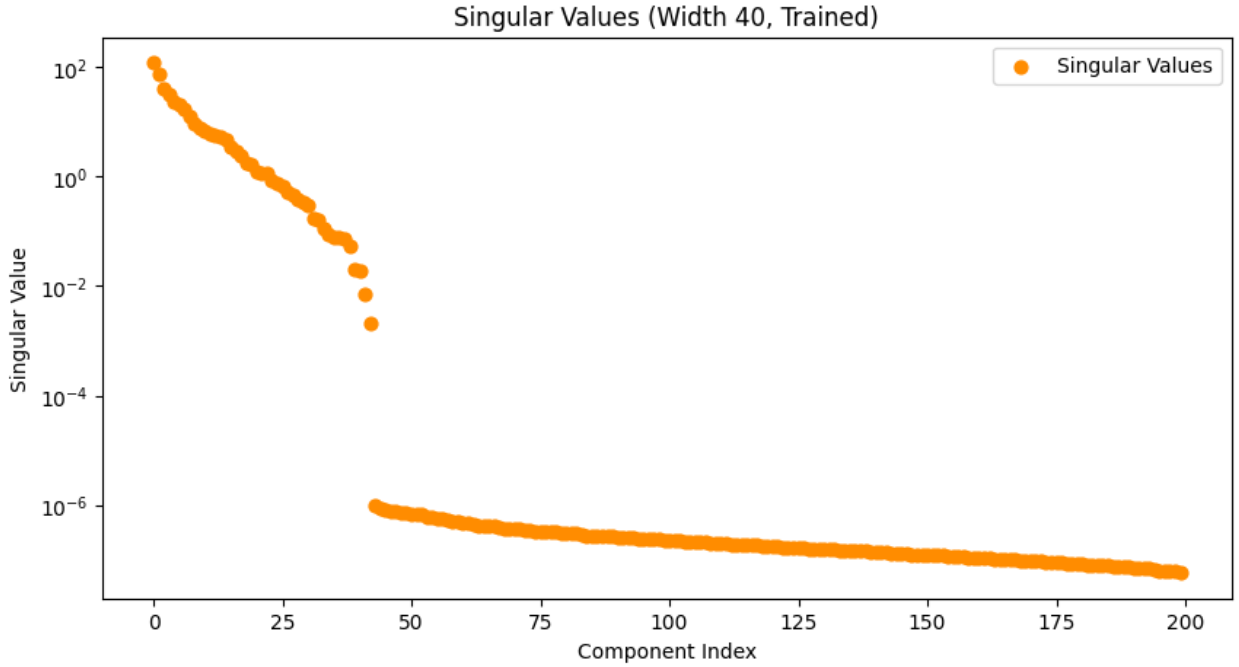


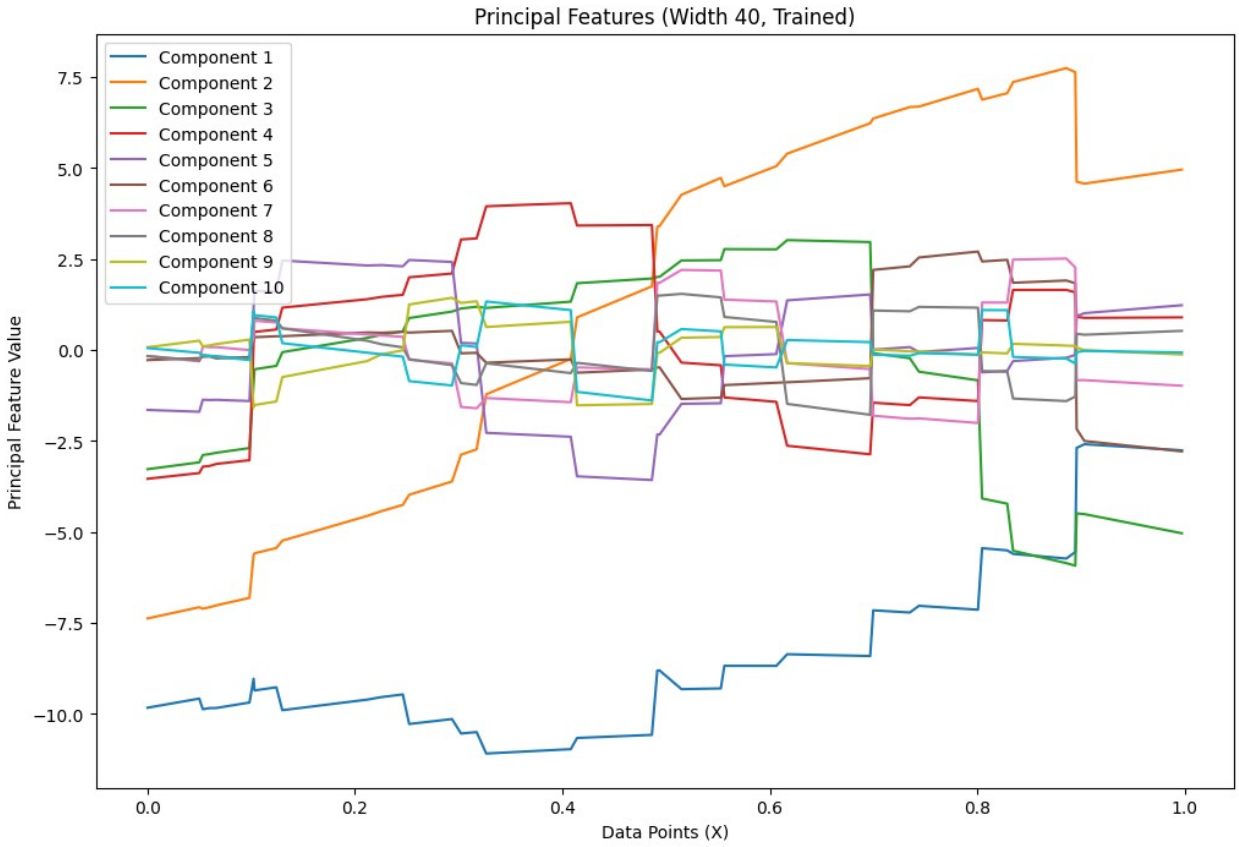

```
=== Two-Layer Network Width 40 - Random Initialization ===
```






```
=== Two-Layer Network Width 40 - After Training ===
```




Principal Features (Width 40, Trained)
Component 1
Component 2
Component 3
Component 4
Component 5
Component 6
Component 7
Component 8
Component 9
Component 10
Principal Feature Value
Data Points (X)
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
0.0
0.2
0.4
0.6
0.8
1.0

5. Analyzing Distributed Training

Step 1: Analyze All-to-All Communication

- Each of the n workers computes gradients on its data subset, resulting in a gradient vector of size p.
- To update its weights, each worker must receive gradients from all other workers.
- Each worker sends its gradient vector (size p) to every other worker.
- Since there are n workers and each sends to $n-1$ others, the total number of messages is:

$$\text{Number of messages} = n(n-1)$$

- The size of each message is p.

Step 2: Analyze Parameter Server

- There is 1 parameter server and n workers.
- Each worker computes gradients (size p) and sends them to the server: n messages.
- The server updates the model and sends the new parameters (size p) back to each worker: another n messages.
- Total messages: $2n$.
- Each message has size p.

Step 3: Analyze Ring All-Reduce

- n workers are arranged in a ring.
- Each worker computes a gradient vector of size p and splits it into n chunks of size $\frac{p}{n}$.
- The algorithm has two phases:
  - **Reduction phase**: $n-1$ steps, each with n messages.
  - **Broadcast phase**: $n-1$ steps, each with n messages.
- Total messages: $2n(n-1)$.
- Each message has size $\frac{p}{n}$.

Conclusion

Paradigm	Number of Messages Sent	Size of Each Message
All-to-All	$\boxed{n(n-1)}$	p
Parameter Server	$2n$	$\boxed{p}$
Ring All-Reduce	$n(2(n-1))$	$\boxed{\frac{p}{n}}$

Optimization Techniques for "Bad" 😖 Objective Functions

In this homework, we will be learning a 2-parameter model $f(x, y)$ (x and y are the two parameters) using different optimization techniques. Formally, the problem we are trying to solve is

$$\arg\max_{x,y} f(x, y)$$

Our function has the following closed form:

$$f(x, y) = \sin(2.5x - 5y) + \cos(4x + 8y) - (x - 2)^2 - (y - 2)^2.$$

As you might have already guessed, this objective function has a lot of local optima (e.g., when $\nabla f(x, y) = 0$).

In this notebook, you will see how standard gradient descent (ascent here as we are maximizing the function) might run into issues optimizing such a function and learn about a couple of other optimization techniques that overcome the local optima issue.

To get started, let's first write the function in code and visualize it (brighter color = higher value).

```
import numpy as np

def f(x, y):
  return np.sin(2.5 * (x - 2 * y)) + np.cos(4.0 * (x + 2 * y)) - ((x - 2.0) ** 2 + (y - 2.0) ** 2)

import matplotlib
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from ipywidgets import interactive, widgets, Layout, interact

# add contour line
plt.rcParams['contour.negative_linestyle'] = 'solid'

class Visualizer:
  def __init__(self, func, x_lim=(-1, 3), y_lim=(-1, 3), res=100, n_contour_lines=15, quantized_values=None):
    xs = np.linspace(*x_lim, res)
    ys = np.linspace(*y_lim, res)
    self.xs, self.ys = np.meshgrid(xs, ys)
    self.zs = func(self.xs, self.ys)

    index = np.argmax(self.zs.reshape(-1))
    self.opt_x, self.opt_y = self.xs.reshape(-1)[index], self.ys.reshape(-1)[index]
    self.x_lim, self.y_lim = x_lim, y_lim

    self.n_contour_lines = n_contour_lines
    self.quantized_values = quantized_values

  def _plot_base_function(self, ax, no_bg=False, alpha=1.0, new_zs=None, new_opt=None):
    if new_zs is not None:
      zs = new_zs
    else:
      zs = self.zs
    if not no_bg:
      ctf = ax.contourf(self.xs, self.ys, zs, 100, cmap='plasma')
    ax.contour(self.xs, self.ys, zs, self.n_contour_lines, linewidths=1, colors="black", alpha=alpha)

    if self.quantized_values is not None:
      assert not no_bg
      patches = []
      for q in self.quantized_values:
        color = ctf.to_rgba(q)
        patches.append(mpatches.Patch(color=color, label=f'$f_q(x, y)={q}$'))
      legend = plt.legend(handles=patches, loc='best', ncol=3, fontsize="small")
      ax.add_artist(legend)
    else:
      if new_opt is None:
        ax.scatter(self.opt_x, self.opt_y, marker="*", s=50, color="red", label="Global Optimum", zorder=4)
      else:
        ax.scatter(self.opt_x, self.opt_y, marker="*", s=50, color="red", label="Global Optimum (Original)", zorder=4, alpha=0.75)
        ax.scatter(*new_opt, marker="*", s=50, color="darkgreen", label="Global Optimum (Smoothed)", zorder=4, alpha=0.75)

  def draw(self, ax, curves=None, no_bg=False, plot_base=True, base_alpha=1.0,
           new_zs=None, new_opt=None, legend=True):
    ax.clear()
    if plot_base:
```

```
        self._plot_base_function(ax, no_bg=no_bg, alpha=base_alpha, new_zs=new_zs, new_opt=new_opt)
      if curves is not None:
        for label, curve in curves.items():
          xs = curve["x"]
          ys = curve["y"]

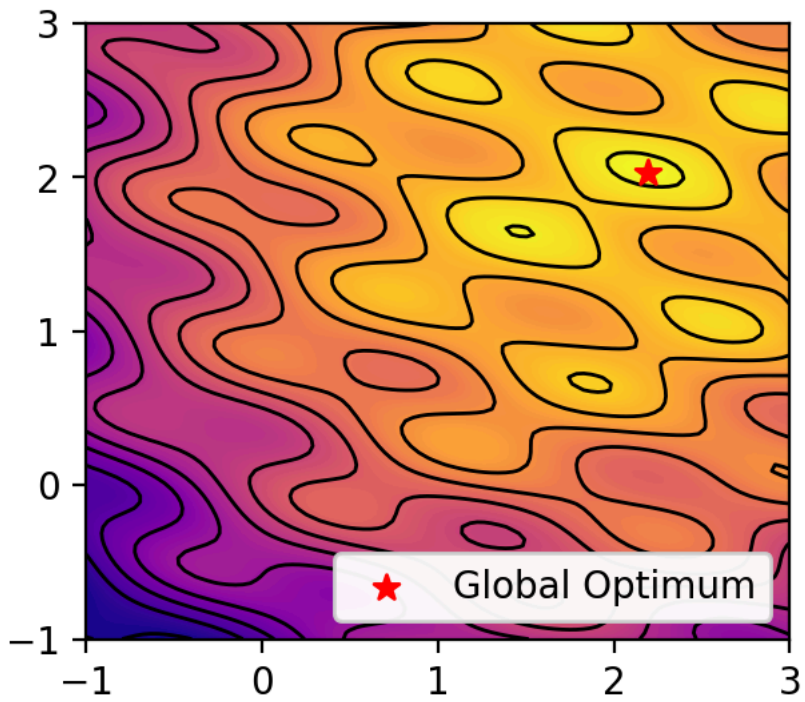
          lines = []
          alphas = np.linspace(0.25, 1., len(xs) - 1)
          for x1, x2, y1, y2 in zip(xs[:-1], xs[1:], ys[:-1], ys[1:]):
              lines.extend([(x1, x2), (y1, y2)])

          ax.plot(*lines, color=curve["color"], linestyle="-", linewidth=2.0, alpha=0.25, zorder=4)
          ax.scatter(xs, ys, marker="o", facecolors='none', edgecolors=curve['color'], s=25, label=label, alpha=0.8, zorder=5)

      ax.set_xticks(np.linspace(-1., 3., 5))
      ax.set_yticks(np.linspace(-1., 3., 5))
      ax.set_xlim(*self.x_lim)
      ax.set_ylim(*self.y_lim)
      if legend:
        handles, labels = ax.get_legend_handles_labels()
        legend = plt.legend(handles=handles, labels=labels, loc='lower right')
        ax.add_artist(legend)


visualizer = Visualizer(f)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3.0), dpi=200)
visualizer.draw(ax)
```



As you may have already noticed, there are a lot of local optima in this function. Would this be an issue for gradient descent method? Let's take a look!

Part 1. Gradient Ascent

In this part, we are going to implement gradient ascent and see how it does on this objective function. Your task is to fill in the `gd_step` function below. The function takes in the parameters x_t, y_t, and the learning rate η and outputs a tuple (x_{t+1}, y_{t+1}) after one gradient step:

$$x_{t+1} \leftarrow x_t + \eta \frac{\partial f}{\partial x}(x_t, y_t),$$
$$y_{t+1} \leftarrow y_t + \eta \frac{\partial f}{\partial x}(x_t, y_t).$$

Your code should be able to take either scalar x and y or numpy arrays of x and y as input (in that case your output would be a tuple of array rather than a tuple of scalars).

Once you are done impelementing the function, run the cell below and see how gradient ascent optimizes this objective function (you will see the trajectory of $\{(x_1, y_1), \cdots, (x_T, y_T)\}$). You should feel free to change the initialization of x, y parameters and see how different initializations change the optimization trajectory (the line where it says `CHANGE ME!` below).

```
def grad_f(x, y):
  dx = 2.5 * np.cos(2.5 * (x - 2 * y)) - 4.0 * np.sin(4.0 * (x + 2 * y)) - 2.0 * (x - 2.0)
  dy = -5.0 * np.cos(2.5 * (x - 2 * y)) - 8.0 * np.sin(4.0 * (x + 2 * y)) - 2.0 * (y - 2.0)
  return dx, dy

num_its = 100

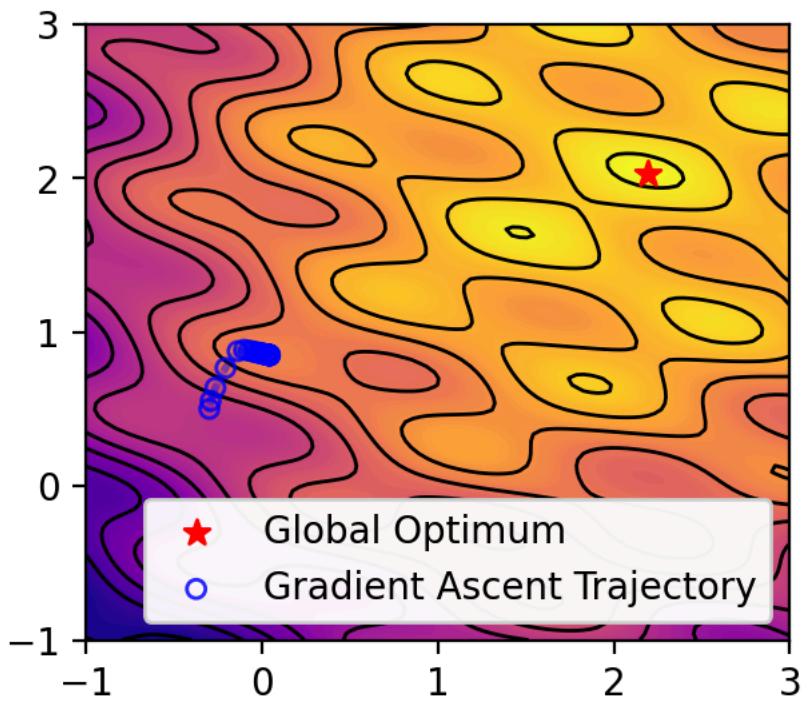
def gd_step(x, y, eta):
  ##############################################################################
  # TODO: Implement gradient ascent step to change your parameters (x, y)     #
  #   in the gradient direction (step size = eta).                            #
  # Hint: You may use the grad_f(x, y) that computes the gradient of f.       #
  ##############################################################################
  dx, dy = grad_f(x, y)
  new_x = x + eta * dx
  new_y = y + eta * dy
  ##############################################################################
  #                             END OF YOUR CODE                               #
  ##############################################################################
  return new_x, new_y

num_its = 30      # number of gradient steps
eta = 0.01        # learning rate
x, y = -0.3, 0.5  # initial parameter values (CHANGE ME!)

# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = gd_step(x, y, eta)
    xs.append(x)
    ys.append(y)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3.0), dpi=200)
visualizer.draw(ax, {f"Gradient Ascent Trajectory": {"color": "blue", "x": xs, "y": ys}})
```



(a). What do you observe from the optimization trajectory of your (x, y) parameters? Where do your parameters converge to? Try a few different initialization values and see how the convergence changes.

The optimization trajectory of the (x, y) parameters using gradient ascent shows that the parameters quickly converge to a nearby local maximum. Because the objective function has many local optima, the specific point of convergence is highly dependent on the initial values of x and y. Trying different initializations demonstrates that the algorithm gets "stuck" in different peaks, rarely finding the global optimum unless it starts very close to it.

In the next part, we are going to cluster these initialization locations by their values at the end of their optimization trajectories.

```
# we put all points that are less than eps apart (Euclidean distance) together.
def manual_clustering(t_xs, t_ys, max_num_clusters=23, eps=1e-2):
  labels = np.zeros_like(t_xs, dtype=int)

  codebook = []
  while True:
    unlabeled_indx, unlabeled_indy = np.nonzero(labels == 0)
    if len(unlabeled_indx) == 0 or len(codebook) > max_num_clusters: break

    first = (unlabeled_indx[0], unlabeled_indy[0])
    codebook.append(np.array([t_xs[first], t_ys[first]]))

    matched = (t_xs[first] - t_xs) ** 2 + (t_ys[first] - t_ys) ** 2 <= eps * eps
    labels[matched] = len(codebook)

  # codebook stores an array of representative point in the cluster
  # labels stores an array of index that represents which cluster each example belongs to
  return np.stack(codebook), labels

# generate a grid of initialization points
t_xs, t_ys = np.linspace(-1, 3, 100), np.linspace(-1, 3, 100)
t_xs, t_ys = np.meshgrid(t_xs, t_ys)
num_its = 1000

# run gradient ascent
t_xs_history, t_ys_history = [t_xs], [t_ys]
for i in range(num_its):
    t_xs, t_ys = gd_step(t_xs, t_ys, 0.01)
    t_xs_history.append(t_xs)
    t_ys_history.append(t_ys)

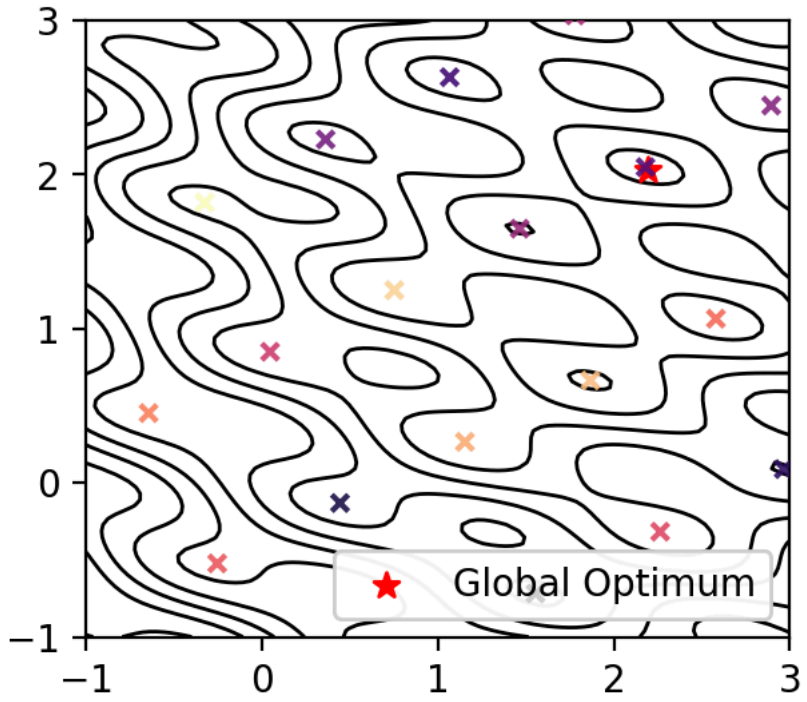
# cluster them based on the final parameter values
codebook, labels = manual_clustering(t_xs, t_ys)
```

Let's take a look at all the local maxima (the "x" symbols) that we obtained from clustering.

```
import random
cmap = matplotlib.colormaps['magma']
colors = [cmap(x) for x in np.linspace(0, 1.0, len(codebook) + 1)]
random.seed(2)
random.shuffle(colors)
colors = np.array(colors)

fig, ax = plt.subplots(1, 1, figsize=(3.4, 3), dpi=200)
visualizer.draw(ax, no_bg=True)
ax.scatter(codebook[:, 0], codebook[:, 1], color=colors[:-1], alpha=0.9, s=20, marker="x", zorder=5)
```

```
<matplotlib.collections.PathCollection at 0x7af9309397c0>
```

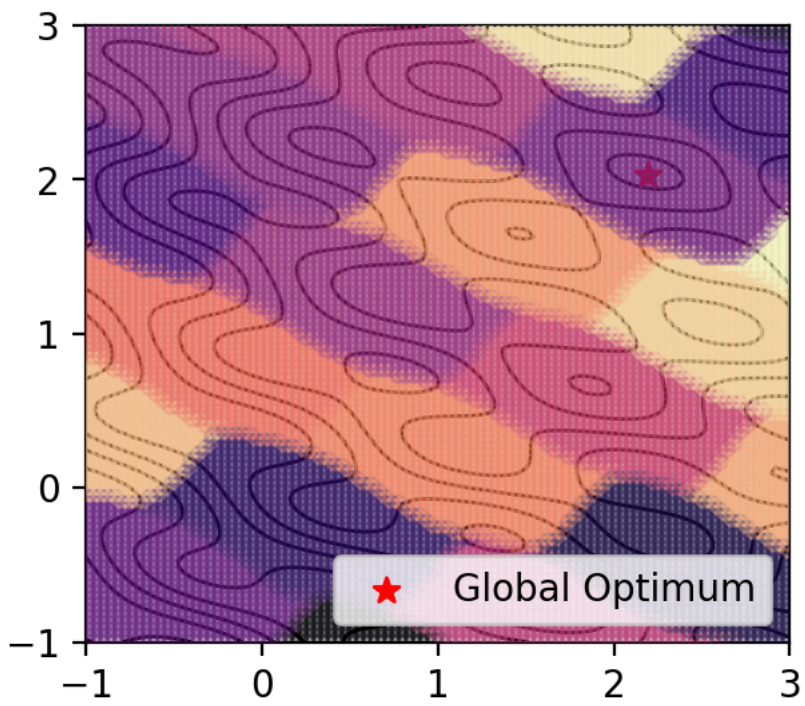


Now, we can visualize the trajectories of these clustered initialization points colored with their corresponding clustering labels via the following interaction widget. The slider corresponds to the gradient step. At step 0, we can see the basins of attraction towards these local optima. Try to drag the slider yourself to see how each initialization moves throughout the optimization process!

```
def update_images(i):
  fig, ax = plt.subplots(1, 1, figsize=(3.4, 3), dpi=200)
  ax.set_facecolor('lightgray')
  visualizer.draw(ax, no_bg=True)
  ax.scatter(t_xs_history[i].reshape(-1), t_ys_history[i].reshape(-1), facecolors='none', edgecolors=colors[labels.reshape(-1)], a
  plt.show()

slider = widgets.IntSlider(
    value=0,
    min=0,
    max=100,
    description="# of gd steps"
)
interact(update_images, i=slider)
```

of gd steps 0



update_images
`def update_images(i)`

<no docstring>

(b) What patterns do you observe from the basins of attraction visualization? How do they relate to the sine and cosine in our objective function?

The basins of attraction form a distinct, grid-like or wavy pattern. These patterns are directly related to the periodic nature of the $\sin(2.5x - 5y)$ and $\cos(4x + 8y)$ terms in the objective function. These trigonometric functions create a landscape of repeating peaks and valleys. Each colored region in the visualization represents a basin, indicating that any initialization point within that region will converge to the same local maximum. The shape and repetition of these basins mirror the periodic behavior of the sine and cosine functions.

Next, we study how the optimization trajectory changes as the learning rate changes. Try to drag the slider to change the learning rate below and see how the trajectories from different initialization points change!

```
import matplotlib
from functools import partial

cmap = matplotlib.colormaps['ocean']
seed = 1437

rng = np.random.default_rng(seed=seed)
num_its = 15
num_inits = 3
init_xs, init_ys = rng.random(num_inits) * 0.9, rng.random(num_inits) * 1.2
colors = [cmap(x) for x in np.linspace(0, 0.6, num_inits)]

def generate_curves(eta, optimizer, num_its, num_inits):
  xs, ys = np.empty((num_its + 1, num_inits)), np.empty((num_its + 1, num_inits))
  xs[0], ys[0] = init_xs, init_ys
  for i in range(1, num_its + 1):
    xs[i], ys[i] = optimizer(xs[i-1], ys[i-1], eta)

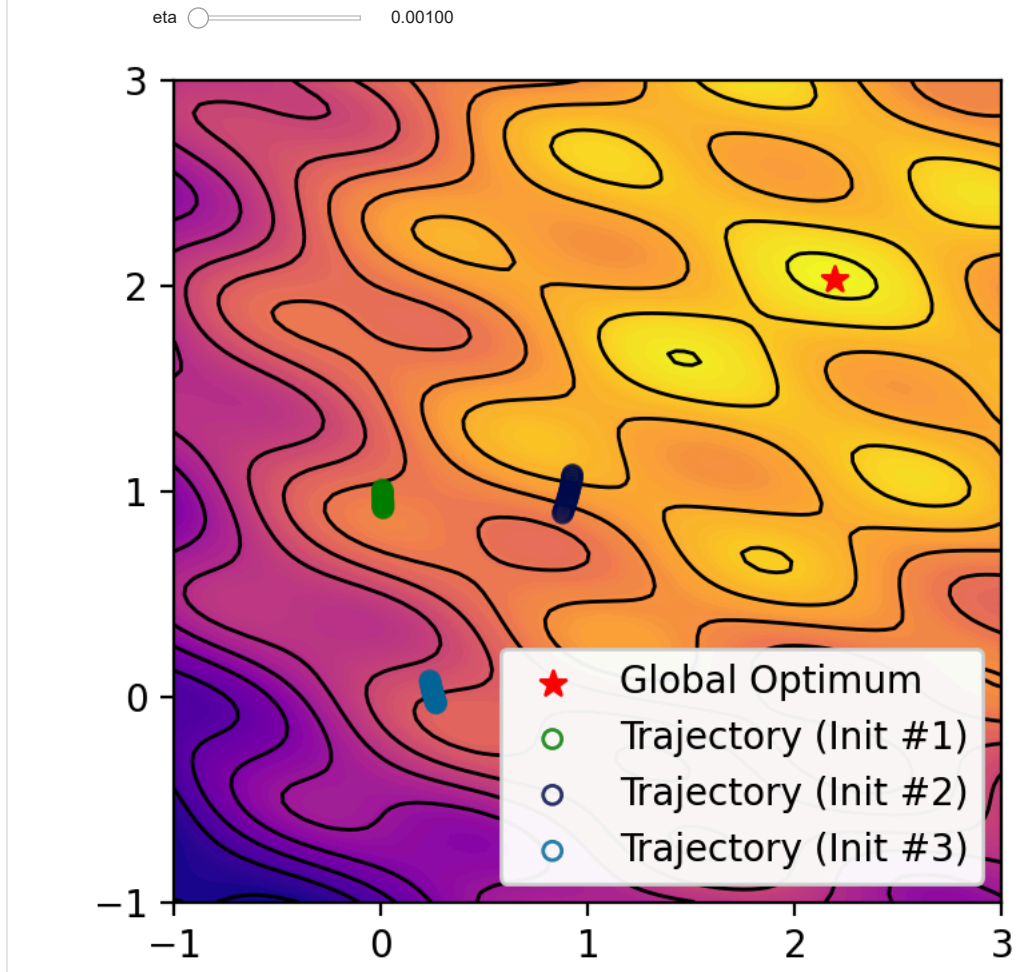
  curves = {}
  for i in range(num_inits):
    curves[f"Trajectory (Init #{i+1})"] = {"color": colors[i], "x": xs[:, i], "y": ys[:, i]}
  return curves
```

```
def get_interactive(visualizer, ax, gen_fn):

  def update_images(eta):
    fig, ax = plt.subplots(1, 1, figsize=(4.0, 4.0), dpi=200)
    curves = gen_fn(eta)
    visualizer.draw(ax, curves)

  slider = widgets.FloatLogSlider(
      value=0.001,
      base=10,
      min=-3, # min exponent of base
      max=-1, # max exponent of base
      step=0.2, # exponent step
      description='eta'
  )
  return interactive(update_images, eta=slider)

ip = get_interactive(visualizer, ax, partial(generate_curves, optimizer=gd_step, num_its=num_its, num_inits=num_inits))
ip
```



(c) What do you notice when the learning rate η changes? Is there a good learning rate that makes it more likely to find the global optimum?

When the learning rate η changes, the optimization trajectory is affected as follows:

- **Low** η: The steps are small, and convergence is slow but stable. The algorithm will reliably find the nearest local optimum.
- **High** η: The steps are large, which can cause the optimizer to overshoot the peak and oscillate or even diverge. However, a moderately high learning rate can sometimes help "jump over" small, insignificant local optima and find a better one.

There is no single "good" learning rate that guarantees finding the global optimum. While a slightly larger η might make it more likely, it's not a reliable strategy.

Other ways to modify the gradient descent algorithm include:

- **Momentum**: This helps the optimizer to continue moving in a consistent direction, potentially "rolling through" minor local optima.
- **Adaptive Learning Rates**: Algorithms like Adam or RMSprop adjust the learning rate for each parameter, which can help navigate complex landscapes more effectively.
- **Stochasticity**: Introducing noise, as seen in techniques like Simulated Annealing or the smoothing methods explored later in the notebook, can help escape local optima.

Part 2: Smoothing the optimzation problem

We have seen that it is very difficult for gradient ascent to find the global optimum relibly unless the initialization is close to the global optimum. This is due to all the small "hills" and "puddles" that trap the optimization trajectories. Can we optimize a different problem that has a similar global optimum but a much nicer objective landscape?

In this part, we explore a smoothing technique that converts the original objective function into a more smoothed one. In particular, we convolve a 2-D Gaussian kernel over the function (convolution, Gaussian kernel):

$$f_\sigma(x, y) = \mathbb{E}_{\tilde{x}\sim\mathcal{N}(x,\sigma^2),\tilde{y}\sim\mathcal{N}(y,\sigma^2)}\left[f(\tilde{x}, \tilde{y})\right]$$

We can view this convolution operation as a continous interpretation of the convolution layer (e.g., as the image approaches inifinite resolution).

Let's first try to visualize the smoothed objective. Since we can't exactly compute the expectation, we are going to approximate it with a Monte-Carlo estimate:

$$\hat{f}_\sigma(x, y) = \frac{1}{N}\sum_i \left[f(\tilde{x}_i, \tilde{y}_i)\right]$$

with $\tilde{x}_1, \tilde{x}_2, \cdots, \tilde{x}_N$ being independent samples from $\mathcal{N}(x, \sigma^2)$ and $\tilde{y}_1, \tilde{y}_2, \cdots, \tilde{y}_N$ being independent samples from $\mathcal{N}(y, \sigma^2)$.

Your task is to implement the `smoothed_f` function below that returns a Monte-Carlo estimate of f_σ using parameters σ and N. The function should return a float scalar.

```
def smoothed_f(x, y, sigma, N):
  ################################################################################
  # TODO: Implement the Monte-Carlo estimate of the smoothed function f using #
  #   N samples.                                                                 #
  ################################################################################
  tx = np.random.normal(x, sigma, N)
  ty = np.random.normal(y, sigma, N)
  return np.mean(f(tx, ty))
  ################################################################################
  #                             END OF YOUR CODE                                 #
  ################################################################################
```

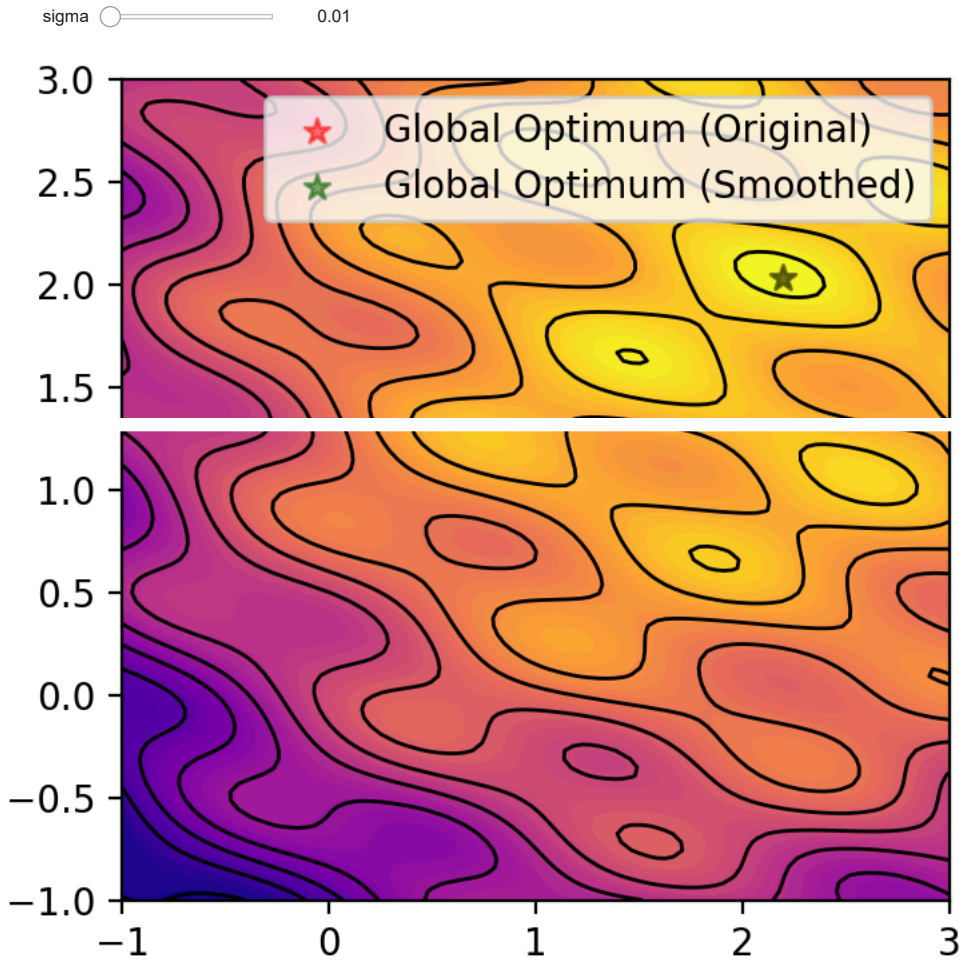
```
zs_dict = {}
opt_dict = {}
for sigma in [0.01, 0.1, 0.2, 0.3, 0.5]:
  zs = []
  xs, ys = visualizer.xs, visualizer.ys
  for x, y in zip(xs.reshape(-1), ys.reshape(-1)):
    zs.append(smoothed_f(x, y, sigma, N=10000))
  zs_dict[sigma] = np.array(zs).reshape(xs.shape)
  index = np.argmax(zs_dict[sigma].reshape(-1))
  opt_dict[sigma] = (xs.reshape(-1)[index], ys.reshape(-1)[index])
```

```
def update_images(sigma):
  fig, ax = plt.subplots(1, 1, figsize=(4.0, 4.0), dpi=200)
  xs, ys = visualizer.xs, visualizer.ys
  zs = zs_dict[sigma]
  ax.contourf(xs, ys, zs, 50, cmap='plasma')
  ax.contour(xs, ys, zs, 15, linewidths=1, colors="black")

  ax.scatter(visualizer.opt_x, visualizer.opt_y, marker="*", s=50, color="red", label="Global Optimum (Original)", alpha=0.5, zorc
  ax.scatter(*opt_dict[sigma], marker="*", s=50, color="darkgreen", label="Global Optimum (Smoothed)", alpha=0.5, zorder=4)
  ax.legend()

slider = widgets.SelectionSlider(
    options=[0.01, 0.1, 0.2, 0.3, 0.5],
    value=0.01,
    description='sigma',
    disabled=False,
    continuous_update=False,
```

```
        orientation='horizontal',
        readout=True
    )
    interactive(update_images, sigma=slider)
```



(d) Take a close look at the global optimum for the original function (red star) and the global optimum for the smoothed function (blue star). What do you observe as σ increases? Why? Can you give an example where the global optimum of the smoothed function is very far from the global optimum of the original function?

As σ (sigma) increases, the smoothed function becomes progressively flatter, and the global optimum of the smoothed function (blue star) tends to move away from the global optimum of the original function (red star).

This happens because smoothing is an averaging process. A larger σ means we are averaging the function's values over a wider area. Sharp, narrow peaks in the original function get averaged with the lower surrounding values, diminishing their height. Consequently, a broader, slightly lower region might have a higher average value, causing the global optimum of the smoothed function to shift towards the center of that broader region.

Example: Imagine a function with a very tall, thin spike at $x = 10$ (the global optimum) and a wide, flat plateau with a slightly lower height centered at $x = 0$. When smoothed with a large σ, the tall spike's value will be averaged down significantly with the surrounding low values. The plateau's value, however, will remain relatively high since it's already averaged over a wide area. The global optimum of the smoothed function could therefore shift to $x = 0$, far from the original optimum at $x = 10$.

Our smoothed function successfully removes all the local optimum except the global optimum. However, we don't know how to optimize this smoothed function efficiently yet. The key challenge here is that we actually don't have a good idea on how to differentiate through f_σ or approximate ∇f_σ.

In the following part, we will be exploring three different optimization techniques to address this issue. We are going to use $\sigma = 0.3$

Part 3a: Finite-difference method

The first method that we are going to explore is the finite-difference method where we are going to approximate the gradient by trying to step in each dimension of θ and evaluate the difference in the objective function:

$$\frac{\partial f_\sigma(x,y)}{\partial x} \approx \frac{\hat{f}_\sigma(x+\delta,y) - \hat{f}_\sigma(x,y)}{\delta}$$
$$\frac{\partial f_\sigma(x,y)}{\partial y} \approx \frac{\hat{f}_\sigma(x,y+\delta) - \hat{f}_\sigma(x,y)}{\delta}$$

where $\hat{f}(x,y)$ is the Monte-Carlo estimate of $f(x,y)$ using random samples of x and y from the Gaussian distribution (same as the $\hat{f}$ in the previous part).

Your task is to implement the `finite_diff_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_σ using the finite-difference method using δ, σ, and N. Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def finite_diff_grad_step(x, y, delta, N, eta, sigma):
  tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
  ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples
  ##########################################################################
  # TODO: Implement the finite-difference gradient estimator of the smoothed  #
  #   function f.                                                          #
  ##########################################################################
  # Compute hat_f(x, y)
  tx = np.random.normal(x, sigma, N)
  ty = np.random.normal(y, sigma, N)
  hat_f_xy = np.mean(f(tx, ty))

  # Compute hat_f(x + delta, y)
  tx_plus = np.random.normal(x + delta, sigma, N)
  ty_plus = np.random.normal(y, sigma, N)
  hat_f_xplus = np.mean(f(tx_plus, ty_plus))

  # Compute hat_f(x, y + delta)
  tx_yplus = np.random.normal(x, sigma, N)
  ty_yplus = np.random.normal(y + delta, sigma, N)
  hat_f_yplus = np.mean(f(tx_yplus, ty_yplus))

  approx_dx = (hat_f_xplus - hat_f_xy) / delta
  approx_dy = (hat_f_yplus - hat_f_xy) / delta

  new_x = x + eta * approx_dx
  new_y = y + eta * approx_dy
  ##########################################################################
  #                             END OF YOUR CODE                           #
  ##########################################################################
  return new_x, new_y

num_its = 30      # number of gradient steps
eta = 0.1         # learning rate
delta = 0.25      # finite difference magnitude
N = 20            # number of samples for MC estimation
sigma = 0.3       # the magnitude of the Gaussian noise.
x, y = -0.3, 0.5  # initial parameter values

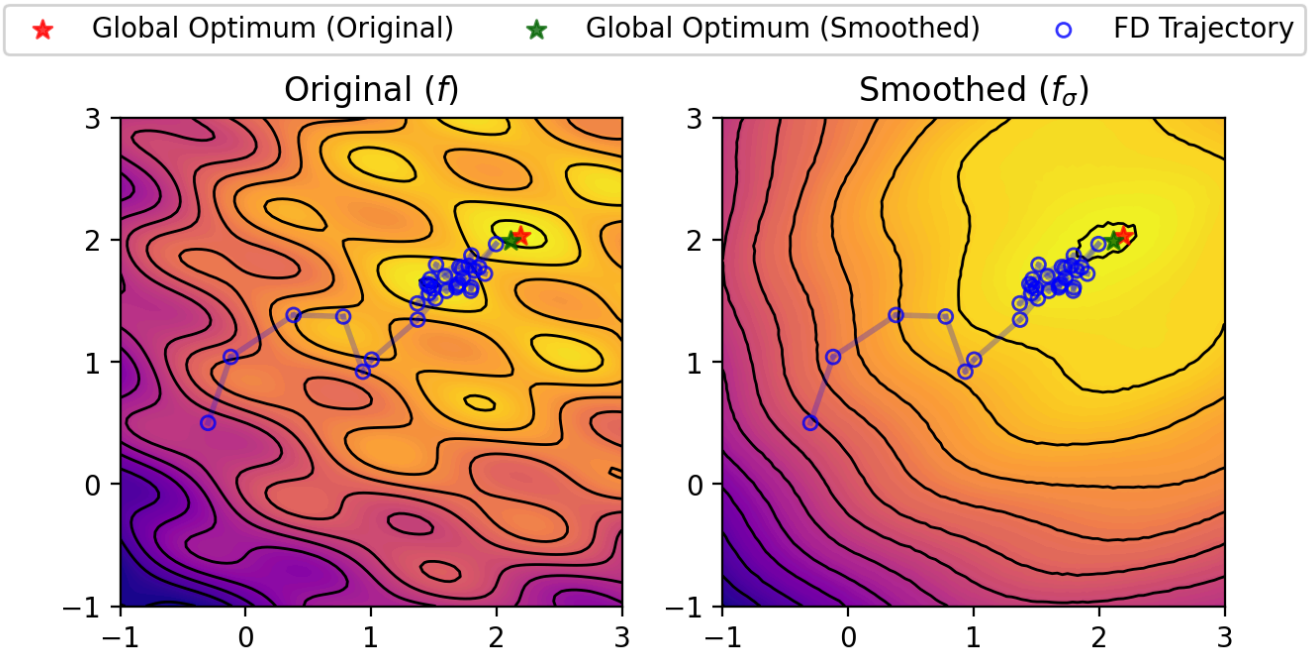
# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = finite_diff_grad_step(x, y, delta, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"FD Trajectory": {"color": "blue", "x": xs, "y": ys}}, new_opt=opt_dict[sigma], legend=False)
visualizer.draw(ax2, {f"FD Trajectory": {"color": "blue", "x": xs, "y": ys}}, new_zs=zs_dict[sigma], new_opt=opt_dict[sigma], lege

ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_\sigma$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.08))
```

```
<>:54: SyntaxWarning: invalid escape sequence '\s'
<>:54: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-4173301894.py:54: SyntaxWarning: invalid escape sequence '\s'
  ax2.set_title("Smoothed ($f_\sigma$)")
<matplotlib.legend.Legend at 0x7af91c790110>
```



Now, we are going to look at how changing η, δ and N influences the learning trajectory. Run the following cell and drag the sliders to try it out yourself!

```
def get_interactive(grad_step_fn, label_name, init_x=-0.3, init_y=0.5, num_its=30, has_delta=False):

  def generate_curves(*args, **kwargs):
    x, y = init_x, init_y
    xs, ys = [init_x], [init_y]
    for i in range(1, num_its + 1):
      x, y = grad_step_fn(x, y, *args, **kwargs)
      xs.append(x)
      ys.append(y)

    return {label_name: {"color": "blue", "x": xs, "y": ys}}

  def update_images(*args, **kwargs):
    sigma=0.3
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
    curves = generate_curves(*args, **kwargs, sigma=sigma)

    visualizer.draw(ax1, curves, new_opt=opt_dict[sigma], legend=False)
    visualizer.draw(ax2, curves, new_zs=zs_dict[sigma], new_opt=opt_dict[sigma], legend=False)

    ax1.set_title("Original ($f$)")
    ax2.set_title("Smoothed ($f_\sigma$)")
    handles, labels = ax2.get_legend_handles_labels()
    fig.legend(handles, labels, loc='upper center', ncol=3, bbox_to_anchor=(0.5, 1.08))
    plt.show()

  eta_slider = widgets.FloatLogSlider(
      value=0.1,
      base=10,
      min=-3, # min exponent of base
```

```
        max=0, # max exponent of base
        step=0.2, # exponent step
        description='eta'
    )

    N_slider = widgets.SelectionSlider(
        options=[1, 2, 5, 10, 20, 50, 100],
        value=20,
        description='N',
        disabled=False,
        continuous_update=False,
        orientation='horizontal',
        readout=True
    )
    if has_delta:
      delta_slider = widgets.FloatLogSlider(
          value=0.3,
          base=10,
          min=-3, # min exponent of base
          max=0, # max exponent of base
          step=0.2, # exponent step
          description='delta'
      )
      return interactive(update_images, N=N_slider, eta=eta_slider, delta=delta_slider)
    return interactive(update_images, N=N_slider, eta=eta_slider)

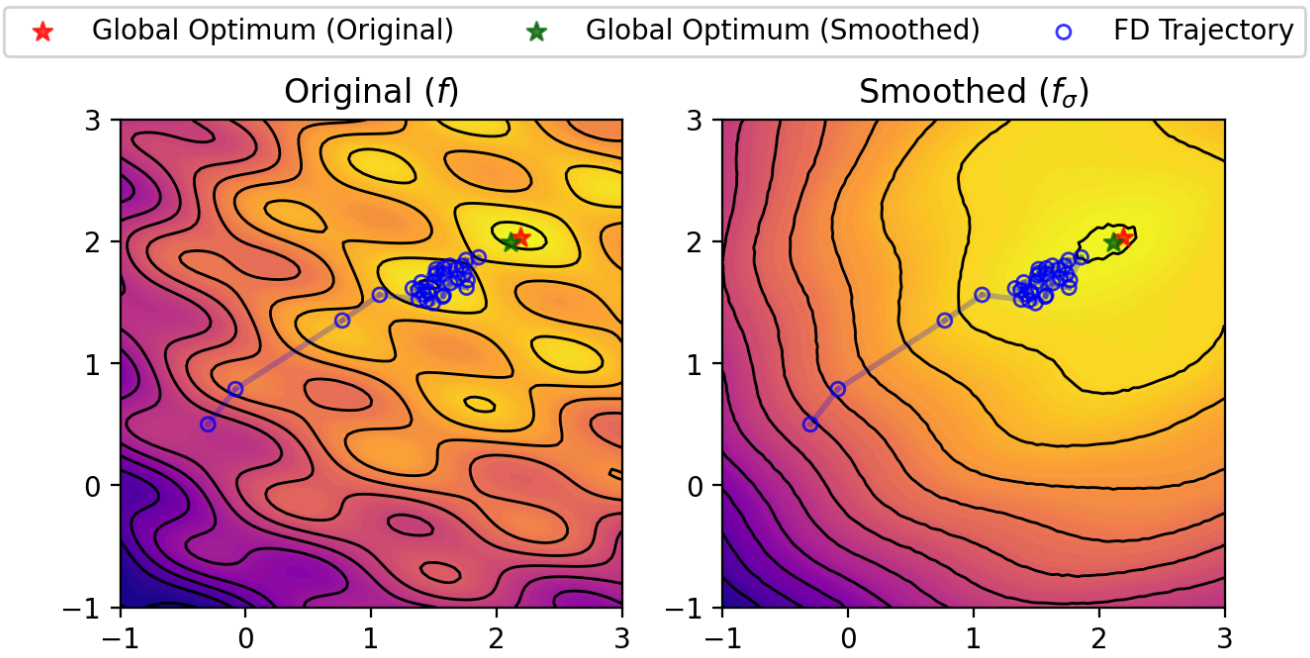
get_interactive(finite_diff_grad_step, "FD Trajectory", has_delta=True)
```

```
<>:22: SyntaxWarning: invalid escape sequence '\s'
<>:22: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-181469160.py:22: SyntaxWarning: invalid escape sequence '\s'
  ax2.set_title("Smoothed ($f_\sigma$)")
```

N 20

eta 0.100

delta 0.300



(e) What do you observe when you change δ, η (learning rate) and N (the sample size for Monte-Carlo estimate)?

- δ **(delta)**: This controls the step size for the finite-difference approximation. If δ is too small, the gradient estimate becomes noisy and unstable, as it relies on the difference between two very close, noisy function evaluations. If δ is too large, the estimate becomes inaccurate because it no longer approximates the true local gradient.
- η **(eta)**: The learning rate has its usual effect. A small η leads to slow but stable convergence. A large η can cause the optimization to overshoot the target and become unstable.
- N: This is the number of samples for the Monte-Carlo estimate. A small N results in a very noisy approximation of the smoothed function and its gradient, leading to a jagged and erratic optimization path. As N increases, the gradient estimate becomes more accurate and stable, resulting in a much smoother and more direct trajectory toward the optimum.

Part 3b: Policy Gradient

Finite-difference method uses zero gradient information about the objective function. Can we come up with a technique that uses more information about our smoothed function?

In this part, you will learn about a technique that leverages the gradient information of the probability density function of the Gaussian kernel to better estimate the gradient of the smoothed function.

To start with, let's first write out the smoothed function in the integral form:

$$f_\sigma(x,y) = \mathbb{E}_{\tilde{x},\tilde{y}\sim\mathcal{N}([x,y]^\top,\sigma^2 I)}\left[f(\tilde{x},\tilde{y})\right] = \int_{\tilde{x}}\int_{\tilde{y}} p(\tilde{x},\tilde{y}|x,y) f(\tilde{x},\tilde{y}) d\tilde{x} d\tilde{y}$$

where $p(\tilde{x},\tilde{y}|x,y) = \frac{1}{2\pi\sigma}\exp\left(-\frac{(\tilde{x}-x)^2+(\tilde{y}-y)^2}{2\sigma^2}\right)$.

Now, we can take the derivative with respect to x:

$$\frac{\partial f_\sigma(x,y)}{\partial x} = \int\int \frac{\partial p(\tilde{x},\tilde{y}|x,y)}{\partial x} f(\tilde{x},\tilde{y}) d\tilde{x} d\tilde{y} = \int\int \frac{1}{2\pi\sigma}\exp\left(-\frac{(\tilde{x}-x)^2+(\tilde{y}-y)^2}{2\sigma^2}\right)\left[(\tilde{x}-x)/\sigma^2\right] f(\tilde{x},\tilde{y}) d\tilde{x} d\tilde{y}$$

While it is nice that we can represent it in the integral form, we don't know how to tractably compute it. So how should we proceed from here?

You might have already observed that there is a term in the operand of the integral that is same as $p(\tilde{x},\tilde{y}|x,y)$. This means that we can actually put the result back to the expectation form!

$$\frac{\partial f_\sigma(x,y)}{\partial x} = \int\int p(\tilde{x},\tilde{y}|x,y)(x-\tilde{x}) f(\tilde{x},\tilde{y})/\sigma^2 d\tilde{x} d\tilde{y} = \mathbb{E}\left[(\tilde{x}-x) f(\tilde{x},\tilde{y})/\sigma^2\right]$$

By symmetry, we also have

$$\frac{\partial f_\sigma(x,y)}{\partial y} = \mathbb{E}\left[(\tilde{y}-y) f(\tilde{x},\tilde{y})/\sigma^2\right]$$

With the gradient of the smoothed function in the expectation form, we can use it to approximate the gradient using Monte-Carlo estimation:

$$\frac{\partial f_\sigma(x,y)}{\partial x} \approx \frac{1}{N}\sum_i \left[(\tilde{x}_i-x) f(\tilde{x}_i,\tilde{y}_i)/\sigma^2\right]$$
$$\frac{\partial f_\sigma(x,y)}{\partial y} \approx \frac{1}{N}\sum_i \left[(\tilde{y}_i-y) f(\tilde{x}_i,\tilde{y}_i)/\sigma^2\right]$$

where again $\{\tilde{x}_i\}_{i=1}^N$, $\{\tilde{y}_i\}_{i=1}^N$ are independent samples of $\mathcal{N}(x,\sigma^2)$ and $\mathcal{N}(y,\sigma^2)$ respectively. The gradient estimator of such form is called the *policy gradient estimator* or *score function gradient estimator*.

(f) Can you provide an intuitive explanation of why the gradient estimator works? (Hint: which direction is the gradient estimator trying to push (x,y) into? By what magnitude?)

The policy gradient estimator, $E[(\tilde{x}-x)\cdot f(\tilde{x},\tilde{y})/\sigma^2]$, works by sampling points $(\tilde{x},\tilde{y})$ around the current point (x,y) and pushing (x,y) in the direction of the "good" samples.

- **Direction**: The term $(\tilde{x}-x)$ is a vector pointing from the current location x to a sampled location $\tilde{x}$.
- **Magnitude**: This direction vector is weighted by the function value $f(\tilde{x},\tilde{y})$ at that sampled point.

Intuitively, if samples in a particular direction yield high function values (a "high reward"), they contribute more to the average gradient, pulling the update in that direction. Conversely, samples that yield low or negative function values will pull the update away from their direction. The estimator effectively pushes (x,y) towards regions where the expected function value is higher.

Now comes the question -- What if $p(\tilde{x},\tilde{y}|x,y)$ is not Gaussian? Is there still a nice way to rewrite the gradient of the smoothed function as an expectation of something? You will explore this more in the written part of your homework!

For now, your task is to implement the `pg_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_σ using the policy gradient method using σ, and N. Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def pg_grad_step(x, y, N, eta, sigma):
  tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
  ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples
  ############################################################################
  # TODO: Implement the policy gradient estimator for the smoothed function f #
  ############################################################################
  tx = np.random.normal(x, sigma, size=(N,))
  ty = np.random.normal(y, sigma, size=(N,))
  approx_dx = np.mean(((tx - x) / sigma**2) * f(tx, ty))
  approx_dy = np.mean(((ty - y) / sigma**2) * f(tx, ty))
  new_x = x + eta * approx_dx
  new_y = y + eta * approx_dy
  ############################################################################
  #                             END OF YOUR CODE                             #
  ############################################################################
  return new_x, new_y

num_its = 30      # number of gradient steps
eta = 0.1         # learning rate
N = 20            # number of samples for MC estimation
sigma = 0.3       # the magnitude of the Gaussian noise
x, y = -0.3, 0.5  # initial parameter values

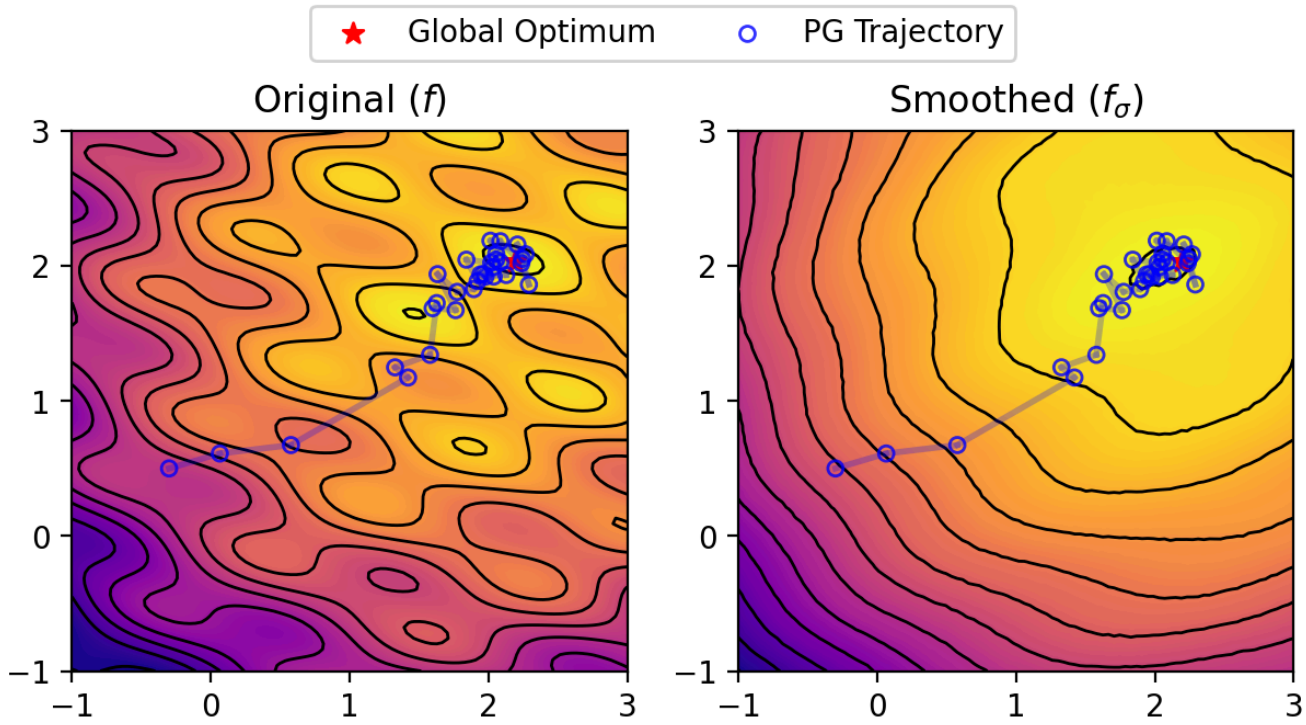
# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = pg_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"PG Trajectory": {"color": "blue", "x": xs, "y": ys}}, legend=False)
visualizer.draw(ax2, {f"PG Trajectory": {"color": "blue", "x": xs, "y": ys}}, new_zs=zs_dict[sigma], legend=False)

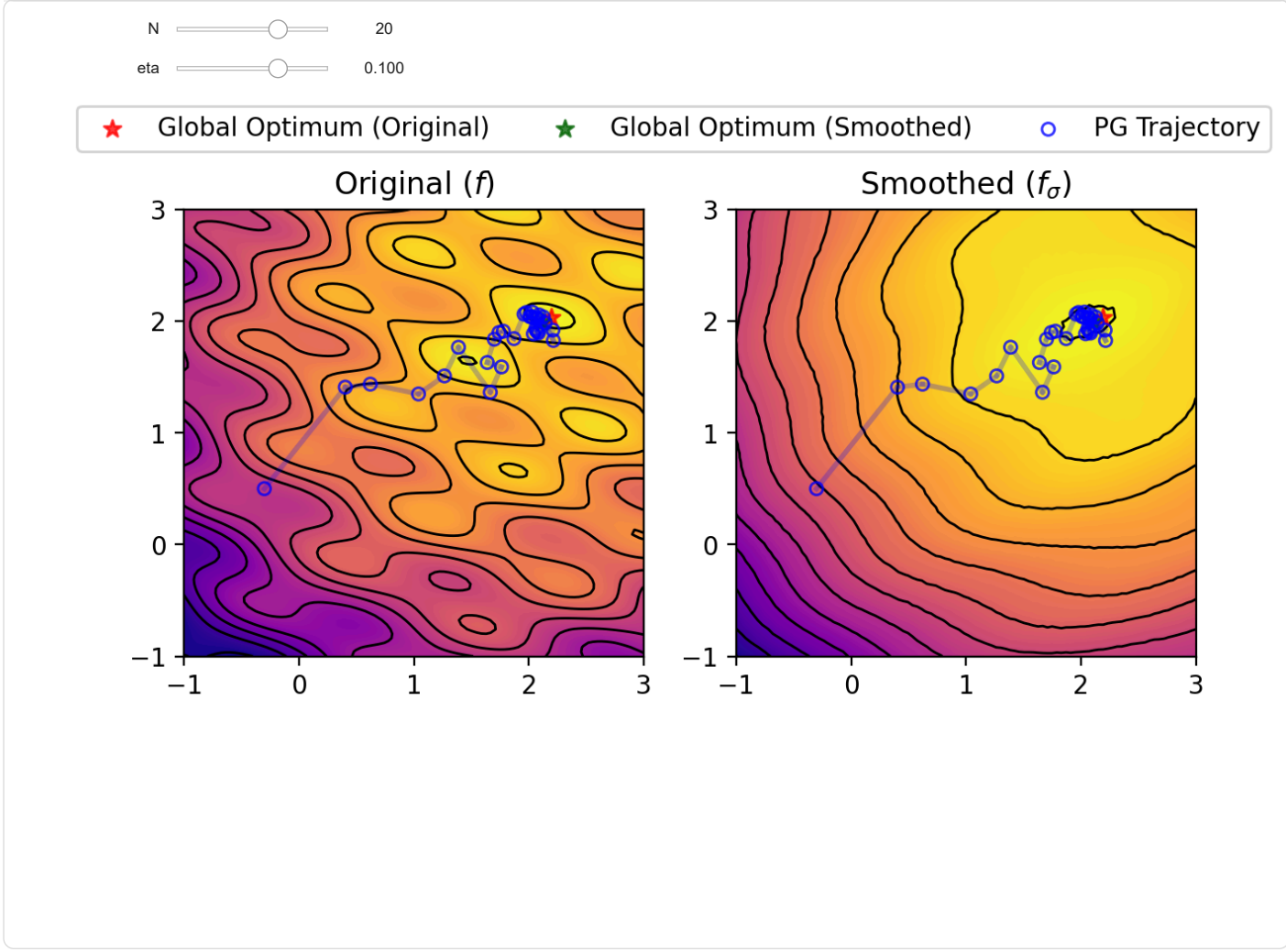
ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_\sigma$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=2, bbox_to_anchor=(0.5, 1.08))
pass
```

```
<>:38: SyntaxWarning: invalid escape sequence '\s'
<>:38: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-623985168.py:38: SyntaxWarning: invalid escape sequence '\s'
  ax2.set_title("Smoothed ($f_\sigma$)")
```



Now, we are going to look at how changing η and N influences the learning trajectory. Run the following cell and drag the sliders to try it out yourself!

```
get_interactive(pg_grad_step, "PG Trajectory", has_delta=False)
```



(g) What do you observe when you change η (learning rate) and N (the sample size for Monte-Carlo estimate)?

Changing η and N for the policy gradient method has a similar effect as with the finite-difference method:

- η: Controls the step size. Too large leads to instability; too small leads to slow convergence.
- N: A larger N reduces the variance of the gradient estimate, leading to a smoother, more stable optimization trajectory. A smaller N results in a noisy, erratic path.

Comparison to Finite-Difference Method: For the same number of samples N, the policy gradient method often produces a more stable and less noisy trajectory than the finite-difference method. This is because the finite-difference method estimates the gradient by subtracting two noisy function evaluations ($f(x+\delta) - f(x)$), which can amplify variance. The policy gradient method, by contrast, directly computes an expected gradient and tends to have lower variance, making it more sample-efficient.

Part 3c: Reparameterization Gradient

Policy gradient uses the information about the smoothing kernel to provide a better gradient estimation. Can we use any gradient information about $f(x)$? This might seem a bit unintuitive, as the function that has lots of local optima probably have poor gradient information locally. As we will show in this part, we can actually harness the poor gradient information to provide better gradient estimation of the smoothed function.

Let's go back to the definition of the smoothed function again (and take the gradient operator with respect to $[x, y]$):

$$\nabla f_\sigma(x, y) = \nabla \mathbb{E}_{\tilde{x},\tilde{y}\sim\mathcal{N}([x,y],\sigma^2 I)}\left[f(\tilde{x},\tilde{y})\right]$$

It would be so nice if we could move the gradient inside the expectation, but that is not possible immediately because the expectation distribution depends on x and y and the term inside the expectation does not directly depend on x and y. Can we somehow rewrite the expectation such that the distribution does not depend on x and y? Or in another word, can we separate out the noise in $\tilde{x}$ and $\tilde{y}$ such that they can be rewritten as a deterministic function of x, y and some noise that is independent of x and y?

Yes! Let z_x, z_y be independent samples from the Gaussian $\mathcal{N}(0, \sigma^2)$. We can simply rewrite $\tilde{x}$ and $\tilde{y}$ as follows:

$$\tilde{x} = z_x + x,$$
$$\tilde{y} = z_y + y.$$

With such a reparameterization, we can now express the smoothed function using an expectation where its distribution is independent of x and y! This would allow us to move the gradient operator inside:

$$\nabla f_\sigma(x, y) = \nabla \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0,\sigma^2)} \left[f(x + z_x, y + z_y) \right] = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0,\sigma^2)} \left[\nabla f(x + z_x, y + z_y) \right]$$

Writing out the derivative for x and y separately:

$$\frac{\partial f_\sigma(x, y)}{\partial x} = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0,\sigma^2)} \left[f_x(x + z_x, y + z_y) \right]$$
$$\frac{\partial f_\sigma(x, y)}{\partial y} = \mathbb{E}_{z_x, z_y \sim \mathcal{N}(0,\sigma^2)} \left[f_y(x + z_x, y + z_y) \right]$$

where f_x is the partial derivative of f with respect to the first input (x) and f_y is the partial derivative of f with respect to the second input (y).

As you might have already guessed it, we can perform Monte-Carlo estimate of each of the derivative by drawing z_x and z_y samples. The gradient estimator of such form is called the *reparameterization gradient estimator*.

Your task is to implement the `reparam_grad_step` function below. In the function, you should first estimate the gradient of the smoothed function f_σ using the reparameterization gradient method using σ, and N. Then, you should use the gradient estimate to take a step with a step size of η and return the new parameter values.

```
def reparam_grad_step(x, y, N, eta, sigma):
  tx = np.random.normal(x, sigma, size=(N,)) # tilde x samples
  ty = np.random.normal(y, sigma, size=(N,)) # tilde y samples
  ##############################################################################
  # TODO: Implement the reparameterization gradient estimator of the smoothed #
  #   function f.                                                              #
  ##############################################################################
  tx = np.random.normal(x, sigma, size=(N,))
  ty = np.random.normal(y, sigma, size=(N,))
  dtx, dty = grad_f(tx, ty)
  approx_dx = np.mean(dtx)
  approx_dy = np.mean(dty)
  new_x = x + eta * approx_dx
  new_y = y + eta * approx_dy
  ##############################################################################
  #                             END OF YOUR CODE                               #
  ##############################################################################
  return new_x, new_y


num_its = 30      # number of gradient steps
eta = 0.1         # learning rate
N = 20            # number of samples for MC estimation
sigma = 0.3       # the magnitude of the Gaussian noise
x, y = -0.3, 0.5  # initial parameter values

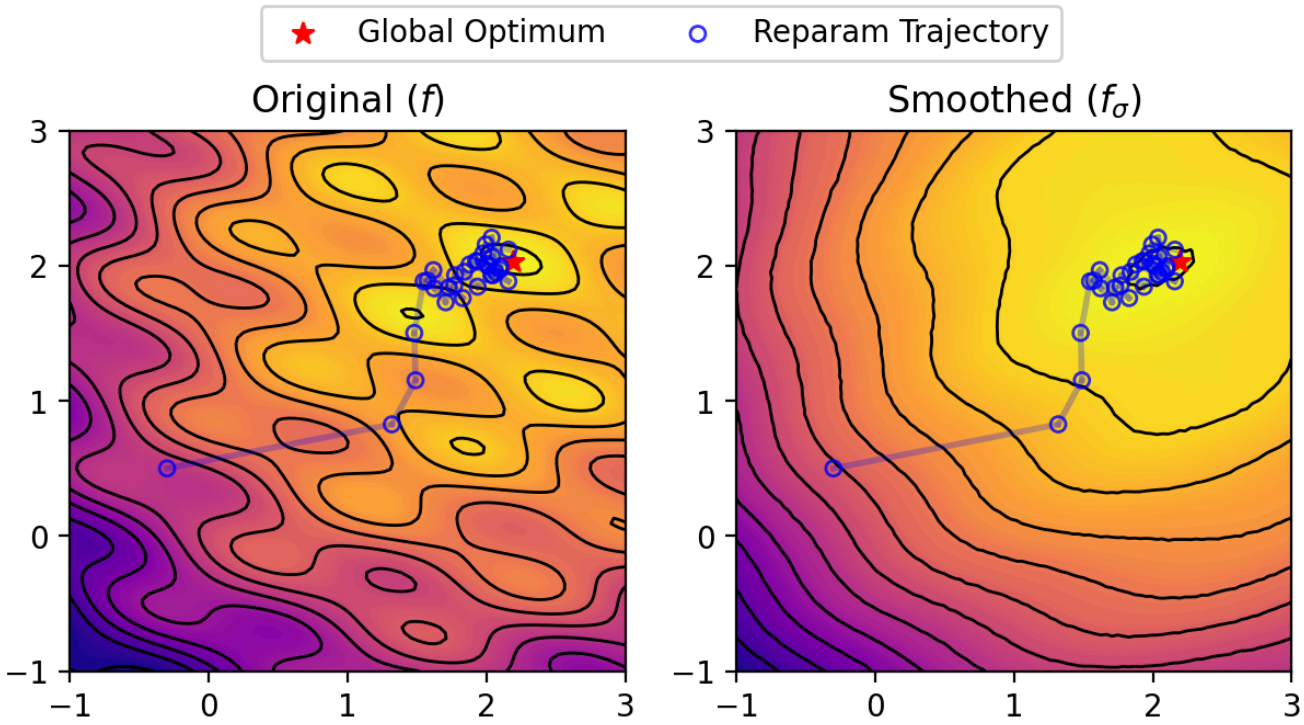
# history of parameter values
xs = [x]
ys = [y]

for i in range(num_its):
    x, y = pg_grad_step(x, y, N, eta, sigma)
    xs.append(x)
    ys.append(y)

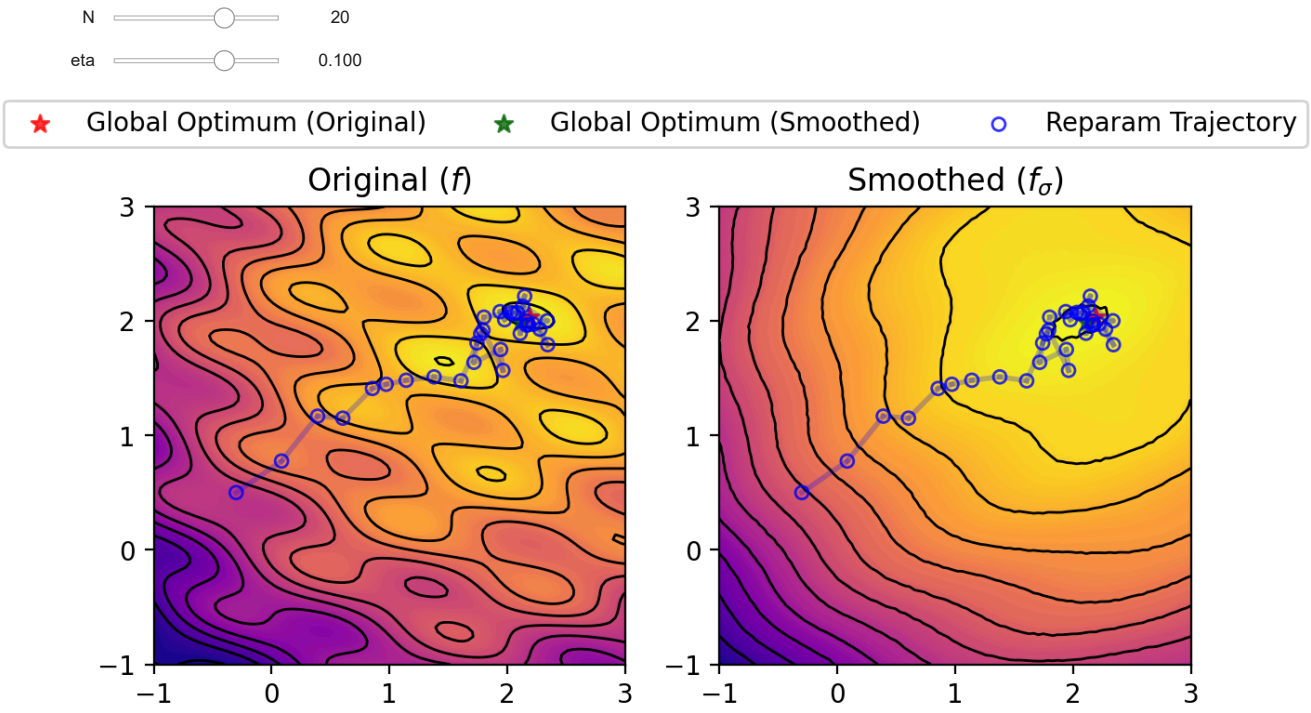
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(7.2, 3.2), dpi=200)
visualizer.draw(ax1, {f"Reparam Trajectory": {"color": "blue", "x": xs, "y": ys}}, legend=False)
visualizer.draw(ax2, {f"Reparam Trajectory": {"color": "blue", "x": xs, "y": ys}}, new_zs=zs_dict[sigma], legend=False)

ax1.set_title("Original ($f$)")
ax2.set_title("Smoothed ($f_\sigma$)")
handles, labels = ax2.get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', ncol=2, bbox_to_anchor=(0.5, 1.08))
pass
```

```
<>:41: SyntaxWarning: invalid escape sequence '\s'
<>:41: SyntaxWarning: invalid escape sequence '\s'
/tmp/ipython-input-1920199948.py:41: SyntaxWarning: invalid escape sequence '\s'
  ax2.set_title("Smoothed ($f_\sigma$)")
```




```
get_interactive(reparam_grad_step, "Reparam Trajectory", has_delta=False)
```



(h) Compare the trajectory of reparam gradient vs. policy gradient. What do you observe? Is one more straight than the other one? Why do you think that is the case?

The trajectory of the reparameterization gradient is significantly straighter and more direct than the trajectory of the policy gradient.

This is because the reparameterization gradient estimator has much lower variance. It leverages the gradient of the original function, ∇f, by averaging the gradients at points sampled around the current parameters ($E[\nabla f(x+z)]$). This directly incorporates valuable information about the function's local slope.

The policy gradient estimator, on the other hand, does not use ∇f. It only uses the function's values ($f(x)$), which is less informative. It essentially performs a weighted search, which is inherently more random and less direct. By using the actual gradient information, the reparameterization trick provides a much more accurate and stable estimate of the smoothed function's gradient, resulting in a cleaner, more direct path to the optimum.

Part 4: Objective functions with discontinuities

In the previous part, we introduced three optimization techniques to find the global optimum of a smoothed function. In this part, we are going to explore a more extreme objective function that contains discontinuities, $f_q(x, y)$. f_q is obtained by quantizing the objective function that we were using in the previous parts. Can we use what we have learned so far to optimize this objective function?

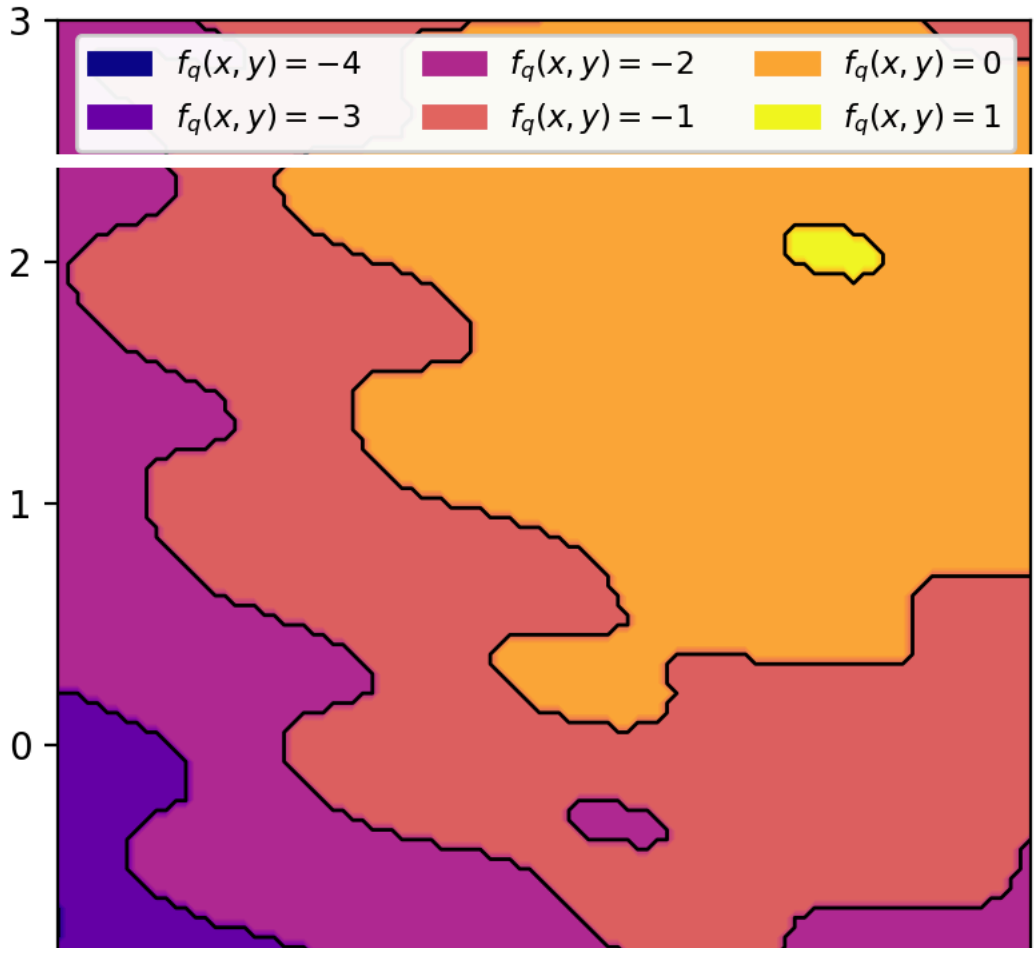
Let's first visualize what the function looks like!

```
def quantize(s):
  return np.floor(s / 4.5 + 0.64)

def qf(x, y):
  return quantize(np.sin(2.5 * (x - 2 * y)) + np.cos(4.0 * (x + 2 * y)) - ((x - 2.0) ** 2 + (y - 2.0) ** 2))

q_visualizer = Visualizer(qf, n_contour_lines=5, quantized_values=[-4, -3, -2, -1, 0, 1])
```

```
fig, ax = plt.subplots(1, 1, figsize=(4.70, 4.70), dpi=200)
```



7. Homework Process & Study Group

(a) Lecture Notes

(b) None

(c) About 10 hours ...